computational-biology-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `computational_biology_ai999x.about()` for the same answer.

Every capability computational-biology-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import computational_biology_ai999x

result = computational_biology_ai999x.solve('bio_reverse_complement', seq='ATGCCGTA')
result.show()
# revcomp = TACGGCAT
```

By topic

topic	capabilities
Alignment / pairwise	Global alignment score (Needleman-Wunsch, linear gaps), Global alignment score with affine gaps (Gotoh), Local alignment score (Smith-Waterman, linear gaps), Local alignment score with affine gaps, Semi-global (end-gap-free) alignment score
Alignment / protein	Global protein alignment score (BLOSUM62, affine gaps), Local protein alignment score (BLOSUM62, affine gaps)
Ecology / biogeography	Species-area relationship fit
Ecology / community comparison	Beta diversity between two samples
Ecology / diversity indices	Shannon diversity index, Simpson diversity indices, Margalef and Menhinick richness indices, Berger-Parker and McIntosh indices, Brillouin diversity index
Ecology / phylogenetic diversity	Faith's phylogenetic diversity
Ecology / population estimation	Mark-recapture population estimate (Petersen MLE), Schumacher-Eschmeyer multiple-census estimate
Ecology / richness estimation	Chao1 species richness estimator, ACE species richness estimator, Rarefied species richness
Ecology / sampling completeness	Good's coverage of a sample
Enzyme kinetics / cooperativity	Hill coefficient from a Hill plot, Hill equation nonlinear least-squares fit
Enzyme kinetics / inhibition	Competitive inhibition global fit
Enzyme kinetics / integrated rate law	Substrate remaining in a Michaelis-Menten reaction, Time to reach a conversion in a Michaelis-Menten reaction
Enzyme kinetics / linearised fits	Lineweaver-Burk fit of Michaelis-Menten data, Hanes-Woolf fit of Michaelis-Menten data, Eadie-Hofstee fit of Michaelis-Menten data
Enzyme kinetics / Michaelis-Menten	Michaelis-Menten nonlinear least-squares fit
Genetics / inbreeding	Inbreeding coefficient from a pedigree
Genetics / linkage mapping	Map distance from a recombination fraction, Recombination fraction from a map distance, Recombination frequency from two-point testcross counts, Three-point testcross: gene order, map distances and interference
Genetics / Mendelian inheritance	Probability of an offspring genotype in a cross, Phenotype ratio of a cross (complete dominance)
Microbial growth / continuous culture	Chemostat steady state (Monod kinetics)
Phylogenetics / distance methods	UPGMA tree heights, Neighbour-joining branch lengths
Phylogenetics / parsimony	Parsimony score of a tree (Fitch)
Phylogenetics / substitution models	Jukes-Cantor distance, Kimura two-parameter distance
Phylogenetics / tree comparison	Robinson-Foulds distance between trees
Phylogenetics / tree space	Number of possible phylogenetic trees
Population biology / age structure	Leslie matrix population projection, Leslie matrix growth rate and stable age distribution
Population genetics / coalescent	Expected coalescence times
Population genetics / drift	Loss of heterozygosity by genetic drift, Fixation probability in the Moran process, Effective population size with unequal sex ratio, Effective size of a fluctuating population

topic	capabilities
Population genetics / Hardy-Weinberg	Hardy-Weinberg chi-square test, Hardy-Weinberg exact test, Hardy-Weinberg genotype and carrier frequencies
Population genetics / linkage disequilibrium	Linkage disequilibrium D, D' and r^2
Population genetics / mutation	Mutation-selection balance
Population genetics / natural selection	Allele frequency under haploid (genic) selection, Decline of a recessive lethal allele, Equilibrium under heterozygote advantage
Population genetics / neutrality tests	Tajima's D
Population genetics / population structure	Fixation index FST from subpopulation allele frequencies
Population genetics / sequence diversity	Nucleotide diversity, Watterson's estimator of theta
Sequences / comparison	P-distance (Hamming) between aligned sequences, Edit (Levenshtein) distance between sequences
Sequences / composition	GC content of a nucleic acid, K-mer counts of a sequence
Sequences / DNA	Reverse complement of a DNA sequence
Sequences / gene expression	Transcription of DNA into mRNA, Translation with the standard genetic code
Sequences / gene finding	Longest open reading frame
Sequences / molecular cloning	Restriction sites and fragments
Sequences / molecular mass	Molecular mass of DNA or RNA, Molecular mass of a protein
Sequences / primers	Melting temperature by the Wallace rule, Melting temperature by nearest-neighbour thermodynamics
Sequences / protein properties	Protein extinction coefficient at 280 nm, Net charge of a protein at a pH, Isoelectric point of a protein, Grand average of hydropathy (GRAVY), Protein instability index, Aromaticity of a protein, Amino-acid composition of a protein, Secondary-structure propensity fractions
Systems biology / metabolic networks	Flux balance analysis (optimal objective)

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
bio_reverse_complement	Reverse complement of a DNA sequence	seq	revcomp
bio_transcription	Transcription of DNA into mRNA	seq, strand	mrna
bio_translation	Translation with the standard genetic code	seq, stops	protein, length
bio_gc_content	GC content of a nucleic acid	seq	gc_fraction, gc_count
bio_kmer_counts	k-mer counts of a sequence	seq, k	total, distinct, max_count, most_frequent
bio_hamming_distance	p-distance (Hamming) between aligned sequences	seq1, seq2	mismatches, p_distance, identity
bio_edit_distance	Edit (Levenshtein) distance between sequences	seq1, seq2	distance
bio_longest_orf	Longest open reading frame	seq	length_nt, protein_length, orf_count
bio_restriction_sites	Restriction sites and fragments	seq, enzyme	n_sites, positions, fragments
bio_nucleic_acid_mass	Molecular mass of DNA or RNA	seq, molecule, strands	mass
bio_protein_mass	Molecular mass of a protein	seq	mass
bio_protein_extinction	Protein extinction coefficient at 280 nm	seq	eps_reduced, eps_cystines, abs_reduced, abs_cystines
bio_protein_net_charge	Net charge of a protein at a pH	seq, pH	charge
bio_isoelectric_point	Isoelectric point of a protein	seq	pI, charge_at_7
bio_gravy	Grand average of hydropathy (GRAVY)	seq	gravy
bio_instability_index	Protein instability index	seq	index
bio_tm_wallace	Melting temperature by the Wallace rule	seq	tm
bio_tm_nearest_neighbor	Melting temperature by nearest-neighbour thermodynamics	seq, Na, dnac1, dnac2	tm
bio_global_alignment_score	Global alignment score (Needleman-Wunsch, linear gaps)	seq1, seq2, match, mismatch, gap	score
bio_global_alignment_affine	Global alignment score with affine gaps (Gotoh)	seq1, seq2, match, mismatch, open, extend	score
bio_local_alignment_score	Local alignment score (Smith-Waterman, linear gaps)	seq1, seq2, match, mismatch, gap	score
bio_local_alignment_affine	Local alignment score with affine gaps	seq1, seq2, match, mismatch, open, extend	score
bio_protein_global_blosum62	Global protein alignment score (BLOSUM62, affine gaps)	seq1, seq2, open, extend	score
bio_protein_local_blosum62	Local protein alignment score (BLOSUM62, affine gaps)	seq1, seq2, open, extend	score
bio_semiglobal_alignment_score	Semi-global (end-gap-free) alignment score	seq1, seq2, match, mismatch, gap	score
bio_cross_genotype_probability	Probability of an offspring genotype in a cross	parent1, parent2, offspring	probability, fraction
bio_cross_phenotype_ratio	Phenotype ratio of a cross (complete dominance)	parent1, parent2	p_all_dominant, p_all_recessive, n_genotypes, n_phenotypes, ratio
bio_hardy_weinberg_test	Hardy-Weinberg chi-square test	n_AA, n_Aa, n_aa	p, q, exp_AA, exp_Aa, exp_aa, chi2, p_value, Ho, He, F
bio_hardy_weinberg_exact_test	Hardy-Weinberg exact test	n_AA, n_Aa, n_aa	p_value, p_observed
bio_haploid_selection	Allele frequency under haploid (genic) selection	p0, s, t	p_t
bio_recessive_lethal	Decline of a recessive lethal allele	q0, t	q_t
bio_heterozygote_advantage	Equilibrium under heterozygote advantage	w_AA, w_Aa, w_aa	p_eq
bio_mutation_selection_balance	Mutation-selection balance	mu, s, h	q_eq
bio_drift_heterozygosity	Loss of heterozygosity by genetic drift	N, i, t	H0, H_t, retained
bio_moran_fixation	Fixation probability in the Moran process	N, i, r	rho
bio_ne_sex_ratio	Effective population size with unequal sex ratio	N_m, N_f	Ne, Ne_over_N
bio_ne_fluctuating_size	Effective size of a fluctuating population	sizes	Ne, retained
bio_fst	Fixation index FST from subpopulation allele frequencies	p	HS, HT, FST

id	operation	inputs	outputs
bio_linkage_disequilibrium	Linkage disequilibrium D, D' and r^2	n_AB, n_Ab, n_aB, n_ab	D, D_prime, r2
bio_map_distance	Map distance from a recombination fraction	r, map	d_cM
bio_recombination_fraction	Recombination fraction from a map distance	d_cM, map	r
bio_pedigree_inbreeding	Inbreeding coefficient from a pedigree	pedigree, individual	F
bio_jukes_cantor_distance	Jukes-Cantor distance	seq1, seq2	p, d
bio_kimura_distance	Kimura two-parameter distance	seq1, seq2	d
bio_upgma	UPGMA tree heights	D	root_height, heights
bio_neighbor_joining	Neighbour-joining branch lengths	D	total_length, branch
bio_tree_count	Number of possible phylogenetic trees	n, rooted	count
bio_robinson_foulds	Robinson-Foulds distance between trees	tree1, tree2	rf, rf_normalized
bio_fitch_parsimony	Parsimony score of a tree (Fitch)	tree, states	score
bio_coalescent_times	Expected coalescence times	n, Ne	t_mrca, total_length
bio_nucleotide_diversity	Nucleotide diversity	seqs	k, pi
bio_watterson_theta	Watterson's estimator of theta	seqs	S, theta, theta_site
bio_tajima_d	Tajima's D	seqs	D, k
bio_lineweaver_burk	Lineweaver-Burk fit of Michaelis-Menten data	S, v	Vmax, Km, r2
bio_hanes_woolf	Hanes-Woolf fit of Michaelis-Menten data	S, v	Vmax, Km, r2
bio_eadie_hofstee	Eadie-Hofstee fit of Michaelis-Menten data	S, v	Vmax, Km, r2
bio_michaelis_menten_fit	Michaelis-Menten nonlinear least-squares fit	S, v	Vmax, Km
bio_michaelis_menten_progress	Substrate remaining in a Michaelis-Menten reaction	S0, Vmax, Km, t	S, P
bio_michaelis_menten_time	Time to reach a conversion in a Michaelis-Menten reaction	S0, Vmax, Km, X	t
bio_hill_plot	Hill coefficient from a Hill plot	S, Y	n, K, r2
bio_hill_fit	Hill equation nonlinear least-squares fit	S, v	Vmax, K, n
bio_competitive_inhibition_fit	Competitive inhibition global fit	S, I, v	Vmax, Km, Ki
bio_aromaticity	Aromaticity of a protein	seq	aromaticity, aromatic_count
bio_amino_acid_composition	Amino-acid composition of a protein	seq	percent, most_common, length
bio_secondary_structure_fraction	Secondary-structure propensity fractions	seq	helix, turn, sheet
bio_leslie_projection	Leslie matrix population projection	F, S, n0, t	n_t, total
bio_leslie_growth_rate	Leslie matrix growth rate and stable age distribution	F, S	lambda1, r, R0, stable
bio_shannon_diversity	Shannon diversity index	counts, base	H, evenness, effective_number
bio_simpson_diversity	Simpson diversity indices	counts, estimator	D, simpson, inverse
bio_chao1	Chao1 species richness estimator	counts, form	chao1, s_obs, singletons, doubletons
bio_ace	ACE species richness estimator	counts, rare_threshold	ace
bio_goods_coverage	Good's coverage of a sample	counts	coverage
bio_richness_indices	Margalef and Menhinick richness indices	counts	margalef, menhinick
bio_dominance_indices	Berger-Parker and McIntosh indices	counts	berger_parker, mcintosh
bio_brillouin_diversity	Brillouin diversity index	counts	H_B
bio_rarefaction	Rarefied species richness	counts, n	expected_species
bio_beta_diversity	Beta diversity between two samples	x, y	bray_curtis, jaccard, sorensen
bio_species_area	Species-area relationship fit	A, S	c, z, r2
bio_petersen_estimate	Mark-recapture population estimate (Petersen MLE)	M, C, R	N_mle
bio_schumacher_eschmeyer	Schumacher-Eschmeyer multiple-census estimate	C, R, M	N

id	operation	inputs	outputs
bio_chemostat_steady_state	Chemostat steady state (Monod kinetics)	mu_max, Ks, Y, D, S0	S, X
bio_flux_balance_analysis	Flux balance analysis (optimal objective)	S, lb, ub, c	objective
bio_faith_pd	Faith's phylogenetic diversity	tree, taxa	pd
bio_hardy_weinberg_frequencies	Hardy-Weinberg genotype and carrier frequencies	p, q, q2, N	p, q, p2, het_2pq, q2, carrier_frequency, carrier_among_unaffected, dominant_phenotype, exp_AA, exp_Aa, exp_aa
bio_testcross_recombination	Recombination frequency from two-point testcross counts	parental, recombinant	RF, RF_percent, d_cM, SE, linkage
bio_three_point_testcross	Three-point testcross: gene order, map distances and interference	classes, counts	gene_order, middle_gene, parental_classes, dco_classes, RF_left, RF_right, d_left_cM, d_right_cM, d_total_cM, RF_outer_direct, coincidence, interference

Reverse complement of a DNA sequence, `bio_reverse_complement`

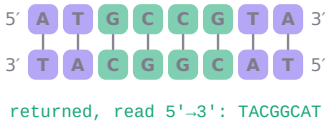
The reverse complement of a DNA strand (the partner strand read 5'->3').

$$5'\text{-}x_1 \dots x_n\text{-}3' \mapsto 5'\text{-}\bar{x}_n \dots \bar{x}_1\text{-}3'$$

input	meaning	condition
`seq`	DNA sequence 5'->3' (A, C, G, T)	

output	meaning
`revcomp`	reverse complement 5'->3'

- **Method:** complement each base (A-T, C-G) and reverse the order
- **Assumes:** DNA letters A, C, G, T only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The given strand ATGCCGTA (top, 5′→3′) paired with its partner strand (bottom, written 3′→5′ under it); A–T pairs and G–C pairs in two colours. Reading the partner strand 5′→3′ gives the package's reverse complement. Marked: revcomp = TACGGCAT.

```
computational_biology_ai999x.solve('bio_reverse_complement', seq='ATGCCGTA')
# -> revcomp = TACGGCAT   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_reverse_complement', seq='GAATTC')
# -> revcomp = GAATTC   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_reverse_complement', seq='aaccgt')
# -> revcomp = ACGGTT   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reverse complement: seq = ATGCCGTA	SOLVED	revcomp = TACGGCAT
Reverse complement: seq = GAATTC	SOLVED	revcomp = GAATTC
Reverse complement: seq = aaccgt	SOLVED	revcomp = ACGGTT
Reverse complement:	NEEDS_INPUT	bio_reverse_complement needs seq (DNA sequence 5'->3' (A, C, G, T)). Example: solve('bio_reverse_complement', seq='ATGCC
Reverse complement: seq = ATGCCGTA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Reverse complement: seq = ATGCCGTA and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Reverse complement: seq = ATG1CG	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Reverse complement: seq = ATGXCG	INVALID_INPUT	Reverse complement of a DNA sequence: seq must be a DNA sequence of A, C, G and T only.
Reverse complement: seq = AUGC	INVALID_INPUT	Reverse complement of a DNA sequence: seq must be a DNA sequence of A, C, G and T only.

Transcription of DNA into mRNA, `bio_transcription`

The mRNA transcribed from a DNA strand given 5'->3', stated as the coding (sense) or template (antisense) strand.

$$\text{mRNA} = \text{coding strand with } T \to U = \overline{\text{template}}^{\,R}$$

input	meaning	condition
`seq`	DNA strand 5'->3' (A, C, G, T)	
`strand`	which strand is given: coding (sense) or template (antisense)	

output	meaning
`mrna`	mRNA 5'->3'

- **Method:** coding strand: T -> U; template strand: reverse complement, then T -> U
- **Assumes:** DNA letters only; the strand is stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The DNA ATGGCTTAA (5′→3′) given once as the coding strand and once as the template strand, with the mRNA the package returns for each (5′→3′): coding → AUGGCUUAA (T read as U), template → UUAAGCCAU (the complement read in reverse). T and U shaded. Marked: mrna = AUGGCUUAA and UUAAGCCAU.

```
computational_biology_ai999x.solve('bio_transcription', seq='ATGGCTTAA', strand='coding')
# -> mrna = AUGGCUUAA   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_transcription', seq='TTAAGCCAT', strand='template')
# -> mrna = AUGGCUUAA   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transcription: seq = ATGGCTTAA, strand = coding	SOLVED	mrna = AUGGCUUAA
Transcription: seq = TTAAGCCAT, strand = template	SOLVED	mrna = AUGGCUUAA
Transcription: strand = coding	NEEDS_INPUT	bio_transcription needs seq (DNA strand 5'->3' (A, C, G, T)). Example: solve('bio_transcription', seq='ATGGCTTAA', stran
Transcription: seq = ATGGCTTAA, strand = coding, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Transcription: seq = ATGGCTTAA, strand = coding and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Transcription: seq = ATGGCTTAA	NEEDS_INPUT	bio_transcription needs strand (which strand is given: coding (sense) or template (antisense)). Example: solve('bio_tran

Translation with the standard genetic code, `bio_translation`

The protein encoded by a DNA or mRNA sequence read in frame 1 with the standard genetic code; stop codons shown as '*' or the protein truncated at the first stop.

$$\text{codon } x_{3i+1}x_{3i+2}x_{3i+3} \mapsto \text{amino acid}_i$$

input	meaning	condition
`seq`	DNA or mRNA sequence 5'->3', length a multiple of 3	
`stops`	show stop codons as '*' or truncate at the first stop	

output	meaning
`protein`	amino-acid sequence (one-letter codes)
`length`	number of residues shown

- **Method:** read codons in frame 1 and look each up in the standard genetic code (NCBI table 1)
- **Assumes:** length a multiple of 3; DNA or RNA letters, not a mixture
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



stops shown: MAIVMGR* · truncated at the stop: MAIVMGR

The coding sequence ATGGCCATTGTAATGGGCCGCTGA read in codons (numbered 1–8, alternately shaded) with the amino acid the package assigns under each codon by the standard genetic code; the stop codon is shown as *. Marked: protein = MAIVMGR* (stops shown), MAIVMGR (truncated at the stop), length 8.

```
computational_biology_ai999x.solve('bio_translation', seq='ATGGCCATTGTAATGGGCCGCTGA', stops='show')
# -> protein = MAIVMGR*, length = 8   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_translation', seq='AUGUUUUGGUAAGGC', stops='truncate')
# -> protein = MFW, length = 3   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_translation', seq='ATGAGATGA', stops='show')
# -> protein = MR*, length = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Translation: seq = ATGGCCATTGTAATGGGCCGCTGA, stops = show	SOLVED	protein = MAIVMGR*, length = 8
Translation: seq = AUGUUUUGGUAAGGC, stops = truncate	SOLVED	protein = MFW, length = 3
Translation: seq = ATGAGATGA, stops = show	SOLVED	protein = MR*, length = 3
Translation: stops = show	NEEDS_INPUT	bio_translation needs seq (DNA or mRNA sequence 5'->3', length a multiple of 3). Example: solve('bio_translation', seq='
Translation: seq = ATGGCCATTGTAATGGGCCGCTGA, stops = show, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Translation: seq = ATGGCCATTGTAATGGGCCGCTGA, stops = show and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Translation: seq = ATGGC, stops = show	INVALID_INPUT	Translation with the standard genetic code: the sequence length must be a multiple of 3 (whole codons read in frame 1).

GC content of a nucleic acid, `bio_gc_content`

The fraction and number of G and C bases in a DNA or RNA sequence.

$$\mathrm{GC} = \frac{n_G + n_C}{n}$$

input	meaning	condition
`seq`	DNA or RNA sequence	

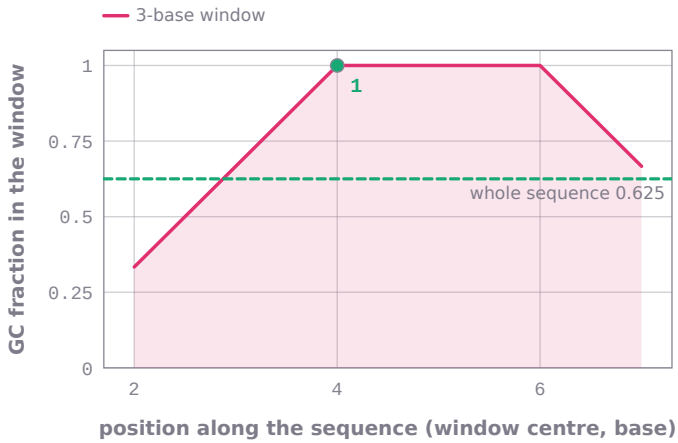
output	meaning
`gc_fraction`	fraction of G + C
`gc_count`	number of G and C bases

- **Method:** count G and C and divide by the length
- **Assumes:** unambiguous DNA or RNA letters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G + C = 5 of 8 bases · GC fraction = 0.625

The sequence ATGCGCGA written base by base, G and C shaded. The package counts 5 G or C bases and returns the GC fraction 0.625. Marked: gc_count = 5, gc_fraction = 0.625.



GC fraction along the worked example's 8-base sequence (ATGCGCGA): each point is the package's GC fraction of one 3-base window (6 windows, step 1). Marked: the richest window, 1; line: the whole sequence, 0.625.

```
computational_biology_ai999x.solve('bio_gc_content', seq='ATGCGCGA')
# -> gc_fraction = 0.625, gc_count = 5   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_gc_content', seq='GGAUCUUCGGAUCU')
# -> gc_fraction = 0.5, gc_count = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
GC content: seq = ATGCGCGA	SOLVED	gc_fraction = 0.625, gc_count = 5
GC content: seq = GGAUCUUCGGAUCU	SOLVED	gc_fraction = 0.5, gc_count = 7
GC content:	NEEDS_INPUT	bio_gc_content needs seq (DNA or RNA sequence). Example: solve('bio_gc_content', seq='ATGCGCGA').
GC content: seq = ATGCGCGA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
GC content: seq = ATGCGCGA and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

k-mer counts of a sequence, `bio_kmer_counts`

Overlapping k-mer counts: the number of k-mers, of distinct k-mers, the highest count and the most frequent k-mer (alphabetically first among ties).

$$N_k = n - k + 1$$

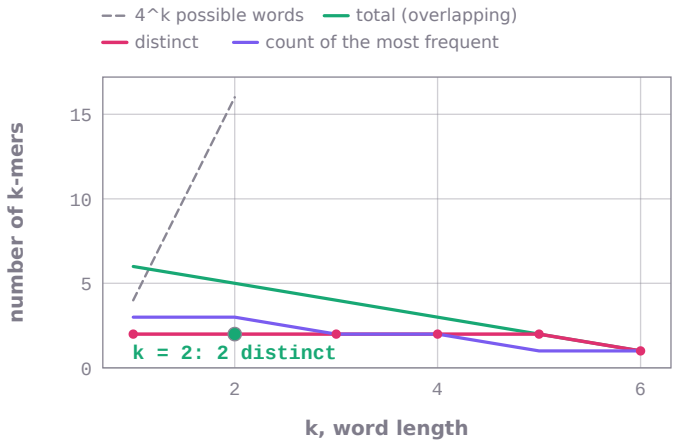
input	meaning	condition
`seq`	DNA or RNA sequence	
`k`	k-mer length	

output	meaning
`total`	number of overlapping k-mers $(n - k + 1)$
`distinct`	number of distinct k-mers
`max_count`	count of the most frequent k-mer
`most_frequent`	most frequent k-mer (alphabetically first among ties)

- **Method:** slide a window of length k one base at a time and count each word
- **Assumes:** 1 <= k <= sequence length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 6-base sequence ATATAT cut into overlapping 2-mers: the package counts 5 2-mers, 2 of them distinct, the most frequent being AT (3 times, shaded where it occurs). Marked: total = 5, distinct = 2, most_frequent = AT, max_count = 3.



k-mer counts of the worked example's 6-base sequence ATATAT for k = 1 … 6, each a run of the package: the total number of overlapping words (6 − k + 1), the number of distinct words and the count of the most frequent; the dashed line is the number of possible words 4^k (drawn while it is under 40). From k = 5 on every word is unique. Marked: the worked example's k = 2.

```
computational_biology_ai999x.solve('bio_kmer_counts', seq='ATATAT', k=2)
# -> total = 5, distinct = 2, max_count = 3, most_frequent = AT   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_kmer_counts', seq='GATTACA', k=3)
# -> total = 5, distinct = 5, max_count = 1, most_frequent = ACA   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
k-mer counts: seq = ATATAT, k = 2	SOLVED	total = 5, distinct = 2, max_count = 3, most_frequent = AT
k-mer counts: seq = GATTACA, k = 3	SOLVED	total = 5, distinct = 5, max_count = 1, most_frequent = ACA
k-mer counts: k = 2	NEEDS_INPUT	bio_kmer_counts needs seq (DNA or RNA sequence). Example: solve('bio_kmer_counts', seq='ATATAT', k=2).
k-mer counts: seq = ATATAT, k = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
k-mer counts: seq = ATATAT, k = 2 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

p-distance (Hamming) between aligned sequences, `bio_hamming_distance`

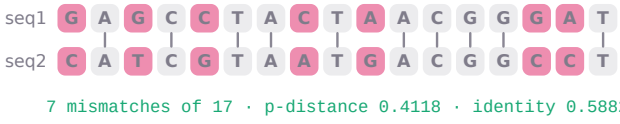
Mismatches, p-distance and identity of two aligned sequences of equal length (no indels): the proportion of sites that differ.

$$p = \frac{1}{n}\sum_{i}[x_i \neq y_i]$$

input	meaning	condition
`seq1`	first aligned sequence	
`seq2`	second aligned sequence (same length)	

output	meaning
`mismatches`	number of differing sites
`p_distance`	proportion of differing sites
`identity`	proportion of identical sites

- **Method:** compare the sequences site by site
- **Assumes:** equal lengths (aligned, no gaps)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two aligned sequences of 17 bases, seq1 = GAGCCTACTAACGGGAT and seq2 = CATCGTAATGACGGCCT: positions where they differ are shaded, equal positions joined by a bar. The package returns mismatches = 7, p_distance = 0.4118 and identity = 0.5882.

```
computational_biology_ai999x.solve('bio_hamming_distance', seq1='GAGCCTACTAACGGGAT', seq2='CATCGTAATGACGGCCT')
# -> mismatches = 7, p_distance = 0.411765, identity = 0.588235   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hamming_distance', seq1='ACGT', seq2='ACGT')
# -> mismatches = 0, p_distance = 0, identity = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
p-distance: seq1 = GAGCCTACTAACGGGAT, seq2 = CATCGTAATGACGGCCT	SOLVED	mismatches = 7, p_distance = 0.411765, identity = 0.588235
p-distance: seq1 = ACGT, seq2 = ACGT	SOLVED	mismatches = 0, p_distance = 0, identity = 1
p-distance: seq2 = CATCGTAATGACGGCCT	NEEDS_INPUT	bio_hamming_distance needs seq1 (first aligned sequence). Example: solve('bio_hamming_distance', seq1='GAGCCTACTAACGGGAT
p-distance: seq1 = GAGCCTACTAACGGGAT, seq2 = CATCGTAATGACGGCCT, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
p-distance: seq1 = GAGCCTACTAACGGGAT, seq2 = CATCGTAATGACGGCCT and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hamming distance: seq1 = ACGT, seq2 = ACG	INVALID_INPUT	p-distance (Hamming) between aligned sequences: the two aligned sequences must have the same length.

Edit (Levenshtein) distance between sequences, `bio_edit_distance`

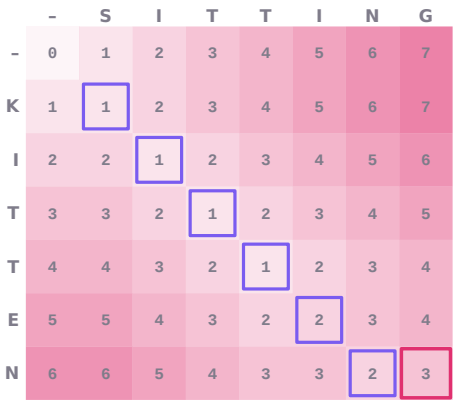
The minimum number of substitutions, insertions and deletions turning one sequence into the other (unit costs; an unaligned comparison, unlike the p-distance).

$$D_{i,j} = \min(D_{i-1,j} + 1,\ D_{i,j-1} + 1,\ D_{i-1,j-1} + [x_i \neq y_j])$$

input	meaning	condition
`seq1`	first sequence	
`seq2`	second sequence	

output	meaning
`distance`	edit distance (unit costs)

- **Method:** dynamic programming over prefixes (Wagner-Fischer)
- **Assumes:** letters only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Edit distance between KITTEN and SITTING as a table: cell (i, j) is the package's edit distance between the first i letters of KITTEN and the first j letters of SITTING (42 runs); the top row and left column are the distances to the empty string (j and i). The outlined cells trace one cheapest series of edits back through cells that satisfy the recurrence (match 0; substitution, insertion, deletion 1). Marked: distance = 3 (bottom right).

```
computational_biology_ai999x.solve('bio_edit_distance', seq1='KITTEN', seq2='SITTING')
# -> distance = 3   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_edit_distance', seq1='GATTACA', seq2='GCATGCT')
# -> distance = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Edit distance: seq1 = KITTEN, seq2 = SITTING	SOLVED	distance = 3
Edit distance: seq1 = GATTACA, seq2 = GCATGCT	SOLVED	distance = 4

question	outcome	result
Edit distance: seq2 = SITTING	NEEDS_INPUT	bio_edit_distance needs seq1 (first sequence). Example: solve('bio_edit_distance', seq1='KITTEN', seq2='SITTING').
Edit distance: seq1 = KITTEN, seq2 = SITTING, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Edit distance: seq1 = KITTEN, seq2 = SITTING and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Longest open reading frame, `bio_longest_orf`

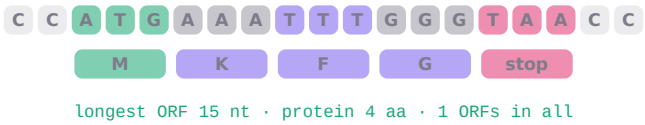
The longest open reading frame (ATG to the first in-frame stop, stop included) over all six frames of a DNA sequence, its protein length and the number of ORFs.

$$\ell = 3(j - i + 1) \text{ for ATG at codon } i, \text{ first stop at codon } j$$

input	meaning	condition
`seq`	DNA sequence 5'->3'	

output	meaning
`length_nt`	length of the longest ORF in nucleotides, stop codon included
`protein_length`	amino acids encoded by the longest ORF (stop excluded)
`orf_count`	number of ORFs (one per stop codon reached from an upstream in-frame ATG)

- **Method:** scan the three frames of both strands codon by codon: open at the first ATG, close at the next stop
- **Assumes:** DNA letters only; standard genetic code, start ATG, stops TAA TAG TGA
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The sequence CCATGAAATTTGGGTAACC (5′→3′) with its longest open reading frame shaded: the start codon ATG, the codons and the stop codon; under it, the package's translation of that frame, MKFG. The package returns length_nt = 15 (stop included), protein_length = 4 and orf_count = 1; the frame's position is located in the sequence as the ATG whose first in-frame stop closes 15 bases later.

```
computational_biology_ai999x.solve('bio_longest_orf', seq='CCATGAAATTTGGGTAACC')
# -> length_nt = 15, protein_length = 4, orf_count = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_longest_orf', seq='TTACATCCCAT')
# -> length_nt = 6, protein_length = 1, orf_count = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_longest_orf', seq='ACGTACGT')
# -> length_nt = 0, protein_length = 0, orf_count = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Longest ORF: seq = CCATGAAATTTGGGTAACC	SOLVED	length_nt = 15, protein_length = 4, orf_count = 1
Longest ORF: seq = TTACATCCCAT	SOLVED	length_nt = 6, protein_length = 1, orf_count = 1
Longest ORF: seq = ACGTACGT	SOLVED	length_nt = 0, protein_length = 0, orf_count = 0
Longest ORF:	NEEDS_INPUT	bio_longest_orf needs seq (DNA sequence 5'->3'). Example: solve('bio_longest_orf', seq='CCATGAAATTTGGGTAACC').
Longest ORF: seq = CCATGAAATTTGGGTAACC, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Longest ORF: seq = CCATGAAATTTGGGTAACC and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Restriction sites and fragments, `bio_restriction_sites`

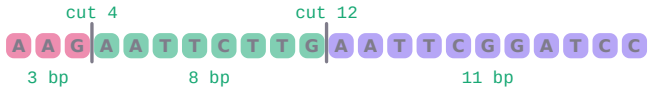
Where a restriction enzyme cuts a linear DNA sequence: the number of sites, the cut positions and the fragment lengths.

$$\text{cut at } i + c \text{ for each site start } i$$

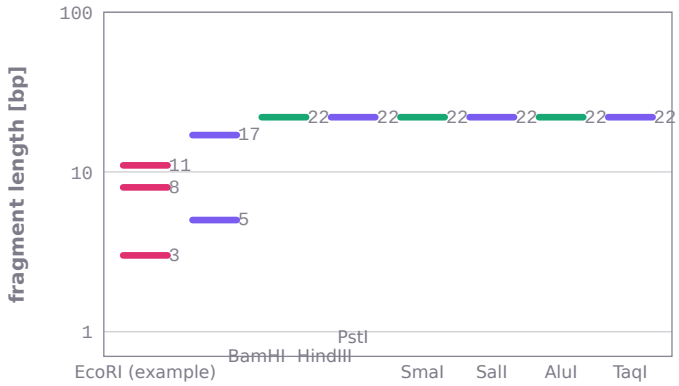
input	meaning	condition
`seq`	linear DNA sequence 5'->3'	
`enzyme`	restriction enzyme (name)	

output	meaning
`n_sites`	number of cuts
`positions`	cut positions (1-based first base after each cut, top strand)
`fragments`	fragment lengths in bp, 5'->3' order

- **Method:** find every occurrence of the recognition site (overlaps included) and cut at the enzyme's offset
- **Assumes:** linear DNA; a listed enzyme
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



EcoRI digest of the sequence AAGAATTCTTGAATTCGGATCC (22 bases, 5′→3′): the package returns 2 cut positions (4, 12, the first base after each cut) and the fragment lengths 3, 8, 11 bp; each fragment is shaded in its own colour. Marked: positions = [4, 12], fragments = [3, 8, 11].



Fragments of the worked example's 22-base sequence (AAGAATTCTTGAATTCGGATCC) cut by 8 enzymes, its own (EcoRI) first, one lane each, on a logarithmic length axis as on a gel: every band is a fragment length the package returns (equal lengths share one band). Number of sites: EcoRI 2, BamHI 1, HindIII 0, PstI 0, SmaI 0, SalI 0, AluI 0, TaqI 0.

```
computational_biology_ai999x.solve('bio_restriction_sites', seq='AAGAATTCTTGAATTCGGATCC', enzyme='EcoRI')
# -> n_sites = 2, positions = [4, 12], fragments = [3, 8, 11]   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_restriction_sites', seq='AAACTGCAGTTT', enzyme='PstI')
# -> n_sites = 1, positions = [9], fragments = [8, 4]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Restriction sites: seq = AAGAATTCTTGAATTCGGATCC, enzyme = EcoRI	SOLVED	n_sites = 2, positions = [4, 12], fragments = [3, 8, 11]
Restriction sites: seq = AAACTGCAGTTT, enzyme = PstI	SOLVED	n_sites = 1, positions = [9], fragments = [8, 4]
Restriction sites: enzyme = EcoRI	NEEDS_INPUT	bio_restriction_sites needs seq (linear DNA sequence 5'->3'). Example: solve('bio_restriction_sites', seq='AAGAATTCTTGAA
Restriction sites: seq = AAGAATTCTTGAATTCGGATCC, enzyme = EcoRI, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Restriction sites: seq = AAGAATTCTTGAATTCGGATCC, enzyme = EcoRI and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Restriction sites: seq = AAGAATTC, enzyme = FooI	INVALID_INPUT	enzyme: one of AluI, ApaI, BamHI, BglII, ClaI, EcoRI, EcoRV, HaeIII, HindIII, KpnI, MluI, MspI, NcoI, NdeI, NheI, NotI,

Molecular mass of DNA or RNA, `bio_nucleic_acid_mass`

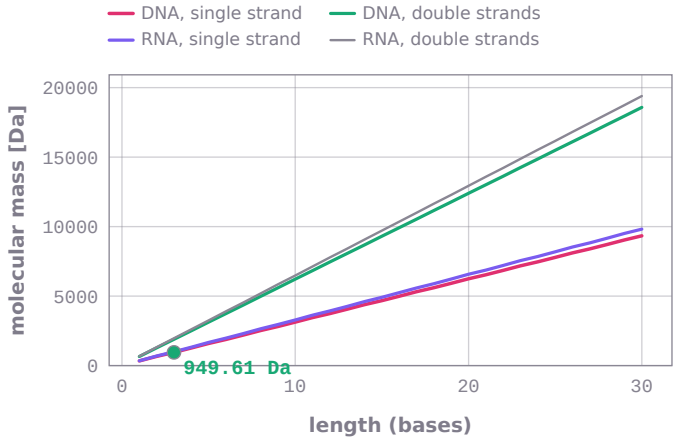
The average molecular mass of a single- or double-stranded linear DNA or RNA with a 5' phosphate.

$$M = \sum_i m(x_i) - (n-1)\, m_{\mathrm{H_2O}}$$

input	meaning	condition
`seq`	DNA or RNA sequence	
`molecule`	molecule type	
`strands`	single- or double-stranded	

output	meaning
`mass`	molecular mass in g/mol (Da)

- **Method:** sum of nucleotide-monophosphate masses minus one water per phosphodiester bond (each strand)
- **Assumes:** the sequence letters match the stated molecule
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Molecular mass against length for chains made by repeating AGC (the worked example's sequence), as single- or double-stranded DNA or RNA: 30 lengths × 4 cases, each a run of the package. Marked: the worked example, single-stranded DNA AGC, 949.6095 Da.

```
computational_biology_ai999x.solve('bio_nucleic_acid_mass', seq='AGC', molecule='DNA', strands='single')
# -> mass = 949.61   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_nucleic_acid_mass', seq='AGC', molecule='RNA', strands='single')
# -> mass = 997.608   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_nucleic_acid_mass', seq='ATGC', molecule='DNA', strands='double')
# -> mass = 2507.61   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nucleic acid mass: seq = AGC, molecule = DNA, strands = single	SOLVED	mass = 949.61
Nucleic acid mass: seq = AGC, molecule = RNA, strands = single	SOLVED	mass = 997.608
Nucleic acid mass: seq = ATGC, molecule = DNA, strands = double	SOLVED	mass = 2507.61
Nucleic acid mass: molecule = DNA, strands = single	NEEDS_INPUT	bio_nucleic_acid_mass needs seq (DNA or RNA sequence). Example: solve('bio_nucleic_acid_mass', seq='AGC', molecule='DNA'
Nucleic acid mass: seq = AGC, molecule = DNA, strands = single, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Nucleic acid mass: seq = AGC, molecule = DNA, strands = single and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Nucleic acid mass: seq = AGU, molecule = DNA, strands = single	INVALID_INPUT	Molecular mass of DNA or RNA: seq must be a DNA sequence of A, C, G and T only.

Molecular mass of a protein, `bio_protein_mass`

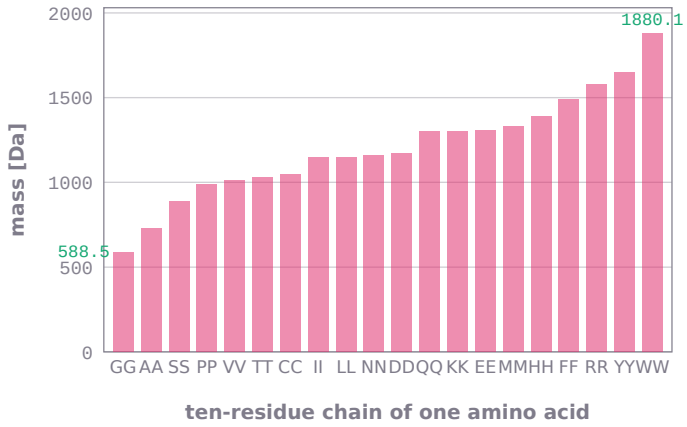
The average molecular mass of a protein from its amino-acid sequence.

$$M = \sum_i m(a_i) - (n-1)\, m_{\mathrm{H_2O}}$$

input	meaning	condition
`seq`	protein sequence (20 standard one-letter codes)	

output	meaning
`mass`	molecular mass in g/mol (Da)

- **Method:** sum of amino-acid masses minus one water per peptide bond
- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mass of a ten-residue chain of each amino acid (poly-G … poly-W), each a run of the package, sorted: the spread shows how residue masses differ. Marked: the lightest, GGGGGGGGGG, 588.53 Da, and the heaviest, WWWWWWWWWW, 1880.1 Da. The 152-residue protein of the worked example weighs 17103.16 Da.

```
computational_biology_ai999x.solve('bio_protein_mass',
seq='MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGL
# -> mass = 17103.2   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_protein_mass', seq='G')
# -> mass = 75.0666   (SOLVED)
```

Questions in words (documented forms), as executed:

question
Protein mass: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK
Protein mass: seq = G
Protein mass:
Protein mass: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK zz = 1
Protein mass: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK and 7 more
Protein mass: seq = MAXB

Protein extinction coefficient at 280 nm, `bio_protein_extinction`

The molar extinction coefficient at 280 nm (cysteines reduced, and all cysteines as cystines) and the absorbance of a 1 g/L solution.

$$\varepsilon_{280} = 5500\, n_W + 1490\, n_Y + 125\, n_{\text{cystine}}$$

input	meaning	condition
`seq`	protein sequence	

output	meaning
`eps_reduced`	extinction coefficient, cysteines reduced (M^-1 cm^-1)
`eps_cystines`	extinction coefficient, cysteines paired as cystines (M^-1 cm^-1)
`abs_reduced`	absorbance of 1 g/L, 1 cm, cysteines reduced
`abs_cystines`	absorbance of 1 g/L, 1 cm, cystines

- **Method:** 5500 per Trp + 1490 per Tyr (+ 125 per cystine); divide by the molecular mass for 1 g/L
- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	Y×0	Y×1	Y×2	Y×3	Y×4
W×0	0	1490	2980	4470	5960
W×1	5500	6990	8480	9970	11460
W×2	11000	12490	13980	15470	16960
W×3	16500	17990	19480	20970	22460

ε280 [1/(M·cm)] · rows: W count 0–3 · columns: Y count 0–4

Molar extinction coefficient at 280 nm (reduced cysteines) of the peptide G8 followed by a tryptophans and b tyrosines, a = 0–3 (rows), b = 0–4 (columns): each cell a run of the package. Each W adds the same amount, each Y another. The 152-residue protein of the worked example gives eps_reduced = 17420, eps_cystines = 17545, absorbance of 1 g/L 1.019 (reduced) and 1.026 (cystines).

```
computational_biology_ai999x.solve('bio_protein_extinction',
seq='MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGL
# -> eps_reduced = 17420, eps_cystines = 17545, abs_reduced = 1.01853, abs_cystines = 1.02583   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_protein_extinction', seq='MWYCC')
# -> eps_reduced = 6990, eps_cystines = 7115, abs_reduced = 9.91658, abs_cystines = 10.0939   (SOLVED)
```

Questions in words (documented forms), as executed:

question
Protein extinction coefficient: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK
Protein extinction coefficient: seq = MWYCC
Protein extinction coefficient:
Protein extinction coefficient: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK zz = 1
Protein extinction coefficient: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK and 7 more

Net charge of a protein at a pH, `bio_protein_net_charge`

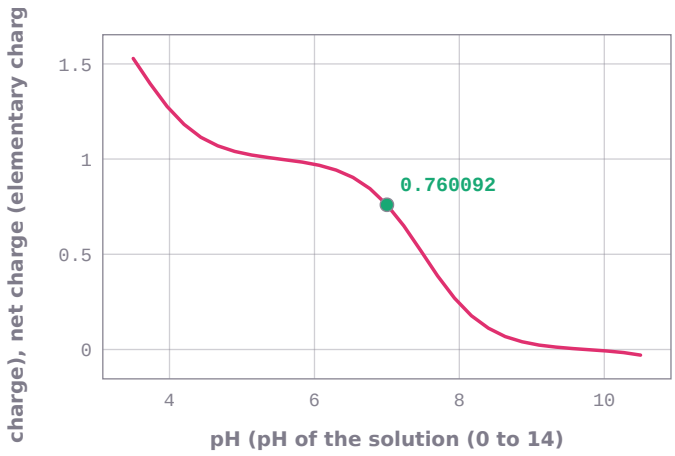
The net charge of a protein at a stated pH from its ionisable groups (Bjellqvist pK set).

$$Z = \sum_{\text{basic}} \frac{n_i}{1 + 10^{\,pH - pK_i}} - \sum_{\text{acidic}} \frac{n_j}{1 + 10^{\,pK_j - pH}}$$

input	meaning	condition
`seq`	protein sequence	
`pH`	pH of the solution (0 to 14)	

output	meaning
`charge`	net charge (elementary charges)

- **Method:** Henderson-Hasselbalch partial charges of the termini and the K, R, H, D, E, C, Y side chains
- **Assumes:** standard amino acids only; $0 \le pH \le 14$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



charge (net charge (elementary charges)) against pH (pH of the solution (0 to 14)) from 3.5 to 10.5. Other inputs as in the worked example: seq = INGAR. 31 points computed by computational-biology-ai999x; marked: the worked example, pH = 7, charge = 0.760092.

```
computational_biology_ai999x.solve('bio_protein_net_charge', seq='INGAR', pH=7)
# -> charge = 0.760092   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_protein_net_charge', seq='PETER', pH=4.53)
# -> charge = 0.00279563   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Protein net charge: seq = INGAR, pH = 7	SOLVED	charge = 0.760092
Protein net charge: seq = PETER, pH = 4.53	SOLVED	charge = 0.00279563
Protein net charge: pH = 7	NEEDS_INPUT	bio_protein_net_charge needs seq (protein sequence). Example: solve('bio_protein_net_charge', seq='INGAR', pH=7).
Protein net charge: seq = INGAR, pH = 7, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Protein net charge: seq = INGAR, pH = 7 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Isoelectric point of a protein, `bio_isoelectric_point`

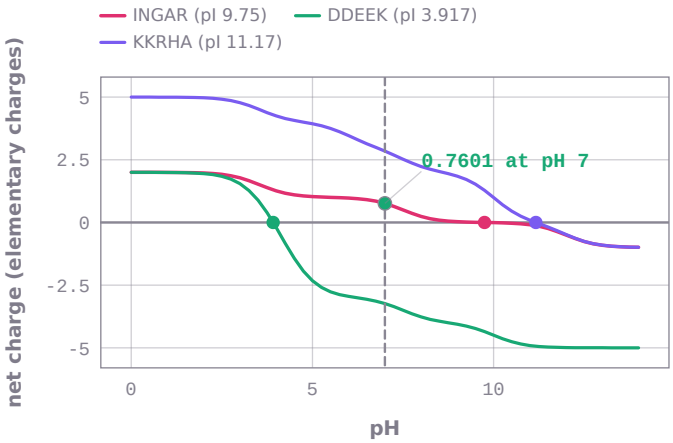
The isoelectric point (pH of zero net charge) of a protein, with its net charge at pH 7 (Bjellqvist pK set).

$$Z(\mathrm{pI}) = 0$$

input	meaning	condition
`seq`	protein sequence	

output	meaning
`pI`	isoelectric point
`charge_at_7`	net charge at pH 7

- **Method:** the net charge decreases with pH: bisection for the pH where it is zero, to full precision
- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Net charge against pH for three peptides (the worked example INGAR, an acidic DDEEK and a basic KKRHA), each curve 57 runs of the package's net-charge capability; the dot on the zero line is the isoelectric point the package returns: INGAR pI = 9.75, DDEEK pI = 3.917, KKRHA pI = 11.17. Marked: INGAR carries 0.7601 at pH 7 (charge_at_7).

```
computational_biology_ai999x.solve('bio_isoelectric_point', seq='INGAR')
# -> pI = 9.75002, charge_at_7 = 0.760092   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_isoelectric_point', seq='PETER')
# -> pI = 4.53209, charge_at_7 = -1.03586   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Isoelectric point: seq = INGAR	SOLVED	pI = 9.75002, charge_at_7 = 0.760092
Isoelectric point: seq = PETER	SOLVED	pI = 4.53209, charge_at_7 = -1.03586
Isoelectric point:	NEEDS_INPUT	bio_isoelectric_point needs seq (protein sequence). Example: solve('bio_isoelectric_point', seq='INGAR').
Isoelectric point: seq = INGAR, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Isoelectric point: seq = INGAR and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Grand average of hydropathy (GRAVY), `bio_gravy`

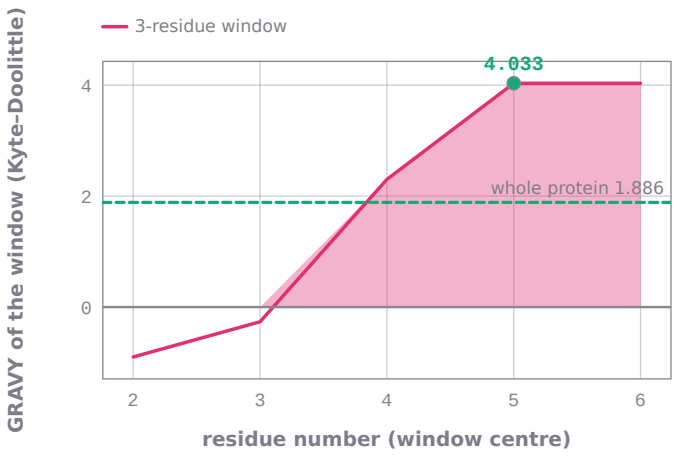
The Kyte-Doolittle grand average of hydropathy of a protein (positive = hydrophobic).

$$\mathrm{GRAVY} = \frac{1}{n}\sum_{i} h(a_i)$$

input	meaning	condition
`seq`	protein sequence	

output	meaning
`gravy`	GRAVY score

- **Method:** mean Kyte-Doolittle hydropathy over the residues
- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hydropathy plot of the worked example's 7-residue protein (MKTLLIL): each point is the package's GRAVY (mean Kyte–Doolittle hydropathy) of one 3-residue window (5 windows); stretches above zero (shaded) are hydrophobic. Marked: the most hydrophobic window, centred at residue 5, GRAVY 4.033; line: the whole protein, 1.886.

```
computational_biology_ai999x.solve('bio_gravy', seq='MKTLLIL')
# -> gravy = 1.88571   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_gravy', seq='DDEEKR')
# -> gravy = -3.73333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
GRAVY: seq = MKTLLIL	SOLVED	gravy = 1.88571
GRAVY: seq = DDEEKR	SOLVED	gravy = -3.73333
GRAVY:	NEEDS_INPUT	bio_gravy needs seq (protein sequence). Example: solve('bio_gravy', seq='MKTLLIL').
GRAVY: seq = MKTLLIL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
GRAVY: seq = MKTLLIL and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Protein instability index, `bio_instability_index`

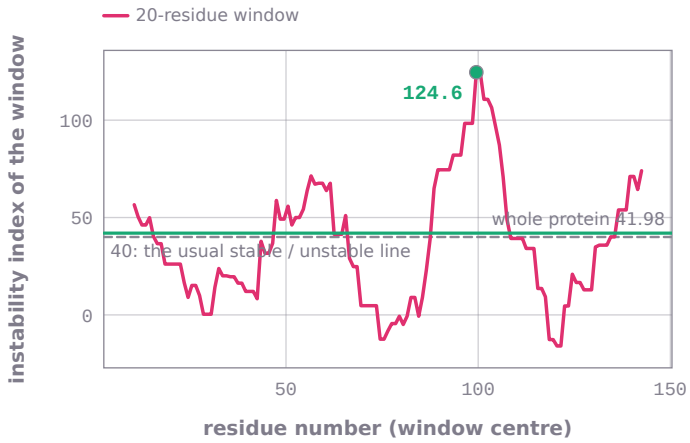
The Guruprasad instability index of a protein from its dipeptides (above 40 suggests an unstable protein in vivo).

$$II = \frac{10}{L}\sum_{i=1}^{L-1}\mathrm{DIWV}(a_i, a_{i+1})$$

input	meaning	condition
`seq`	protein sequence (at least 2 residues)	

output	meaning
`index`	instability index

- **Method:** sum of the dipeptide instability weights (DIWV) times $10/L$
- **Assumes:** standard amino acids, length >= 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Instability index along the 152-residue protein of the package's worked examples: each point is the package's index of one 20-residue window (133 windows), against the conventional threshold of 40 above which a protein is predicted unstable. Marked: the highest window, centred at residue 99.5 (124.6); line: the whole protein, index 41.98.

```
computational_biology_ai999x.solve('bio_instability_index',
seq='MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGL
# -> index = 41.9803   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_instability_index', seq='AC')
# -> index = 224.7   (SOLVED)
```

Questions in words (documented forms), as executed:

question
Instability index: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK
Instability index: seq = AC
Instability index:
Instability index: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK zz = 1
Instability index: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK and 7 more

Melting temperature by the Wallace rule, `bio_tm_wallace`

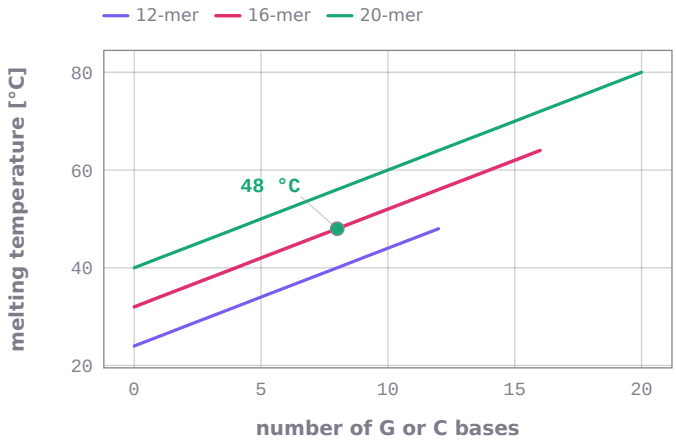
The approximate melting temperature of a short DNA primer (14-20 nt) by the Wallace rule.

$$T_m = 2(n_A + n_T) + 4(n_G + n_C)\ ^\circ\mathrm{C}$$

input	meaning	condition
`seq`	DNA primer sequence	

output	meaning
`tm`	melting temperature in degC

- **Method:** 2 degC per A or T plus 4 degC per G or C
- **Assumes:** unambiguous DNA letters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Wallace-rule melting temperature of 12-, 16- and 20-base oligonucleotides against their number of G or C bases (sequences G…GA…A), each point a run of the package: every G or C adds twice what an A or T adds. Marked: the worked example ACGTTGCAATGCCGTA (16 bases, 8 G or C counted in it), tm = 48 °C.

```
computational_biology_ai999x.solve('bio_tm_wallace', seq='ACGTTGCAATGCCGTA')
# -> tm = 48   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_tm_wallace', seq='AAAAATTTTTGGGGG')
# -> tm = 40   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wallace melting temperature: seq = ACGTTGCAATGCCGTA	SOLVED	tm = 48
Wallace melting temperature: seq = AAAAATTTTTGGGGG	SOLVED	tm = 40
Wallace melting temperature:	NEEDS_INPUT	bio_tm_wallace needs seq (DNA primer sequence). Example: solve('bio_tm_wallace', seq='ACGTTGCAATGCCGTA').
Wallace melting temperature: seq = ACGTTGCAATGCCGTA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Wallace melting temperature: seq = ACGTTGCAATGCCGTA and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Melting temperature by nearest-neighbour thermodynamics, `bio_tm_nearest_neighbor`

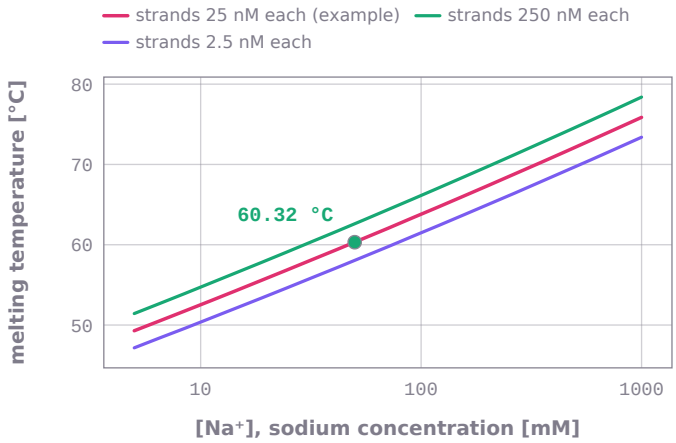
The melting temperature of a DNA duplex (perfect complement) from nearest-neighbour thermodynamics (Allawi-SantaLucia 1997) with the SantaLucia 1998 sodium correction.

$$T_m = \frac{1000\,\Delta H}{\Delta S + 0.368(N-1)\ln[\mathrm{Na}^+] + R\ln k} - 273.15$$

input	meaning	condition
`seq`	DNA sequence 5'->3'	
`Na`	sodium concentration in mM	
`dnac1`	concentration of the more concentrated strand in nM	
`dnac2`	concentration of the less concentrated strand in nM	

output	meaning
`tm`	melting temperature in degC

- **Method:** sum nearest-neighbour enthalpies and entropies with initiation terms; $Tm = 1000$ dH / (dS + R ln k) - 273.15 with dS corrected by 0.368 $(N-1)$ ln[Na+]
- **Assumes:** unambiguous DNA, length >= 2; $Na > 0$; dnac1 > dnac2/2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Nearest-neighbour melting temperature of the 28-mer CGTTCCAAAGATGTGGGCATGAGCTTAC against sodium concentration (5–1000 mM, logarithmic) at three strand concentrations, each point a run of the package: more salt and more strands both stabilise the duplex. Marked: the worked example, Na = 50 mM, 25 nM each, tm = 60.32 °C.

```
computational_biology_ai999x.solve('bio_tm_nearest_neighbor', seq='CGTTCCAAAGATGTGGGCATGAGCTTAC', Na=50, dnac1=25,
dnac2=25)
# -> tm = 60.3209   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_tm_nearest_neighbor', seq='GCGAATTCGC', Na=100, dnac1=500, dnac2=0)
# -> tm = 39.5273   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nearest-neighbour melting temperature: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50, dnac1 = 25, dnac2 = 25	SOLVED	tm = 60.3209
Nearest-neighbour melting temperature: seq = GCGAATTCGC, Na = 100, dnac1 = 500, dnac2 = 0	SOLVED	tm = 39.5273
Nearest-neighbour melting temperature: Na = 50, dnac1 = 25, dnac2 = 25	NEEDS_INPUT	bio_tm_nearest_neighbor needs seq (DNA sequence 5'->3'). Example: solve('bio_tm_nearest_neighbor', seq='CGTTCCAAAGATGTGG
Nearest-neighbour melting temperature: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50, dnac1 = 25, dnac2 = 25, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Nearest-neighbour melting temperature: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50, dnac1 = 25, dnac2 = 25 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Global alignment score (Needleman-Wunsch, linear gaps), `bio_global_alignment_score`

The optimal global (Needleman-Wunsch) alignment score of two sequences with stated match, mismatch and linear gap scores.

$$S_{i,j} = \max(S_{i-1,j-1} + s(x_i, y_j),\ S_{i-1,j} + g,\ S_{i,j-1} + g)$$

input	meaning	condition
`seq1`	first sequence	
`seq2`	second sequence	
`match`	score of a match	
`mismatch`	score of a mismatch	
`gap`	score of each gap position (<= 0)	

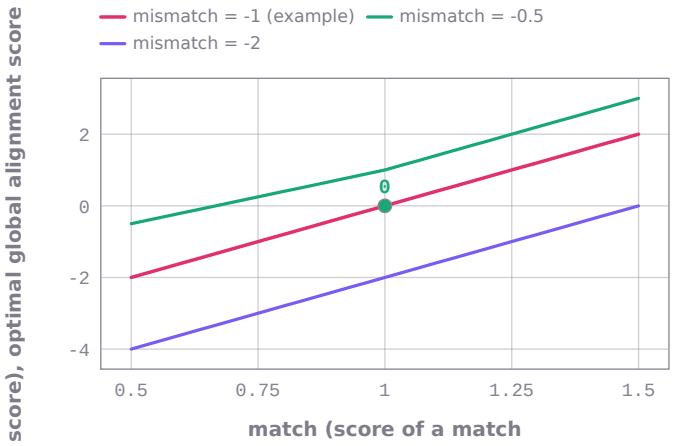
output	meaning
`score`	optimal global alignment score

- **Method:** dynamic programming over prefixes (Needleman-Wunsch)
- **Assumes:** gap <= 0; end gaps scored like internal gaps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	–	G	C	A	T	G	C	U
–	0	-1	-2	-3	-4	-5	-6	-7
G	-1	1	0	-1	-2	-3	-4	-5
A	-2	0	0	1	0	-1	-2	-3
T	-3	-1	-1	0	2	1	0	-1
T	-4	-2	-2	-1	1	1	0	-1
A	-5	-3	-3	-1	0	0	0	-1
C	-6	-4	-2	-2	-1	-1	1	0
A	-7	-5	-3	-1	-2	-2	0	0

optimal global score = 0

Needleman–Wunsch table for GATTACA against GCATGCU (match 1, mismatch -1, gap -1): cell (i, j) is the package's optimal global score of the first i letters of GATTACA and the first j of GCATGCU (49 runs); the top row and left column are the all-gap scores j·gap and i·gap. The outlined cells trace one optimal alignment back through cells that satisfy the recurrence. Marked: score = 0 (bottom right).



score (optimal global alignment score) against match (score of a match) from 0.5 to 1.5. One curve for each value of mismatch (score of a mismatch): -1, -0.5, -2. Other inputs as in the worked example: seq1 = GATTACA, seq2 = GCATGCU, gap = -1.0. 31 points per curve computed by computational-biology-ai999x; marked: the worked example, match = 1, score = 0.

```
computational_biology_ai999x.solve('bio_global_alignment_score', seq1='GATTACA', seq2='GCATGCU', match=1, mismatch=-1, gap=-1)
# -> score = 0   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_global_alignment_score', seq1='ACGT', seq2='ACT', match=2, mismatch=-1, gap=-2)
# -> score = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Global alignment score: seq1 = GATTACA, seq2 = GCATGCU, match = 1, mismatch = -1, gap = -1	SOLVED	score = 0
Global alignment score: seq1 = ACGT, seq2 = ACT, match = 2, mismatch = -1, gap = -2	SOLVED	score = 4
Global alignment score: seq2 = GCATGCU, match = 1, mismatch = -1, gap = -1	NEEDS_INPUT	bio_global_alignment_score needs seq1 (first sequence). Example: solve('bio_global_alignment_score', seq1='GATTACA', seq
Global alignment score: seq1 = GATTACA, seq2 = GCATGCU, match = 1, mismatch = -1, gap = -1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Global alignment score: seq1 = GATTACA, seq2 = GCATGCU, match = 1, mismatch = -1, gap = -1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Global alignment score with affine gaps (Gotoh), `bio_global_alignment_affine`

The optimal global alignment score with affine gaps: a gap of length L scores open + (L-1)*extend.

$$g(L) = o + (L-1)\,e$$

input	meaning	condition
`seq1`	first sequence	
`seq2`	second sequence	
`match`	score of a match	
`mismatch`	score of a mismatch	
`open`	score of the first position of a gap (<= 0)	
`extend`	score of each further gap position (<= 0)	

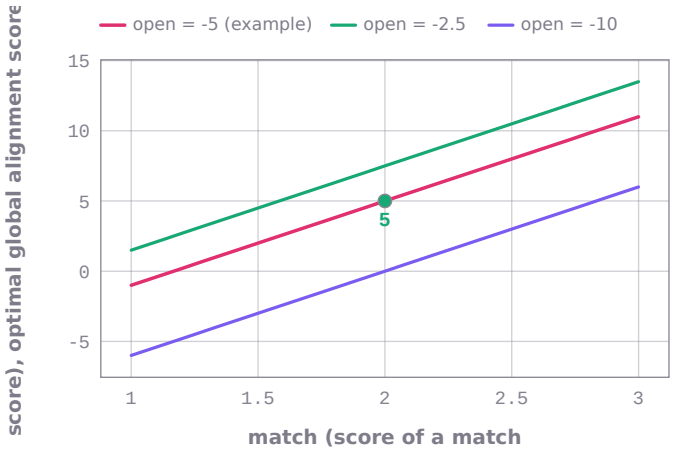
output	meaning
`score`	optimal global alignment score

- **Method:** Gotoh's three-state dynamic programming
- **Assumes:** open, extend <= 0; a gap of length L scores open + (L-1) extend; end gaps scored
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

		A	C	G	A	C	G
A		2	-3	-4	-5	-6	-7
C		-3	4	-1	-2	-3	-4
G		-4	-1	6	1	0	-1
T		-5	-2	1	5	0	-1
T		-6	-3	0	0	4	-1
T		-7	-4	-1	-1	-1	3
A		-8	-5	-2	1	-2	-2
C		-9	-6	-3	-3	3	-2
G		-10	-7	-4	-4	-2	5

`optimal score = 5`

Global alignment score with affine gaps (Gotoh) of ACGTTTACG against ACGACG (match 2, mismatch -1, open -5, extend -1) as a table: cell (i, j) is the package's optimal global score of the first i letters of ACGTTTACG and the first j letters of ACGACG (54 runs). The bottom-right cell is the returned score 5. Marked: score = 5.



score (optimal global alignment score) against match (score of a match) from 1 to 3. One curve for each value of open (score of the first position of a gap (<= 0)): -5, -2.5, -10. Other inputs as in the worked example: seq1 = ACGTTTACG, seq2 = ACGACG, mismatch = -1.0, extend = -1.0. 31 points per curve computed by computational-biology-ai999x; marked: the worked example, match = 2, score = 5.

```
computational_biology_ai999x.solve('bio_global_alignment_affine', seq1='ACGTTTACG', seq2='ACGACG', match=2,
mismatch=-1, open=-5, extend=-1)
# -> score = 5   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_global_alignment_affine', seq1='AAAA', seq2='AA', match=1, mismatch=-1,
open=-2, extend=-0.5)
# -> score = -0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Global alignment score with affine gaps: seq1 = ACGTTTACG, seq2 = ACGACG, match = 2, mismatch = -1, open = -5, extend = -1	SOLVED	score = 5
Global alignment score with affine gaps: seq1 = AAAA, seq2 = AA, match = 1, mismatch = -1, open = -2, extend = -0.5	SOLVED	score = -0.5
Global alignment score with affine gaps: seq2 = ACGACG, match = 2, mismatch = -1, open = -5, extend = -1	NEEDS_INPUT	bio_global_alignment_affine needs seq1 (first sequence). Example: solve('bio_global_alignment_affine', seq1='ACGTTTACG',
Global alignment score with affine gaps: seq1 = ACGTTTACG, seq2 = ACGACG, match = 2, mismatch = -1, open = -5, extend = -1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Global alignment score with affine gaps: seq1 = ACGTTTACG, seq2 = ACGACG, match = 2, mismatch = -1, open = -5, extend = -1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Local alignment score (Smith-Waterman, linear gaps), `bio_local_alignment_score`

The optimal local (Smith-Waterman) alignment score of two sequences with stated match, mismatch and linear gap scores.

$$S_{i,j} = \max(0,\ S_{i-1,j-1} + s(x_i, y_j),\ S_{i-1,j} + g,\ S_{i,j-1} + g)$$

input	meaning	condition
`seq1`	first sequence	
`seq2`	second sequence	
`match`	score of a match	
`mismatch`	score of a mismatch	
`gap`	score of each gap position (<= 0)	

output	meaning
`score`	optimal local alignment score (>= 0)

- **Method:** dynamic programming with a floor at 0 (Smith-Waterman)
- **Assumes:** match > 0; gap <= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

		G	G	T	T	G	A	C	T	A
	T	0	0	3	3	3	3	3	3	3
	G	3	3	3	3	6	6	6	6	6
	T	3	3	6	6	6	6	6	6	6
	T	3	3	6	9	9	9	9	9	9
	A	3	3	6	9	9	10	10	10	10
	C	3	3	6	9	9	10	13	13	13
	G	3	3	6	9	9	10	13	13	13
	G	3	6	6	9	9	10	13	13	13

optimal score = 13

Local alignment score (Smith-Waterman, linear gaps) of TGTTACGG against GGTTGACTA (match 3, mismatch -3, gap -2) as a table: cell (i, j) is the package's best local alignment score found within the first i letters of TGTTACGG and the first j letters of GGTTGACTA (72 runs). The outlined cell is the first where the returned optimum 13 is reached: where the best local alignment ends (row 6, column 7). Marked: score = 13.



score (optimal local alignment score (>= 0)) against match (score of a match) from 1.5 to 4.5. One curve for each value of gap (score of each gap position (<= 0)): -2, -1, -4. Other inputs as in the worked example: seq1 = TGTTACGG, seq2 = GGTTGACTA, mismatch = -3.0. 31 points per curve computed by computational-biology-ai999x; marked: the worked example, match = 3, score = 13.

```
computational_biology_ai999x.solve('bio_local_alignment_score', seq1='TGTTACGG', seq2='GGTTGACTA', match=3,
mismatch=-3, gap=-2)
# -> score = 13   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_local_alignment_score', seq1='AAAA', seq2='CCCC', match=1, mismatch=-1, gap=-1)
# -> score = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Local alignment score: seq1 = TGTTACGG, seq2 = GGTTGACTA, match = 3, mismatch = -3, gap = -2	SOLVED	score = 13
Local alignment score: seq1 = AAAA, seq2 = CCCC, match = 1, mismatch = -1, gap = -1	SOLVED	score = 0
Local alignment score: seq2 = GGTTGACTA, match = 3, mismatch = -3, gap = -2	NEEDS_INPUT	bio_local_alignment_score needs seq1 (first sequence). Example: solve('bio_local_alignment_score', seq1='TGTTACGG', seq2
Local alignment score: seq1 = TGTTACGG, seq2 = GGTTGACTA, match = 3, mismatch = -3, gap = -2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Local alignment score: seq1 = TGTTACGG, seq2 = GGTTGACTA, match = 3, mismatch = -3, gap = -2 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Local alignment score with affine gaps, `bio_local_alignment_affine`

The optimal local alignment score with affine gaps (a gap of length L scores open + (L-1)*extend).

$$g(L) = o + (L-1)\,e, \quad S \geq 0$$

input	meaning	condition
`seq1`	first sequence	
`seq2`	second sequence	
`match`	score of a match	
`mismatch`	score of a mismatch	
`open`	score of the first position of a gap (<= 0)	
`extend`	score of each further gap position (<= 0)	

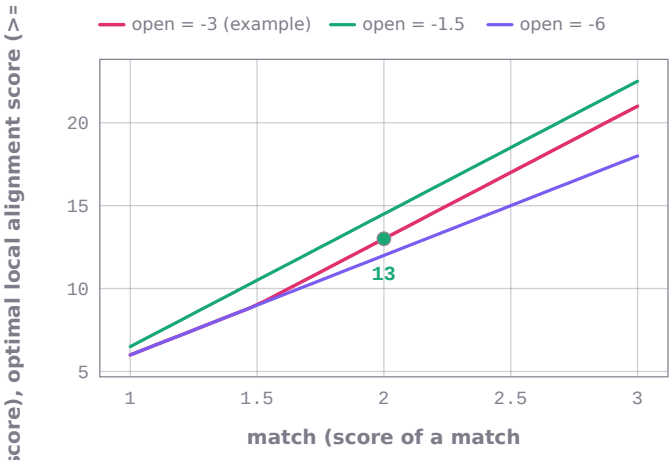
output	meaning
`score`	optimal local alignment score (>= 0)

- **Method:** Gotoh's recursion with a floor at 0
- **Assumes:** match > 0; open, extend <= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

		G	G	A	C	G	T	T	A	C	G	T	G	G
	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	0	0	0	0	0	2	2	2	2	2	2	2	2	2
T	0	0	0	0	0	2	4	4	4	4	4	4	4	4
A	0	0	2	2	2	2	4	6	6	6	6	6	6	6
C	0	0	2	4	4	4	4	6	8	8	8	8	8	8
G	2	2	2	4	6	6	6	6	8	10	10	10	10	10
T	2	2	2	4	6	8	8	8	8	10	12	12	12	12
A	2	2	3	4	6	8	8	8	8	10	12	12	12	12
C	2	2	3	5	6	8	8	8	9	10	12	12	12	12
G	2	2	3	5	7	8	8	8	9	11	12	12	12	12
T	2	2	3	5	7	9	9	9	9	11	13	13	13	13
T	2	2	3	5	7	9	11	11	11	11	13	13	13	13
T	2	2	3	5	7	9	11	11	11	11	13	13	13	13

optimal score = 13

Local alignment score with affine gaps of TTACGTACGTTT against GGACGTTACGTGG (match 2, mismatch -1, open -3, extend -1) as a table: cell (i, j) is the package's best local alignment score found within the first i letters of TTACGTACGTTT and the first j letters of GGACGTTACGTGG (156 runs). The outlined cell is the first where the returned optimum 13 is reached: where the best local alignment ends (row 10, column 11). Marked: score = 13.



score (optimal local alignment score (>= 0)) against match (score of a match) from 1 to 3. One curve for each value of open (score of the first position of a gap (<= 0)): -3, -1.5, -6. Other inputs as in the worked example: seq1 = TTACGTACGTTT, seq2 = GGACGTTACGTGG, mismatch = -1.0, extend = -1.0. 31 points per curve computed by computational-biology-ai999x; marked: the worked example, match = 2, score = 13.

```
computational_biology_ai999x.solve('bio_local_alignment_affine', seq1='TTACGTACGTTT', seq2='GGACGTTACGTGG', match=2,
mismatch=-1, open=-3, extend=-1)
# -> score = 13   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_local_alignment_affine', seq1='AAAA', seq2='CCCC', match=1, mismatch=-1,
open=-2, extend=-1)
# -> score = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Local alignment score with affine gaps: seq1 = TTACGTACGTTT, seq2 = GGACGTTACGTGG, match = 2, mismatch = -1, open = -3, extend = -1	SOLVED	score = 13
Local alignment score with affine gaps: seq1 = AAAA, seq2 = CCCC, match = 1, mismatch = -1, open = -2, extend = -1	SOLVED	score = 0
Local alignment score with affine gaps: seq2 = GGACGTTACGTGG, match = 2, mismatch = -1, open = -3, extend = -1	NEEDS_INPUT	bio_local_alignment_affine needs seq1 (first sequence). Example: solve('bio_local_alignment_affine', seq1='TTACGTACGTTT'
Local alignment score with affine gaps: seq1 = TTACGTACGTTT, seq2 = GGACGTTACGTGG, match = 2, mismatch = -1, open = -3, extend = -1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Local alignment score with affine gaps: seq1 = TTACGTACGTTT, seq2 = GGACGTTACGTGG, match = 2, mismatch = -1, open = -3, extend = -1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Global protein alignment score (BLOSUM62, affine gaps), `bio_protein_global_blosum62`

The optimal global alignment score of two proteins with BLOSUM62 and affine gaps.

$$S = \max_{\text{alignments}} \sum B_{62}(x, y) + \sum_{\text{gaps}} [o + (L-1)e]$$

input	meaning	condition
`seq1`	first protein sequence	
`seq2`	second protein sequence	
`open`	score of the first position of a gap (<= 0)	
`extend`	score of each further gap position (<= 0)	

output	meaning
`score`	optimal global alignment score

- **Method:** Gotoh's dynamic programming with BLOSUM62 substitution scores
- **Assumes:** standard amino acids; open, extend <= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

		P	A	W	H	E	A	E
		-2	-12	-13	-4	-13	-15	-15
H		-11	-3	-13	-13	1	-9	-10
E		-12	-7	-6	-15	-9	5	-5
A		-14	-12	-9	-8	-10	-5	3
G		-14	-10	-15	-11	-9	-6	-6
A		-16	-16	1	-9	-10	-7	-8
W		-17	-16	-9	-1	-11	-8	-9
G		-18	-18	-10	-1	-1	-9	-8
H		-18	-19	-11	-10	4	-2	-4
E		-19	-19	-12	-11	-5	3	3
E								

`optimal score = 3`

Global protein alignment score (BLOSUM62, affine gaps) of HEAGAWGHEE against PAWHEAE (BLOSUM62 substitution scores, open -10, extend -1) as a table: cell (i, j) is the package's optimal global score of the first i letters of HEAGAWGHEE and the first j letters of PAWHEAE (70 runs). The bottom-right cell is the returned score 3. Marked: score = 3.



score (optimal global alignment score) against open (score of the first position of a gap (<= 0)) from -15 to -5. One curve for each value of extend (score of each further gap position (<= 0)): -1, -0.5, -2. Other inputs as in the worked example: seq1 = HEAGAWGHEE, seq2 = PAWHEAE. 31 points per curve computed by computational-biology-ai999x; marked: the worked example, open = -10, score = 3.

```
computational_biology_ai999x.solve('bio_protein_global_blosum62', seq1='HEAGAWGHEE', seq2='PAWHEAE', open=-10,
extend=-1)
# -> score = 3   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_protein_global_blosum62', seq1='MKTAYIAK', seq2='MKTAYIAK', open=-11,
extend=-1)
# -> score = 39   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Global protein alignment score: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, open = -10, extend = -1	SOLVED	score = 3
Global protein alignment score: seq1 = MKTAYIAK, seq2 = MKTAYIAK, open = -11, extend = -1	SOLVED	score = 39
Global protein alignment score: seq2 = PAWHEAE, open = -10, extend = -1	NEEDS_INPUT	bio_protein_global_blosum62 needs seq1 (first protein sequence). Example: solve('bio_protein_global_blosum62', seq1='HEA
Global protein alignment score: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, open = -10, extend = -1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Global protein alignment score: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, open = -10, extend = -1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Local protein alignment score (BLOSUM62, affine gaps), `bio_protein_local_blosum62`

The optimal local (Smith-Waterman) alignment score of two proteins with BLOSUM62 and affine gaps.

$$S = \max\left(0, \max_{\text{local alignments}} \sum B_{62} + \sum[o + (L-1)e]\right)$$

input	meaning	condition
`seq1`	first protein sequence	
`seq2`	second protein sequence	
`open`	score of the first position of a gap (<= 0)	
`extend`	score of each further gap position (<= 0)	

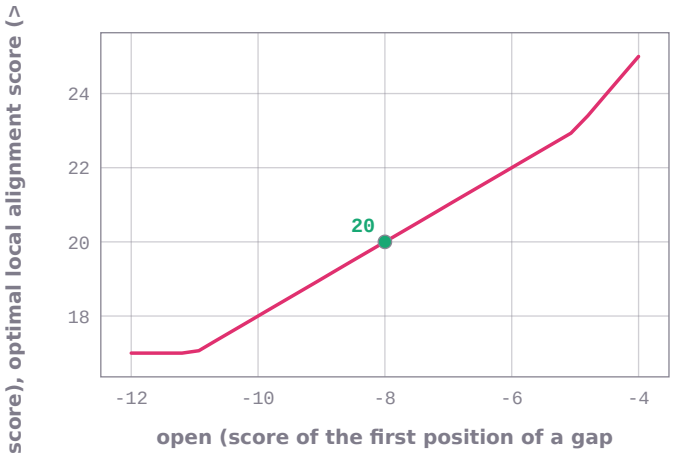
output	meaning
`score`	optimal local alignment score (>= 0)

- **Method:** Gotoh's recursion with BLOSUM62 scores and a floor at 0
- **Assumes:** standard amino acids; open, extend <= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	P	A	W	H	E	A	E
H	0	0	0	8	8	8	8
E	0	0	0	8	13	13	13
A	0	4	4	8	13	17	17
G	0	4	4	8	13	17	17
A	0	4	4	8	13	17	17
W	0	4	15	15	15	17	17
G	0	4	15	15	15	17	17
H	0	4	15	15	15	17	17
E	0	4	15	15	**20**	20	20
E	0	4	15	15	20	20	20

optimal score = 20

Local protein alignment score (BLOSUM62, affine gaps) of HEAGAWGHEE against PAWHEAE (BLOSUM62 substitution scores, open -8, extend -8) as a table: cell (i, j) is the package's best local alignment score found within the first i letters of HEAGAWGHEE and the first j letters of PAWHEAE (70 runs). The outlined cell is the first where the returned optimum 20 is reached: where the best local alignment ends (row 9, column 5). Marked: score = 20.



score (optimal local alignment score (>= 0)) against open (score of the first position of a gap (<= 0)) from -12 to -4. Other inputs as in the worked example: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, extend = -8.0. 31 points computed by computational-biology-ai999x; marked: the worked example, open = -8, score = 20.

```
computational_biology_ai999x.solve('bio_protein_local_blosum62', seq1='HEAGAWGHEE', seq2='PAWHEAE', open=-8, extend=-8)
# -> score = 20   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_protein_local_blosum62', seq1='WWWW', seq2='WW', open=-11, extend=-1)
# -> score = 22   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Local protein alignment score: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, open = -8, extend = -8	SOLVED	score = 20
Local protein alignment score: seq1 = WWWW, seq2 = WW, open = -11, extend = -1	SOLVED	score = 22
Local protein alignment score: seq2 = PAWHEAE, open = -8, extend = -8	NEEDS_INPUT	bio_protein_local_blosum62 needs seq1 (first protein sequence). Example: solve('bio_protein_local_blosum62', seq1='HEAGA
Local protein alignment score: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, open = -8, extend = -8, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Local protein alignment score: seq1 = HEAGAWGHEE, seq2 = PAWHEAE, open = -8, extend = -8 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Semi-global (end-gap-free) alignment score, `bio_semiglobal_alignment_score`

The optimal alignment score when gaps at either end of either sequence are free (fitting a read inside a longer sequence, overlaps), with linear internal gaps.

$$S = \max\left(\max_j S_{n,j},\ \max_i S_{i,m}\right),\ S_{i,0} = S_{0,j} = 0$$

input	meaning	condition
`seq1`	first sequence	
`seq2`	second sequence	
`match`	score of a match	
`mismatch`	score of a mismatch	
`gap`	score of each internal gap position (<= 0)	

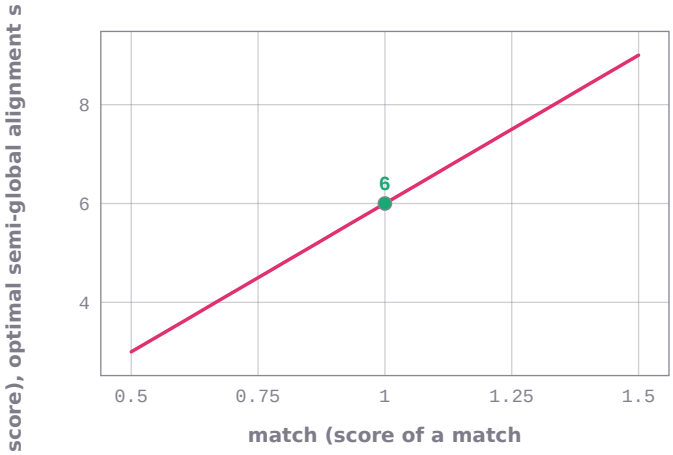
output	meaning
`score`	optimal semi-global alignment score

- **Method:** Needleman-Wunsch with a zero first row and column and the best score on the last row or column
- **Assumes:** gap <= 0; end gaps unscored
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

		A	C	G	T	A	C
		0	0	0	1	1	1
T		0	0	0	1	0	0
T		0	0	0	1	0	0
T		0	0	0	1	0	0
T		0	0	0	1	0	0
A		1	1	1	1	1	1
C		1	2	2	2	2	2
G		1	2	3	3	3	3
T		1	2	3	4	4	4
A		1	2	3	4	5	5
C		1	2	3	4	5	6
G		1	2	3	4	5	6
G		1	2	3	4	5	6
G		1	2	3	4	5	6
G		1	2	3	4	5	6

optimal score = 6

Semi-global (end-gap-free) alignment score of TTTTACGTACGGGG against ACGTAC (match 1, mismatch -1, gap -1) as a table: cell (i, j) is the package's semi-global score (end gaps free) of the first i letters of TTTTACGTACGGGG and the first j letters of ACGTAC (84 runs). The bottom-right cell is the returned score 6. Marked: score = 6.



score (optimal semi-global alignment score) against match (score of a match) from 0.5 to 1.5. Other inputs as in the worked example: seq1 = TTTTACGTACGGGG, seq2 = ACGTAC, mismatch = -1.0, gap = -1.0. 31 points computed by computational-biology-ai999x; marked: the worked example, match = 1, score = 6.

```
computational_biology_ai999x.solve('bio_semiglobal_alignment_score', seq1='TTTTACGTACGGGG', seq2='ACGTAC', match=1,
mismatch=-1, gap=-1)
# -> score = 6   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_semiglobal_alignment_score', seq1='ACGT', seq2='TTTT', match=1, mismatch=-1,
gap=-2)
# -> score = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Semi-global alignment score: seq1 = TTTTACGTACGGGG, seq2 = ACGTAC, match = 1, mismatch = -1, gap = -1	SOLVED	score = 6
Semi-global alignment score: seq1 = ACGT, seq2 = TTTT, match = 1, mismatch = -1, gap = -2	SOLVED	score = 1
Semi-global alignment score: seq2 = ACGTAC, match = 1, mismatch = -1, gap = -1	NEEDS_INPUT	bio_semiglobal_alignment_score needs seq1 (first sequence). Example: solve('bio_semiglobal_alignment_score', seq1='TTTTA

question	outcome	result
Semi-global alignment score: seq1 = TTTTACGTACGGGG, seq2 = ACGTAC, match = 1, mismatch = -1, gap = -1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Semi-global alignment score: seq1 = TTTTACGTACGGGG, seq2 = ACGTAC, match = 1, mismatch = -1, gap = -1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Probability of an offspring genotype in a cross, `bio_cross_genotype_probability`

The probability that an offspring of two parents has a stated genotype (monohybrid, dihybrid or more loci; independent assortment).

$$P = \prod_{\ell} P_{\ell}(g_{\ell})$$

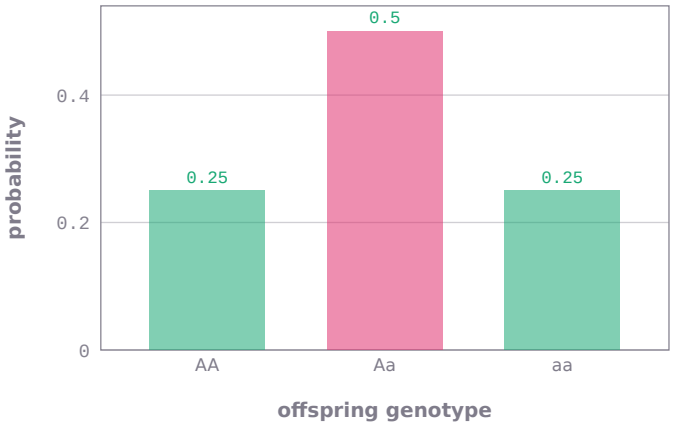
input	meaning	condition
`parent1`	genotype of the first parent, letter pairs (e.g. AaBb)	
`parent2`	genotype of the second parent	
`offspring`	offspring genotype asked about	

output	meaning
`probability`	probability of the offspring genotype
`fraction`	the same as a fraction

- **Method:** product over loci of the Punnett-square probability of each locus genotype
- **Assumes:** the same loci in the same order; independent assortment; at most 6 loci
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Punnett square of the cross Aa × Aa: the gametes of each parent along the edges (one allele of each gene, equally likely), the offspring genotype in each cell; cells of the asked genotype Aa are shaded. The package returns probability = 0.5 (1/2).



Probability of every offspring genotype of the worked example's cross Aa × Aa, each bar a run of the package: AA 0.25, Aa 0.5, aa 0.25; together they sum to 1. Highlighted: the asked genotype, Aa.

```
computational_biology_ai999x.solve('bio_cross_genotype_probability', parent1='Aa', parent2='Aa', offspring='Aa')
# -> probability = 0.5, fraction = 1/2   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_cross_genotype_probability', parent1='AaBb', parent2='AaBb', offspring='AaBB')
# -> probability = 0.125, fraction = 1/8   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_cross_genotype_probability', parent1='AaBbCc', parent2='aaBbCC',
offspring='aabbCc')
# -> probability = 0.0625, fraction = 1/16   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cross genotype probability: parent1 = Aa, parent2 = Aa, offspring = Aa	SOLVED	probability = 0.5, fraction = 1/2
Cross genotype probability: parent1 = AaBb, parent2 = AaBb, offspring = AaBB	SOLVED	probability = 0.125, fraction = 1/8
Cross genotype probability: parent1 = AaBbCc, parent2 = aaBbCC, offspring = aabbCc	SOLVED	probability = 0.0625, fraction = 1/16
Cross genotype probability: parent2 = Aa, offspring = Aa	NEEDS_INPUT	bio_cross_genotype_probability needs parent1 (genotype of the first parent, letter pairs (e.g. AaBb)). Example: solve('b
Cross genotype probability: parent1 = Aa, parent2 = Aa, offspring = Aa, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Cross genotype probability: parent1 = Aa, parent2 = Aa, offspring = Aa and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Cross genotype probability: parent1 = AaBb, parent2 = Aa, offspring = Aa	INVALID_INPUT	Probability of an offspring genotype in a cross: every genotype must list the same loci in the same order.

Phenotype ratio of a cross (complete dominance), `bio_cross_phenotype_ratio`

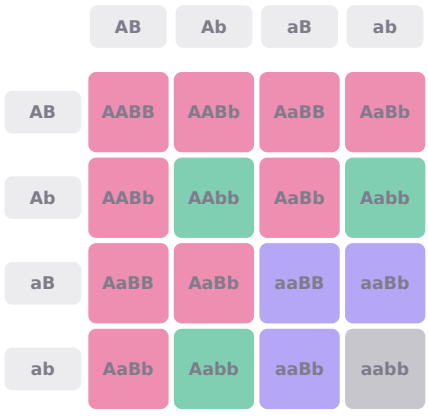
The offspring phenotype ratio of a cross with complete dominance, the probabilities of the all-dominant and all-recessive phenotypes and the numbers of genotypes and phenotypes.

$$(\tfrac{3}{4}D + \tfrac{1}{4}r)^n \Rightarrow 9:3:3:1 \text{ for two loci}$$

input	meaning	condition
`parent1`	genotype of the first parent, letter pairs (e.g. AaBb)	
`parent2`	genotype of the second parent	

output	meaning
`p_all_dominant`	probability of showing every dominant trait
`p_all_recessive`	probability of showing every recessive trait
`n_genotypes`	number of distinct offspring genotypes
`n_phenotypes`	number of distinct offspring phenotypes
`ratio`	phenotype ratio, largest class first (e.g. 9:3:3:1)

- **Method:** per-locus phenotype probabilities (3/4 : $\mathbf{1/4}$ for Aa x Aa) multiplied across loci
- **Assumes:** complete dominance at every locus; independent assortment
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Punnett square of AaBb × AaBb, each offspring cell shaded by its phenotype class (A_ means at least one dominant A, aa recessive; classes and their cell counts read from the square). The package returns ratio = 9:3:3:1, 9 genotypes, 4 phenotypes, p_all_dominant = 0.5625, p_all_recessive = 0.0625.

```
computational_biology_ai999x.solve('bio_cross_phenotype_ratio', parent1='AaBb', parent2='AaBb')
# -> p_all_dominant = 0.5625, p_all_recessive = 0.0625, n_genotypes = 9, n_phenotypes = 4, ratio = 9:3:3:1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_cross_phenotype_ratio', parent1='Aa', parent2='aa')
# -> p_all_dominant = 0.5, p_all_recessive = 0.5, n_genotypes = 2, n_phenotypes = 2, ratio = 1:1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_cross_phenotype_ratio', parent1='AaBbCc', parent2='AaBbCc')
# -> p_all_dominant = 0.421875, p_all_recessive = 0.015625, n_genotypes = 27, n_phenotypes = 8, ratio =
27:9:9:9:3:3:3:1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cross phenotype ratio: parent1 = AaBb, parent2 = AaBb	SOLVED	p_all_dominant = 0.5625, p_all_recessive = 0.0625, n_genotypes = 9, n_phenotypes = 4, ratio = 9:3:3:1
Cross phenotype ratio: parent1 = Aa, parent2 = aa	SOLVED	p_all_dominant = 0.5, p_all_recessive = 0.5, n_genotypes = 2, n_phenotypes = 2, ratio = 1:1
Cross phenotype ratio: parent1 = AaBbCc, parent2 = AaBbCc	SOLVED	p_all_dominant = 0.421875, p_all_recessive = 0.015625, n_genotypes = 27, n_phenotypes = 8, ratio = 27:9:9:9:3:3:3:1
Cross phenotype ratio: parent2 = AaBb	NEEDS_INPUT	bio_cross_phenotype_ratio needs parent1 (genotype of the first parent, letter pairs (e.g. AaBb)). Example: solve('bio_cr
Cross phenotype ratio: parent1 = AaBb, parent2 = AaBb, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Cross phenotype ratio: parent1 = AaBb, parent2 = AaBb and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Hardy-Weinberg chi-square test, `bio_hardy_weinberg_test`

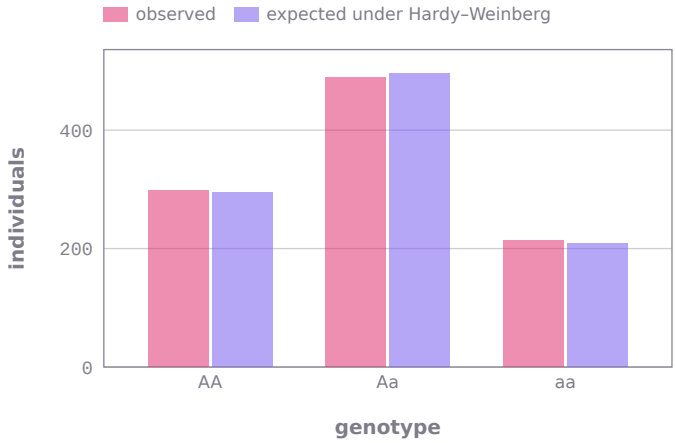
Allele frequencies, Hardy-Weinberg expected genotype counts, the chi-square test of equilibrium (1 degree of freedom), observed and expected heterozygosity and the inbreeding coefficient F.

$$\chi^2 = \sum \frac{(O-E)^2}{E}, \quad E = (Np^2,\ 2Npq,\ Nq^2)$$

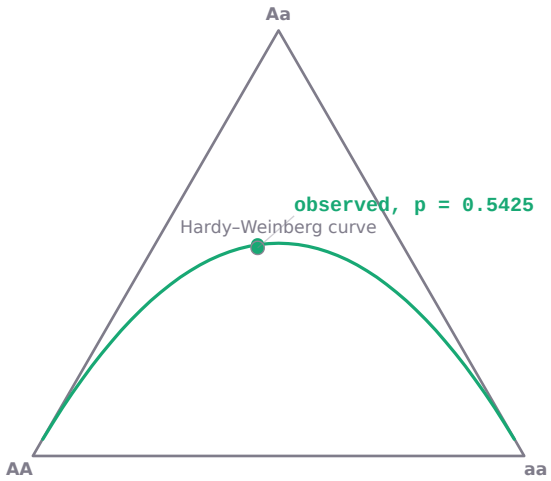
input	meaning	condition
`n_AA`	number of AA individuals	
`n_Aa`	number of Aa individuals	
`n_aa`	number of aa individuals	

output	meaning
`p`	frequency of allele A
`q`	frequency of allele a
`exp_AA`	expected AA count (N p^2)
`exp_Aa`	expected Aa count (2 N p q)
`exp_aa`	expected aa count (N q^2)
`chi2`	chi-square statistic
`p_value`	p-value (chi-square, 1 degree of freedom)
`Ho`	observed heterozygosity
`He`	expected heterozygosity 2pq
`F`	inbreeding coefficient $1 - Ho/He$

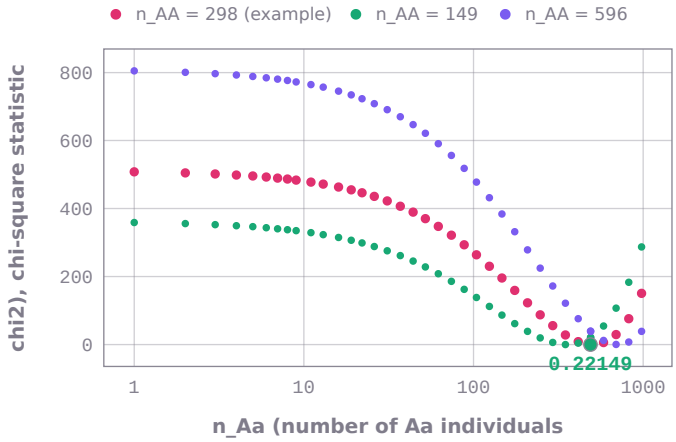
- **Method:** allele counting, expected counts N p^2, 2Npq, N q^2, chi-square with $3 - 1 - 1 = 1$ degree of freedom
- **Assumes:** both alleles present; biallelic locus; no continuity correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Observed genotype counts AA = 298, Aa = 489, aa = 213 beside the counts expected under Hardy–Weinberg equilibrium, computed by the package from the allele frequencies p = 0.5425, q = 0.4575: 294.306, 496.387, 209.306. χ^2 = 0.22149, p-value 0.637907; observed heterozygosity 0.489 against expected 0.496387 (F = 0.0148825).



De Finetti diagram: a population is a point of the triangle whose distances to the sides are its genotype frequencies (AA, Aa, aa at the corners). The curve joins the Hardy–Weinberg proportions the package returns as expected counts (populations of 1000 at 51 allele frequencies). Marked: the worked example AA = 298, Aa = 489, aa = 213 (observed frequencies, counts / 1000) beside its expected point (exp_AA 294.3, exp_Aa 496.4, exp_aa 209.3); χ^2 = 0.2215, p-value 0.6379.



chi2 (chi-square statistic) against n_Aa (number of Aa individuals) from 1 to 978 on a logarithmic axis. One curve for each value of n_AA (number of AA individuals): 298, 149, 596. Other inputs as in the worked example: n_aa = 213. 37 points per curve computed by computational-biology-ai999x; marked: the worked example, n_Aa = 489, chi2 = 0.22149.

```
computational_biology_ai999x.solve('bio_hardy_weinberg_test', n_AA=298, n_Aa=489, n_aa=213)
# -> p = 0.5425, q = 0.4575, exp_AA = 294.306, exp_Aa = 496.387, exp_aa = 209.306, chi2 = 0.22149, p_value = 0.637907,
Ho = 0.489, He = 0.496387, F = 0.0148825   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hardy_weinberg_test', n_AA=50, n_Aa=0, n_aa=50)
# -> p = 0.5, q = 0.5, exp_AA = 25, exp_Aa = 50, exp_aa = 25, chi2 = 100, p_value = 1.52397e-23, Ho = 0, He = 0.5, F =
1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hardy-Weinberg test: n_AA = 298, n_Aa = 489, n_aa = 213	SOLVED	p = 0.5425, q = 0.4575, exp_AA = 294.306, exp_Aa = 496.387, exp_aa = 209.306, chi2 = 0.22149, p_value = 0.637907, Ho = 0.489, He = 0.496387, F = 0.0148825
Hardy-Weinberg test: n_AA = 50, n_Aa = 0, n_aa = 50	SOLVED	p = 0.5, q = 0.5, exp_AA = 25, exp_Aa = 50, exp_aa = 25, chi2 = 100, p_value = 1.52397e-23, Ho = 0, He = 0.5, F = 1
Hardy-Weinberg test: n_Aa = 489, n_aa = 213	NEEDS_INPUT	bio_hardy_weinberg_test needs n_AA (number of AA individuals). Example: solve('bio_hardy_weinberg_test', n_AA=298, n_Aa=
Hardy-Weinberg test: n_AA = 298, n_Aa = 489, n_aa = 213, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hardy-Weinberg test: n_AA = 298, n_Aa = 489, n_aa = 213 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hardy-Weinberg test: n_AA = 10, n_Aa = 0, n_aa = 0	INVALID_INPUT	Hardy-Weinberg chi-square test: both alleles must be present (otherwise the expected counts are zero).

Hardy-Weinberg exact test, `bio_hardy_weinberg_exact_test`

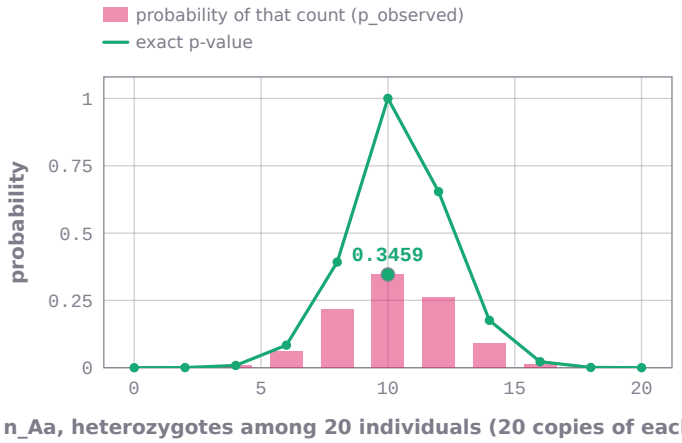
The exact (Wigginton) test of Hardy-Weinberg equilibrium for a biallelic locus: the two-sided p-value and the probability of the observed number of heterozygotes given the allele counts.

$$P(n_{AB} \mid n_A) = \frac{N!}{n_{AA}!\, n_{AB}!\, n_{BB}!} \, \frac{2^{n_{AB}}\, n_A!\, n_B!}{(2N)!}$$

input	meaning	condition
`n_AA`	number of AA individuals	
`n_Aa`	number of Aa individuals	
`n_aa`	number of aa individuals	

output	meaning
`p_value`	exact two-sided p-value (sum of probabilities not above the observed one)
`p_observed`	probability of the observed heterozygote count

- **Method:** exact conditional distribution of heterozygote counts given the allele counts (rational arithmetic)
- **Assumes:** both alleles present; $N \leq 1000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Exact Hardy–Weinberg test for 20 individuals carrying 20 copies of each allele: for every possible number of heterozygotes 0, 2, …, 20 (the homozygotes split equally), the package's probability of that configuration (bars) and its two-sided exact p-value (dots). Too few or too many heterozygotes are both improbable. Marked: the most probable count, 10, 0.3459. The worked example (AA 2, Aa 0, aa 2) gives p-value 0.08571.

```
computational_biology_ai999x.solve('bio_hardy_weinberg_exact_test', n_AA=2, n_Aa=0, n_aa=2)
# -> p_value = 0.0857143, p_observed = 0.0857143   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hardy_weinberg_exact_test', n_AA=1, n_Aa=2, n_aa=1)
# -> p_value = 1, p_observed = 0.685714   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hardy-Weinberg exact test: n_AA = 2, n_Aa = 0, n_aa = 2	SOLVED	p_value = 0.0857143, p_observed = 0.0857143
Hardy-Weinberg exact test: n_AA = 1, n_Aa = 2, n_aa = 1	SOLVED	p_value = 1, p_observed = 0.685714
Hardy-Weinberg exact test: n_Aa = 0, n_aa = 2	NEEDS_INPUT	bio_hardy_weinberg_exact_test needs n_AA (number of AA individuals). Example: solve('bio_hardy_weinberg_exact_test', n_A
Hardy-Weinberg exact test: n_AA = 2, n_Aa = 0, n_aa = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hardy-Weinberg exact test: n_AA = 2, n_Aa = 0, n_aa = 2 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Allele frequency under haploid (genic) selection, `bio_haploid_selection`

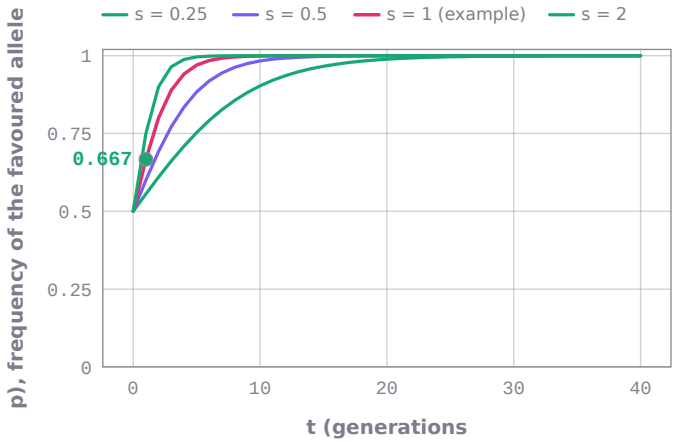
The frequency of allele A after t generations of haploid selection (relative fitness $1 + s$ against 1).

$$p' = \frac{p(1+s)}{1+sp}$$

input	meaning	condition
`p0`	initial frequency of A	
`s`	selection coefficient (fitness of A = 1 + s)	
`t`	number of generations	

output	meaning
`p_t`	frequency of A after t generations

- **Method:** iterate p' = p(1+s)/(1+sp) generation by generation
- **Assumes:** $0 < p_0 < 1$; $s > -1$; discrete generations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Spread of a favoured allele under haploid selection from the worked example's p0 = 0.5 for selection coefficients 0.25, 0.5, 1, 2 (its own s = 1 among them): each curve is p_t the package returns at 40 times up to 40 generations (the start p0 drawn at t = 0). Marked: the worked example, s = 1 after 1 generation, p = 0.6667.

```
computational_biology_ai999x.solve('bio_haploid_selection', p0=0.5, s=1, t=1)
# -> p_t = 0.666667   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_haploid_selection', p0=0.1, s=0.1, t=10)
# -> p_t = 0.223719   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Haploid selection: p0 = 0.5, s = 1, t = 1	SOLVED	p_t = 0.666667
Haploid selection: p0 = 0.1, s = 0.1, t = 10	SOLVED	p_t = 0.223719
Haploid selection: s = 1, t = 1	NEEDS_INPUT	bio_haploid_selection needs p0 (initial frequency of A). Example: solve('bio_haploid_selection', p0=0.5, s=1, t=1).
Haploid selection: p0 = 0.5, s = 1, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Haploid selection: p0 = 0.5, s = 1, t = 1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Decline of a recessive lethal allele, `bio_recessive_lethal`

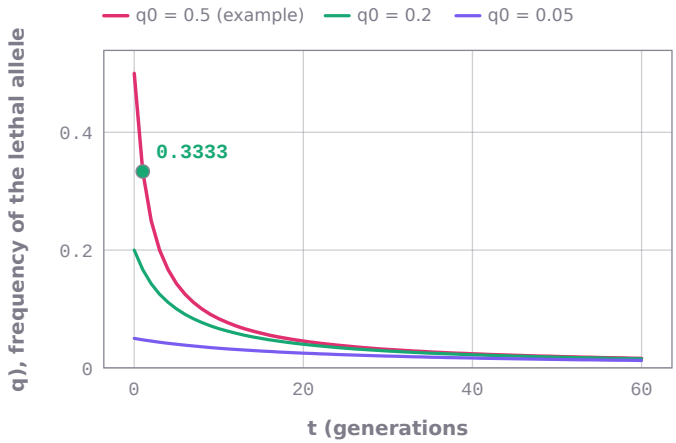
The frequency of a recessive lethal allele after t generations of selection (aa never reproduce).

$$q' = \frac{q}{1+q}$$

input	meaning	condition
`q0`	initial frequency of the lethal allele	
`t`	number of generations	

output	meaning
`q_t`	frequency after t generations

- **Method:** iterate q' = $q/(1+q)$
- **Assumes:** $0 < q_0 \leq 1$; random mating, aa fitness 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Decline of a recessive lethal allele, whose homozygotes die before breeding, from q0 = 0.5, 0.2 and 0.05: each curve is q_t the package returns for t = 1 … 60 (q0 drawn at t = 0). Rare alleles hide in heterozygotes and fall ever more slowly. Marked: the worked example q0 = 0.5, t = 1, q_t = 0.3333.

```
computational_biology_ai999x.solve('bio_recessive_lethal', q0=0.5, t=1)
# -> q_t = 0.333333   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_recessive_lethal', q0=0.1, t=10)
# -> q_t = 0.05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Recessive lethal allele frequency: q0 = 0.5, t = 1	SOLVED	q_t = 0.333333
Recessive lethal allele frequency: q0 = 0.1, t = 10	SOLVED	q_t = 0.05
Recessive lethal allele frequency: t = 1	NEEDS_INPUT	bio_recessive_lethal needs q0 (initial frequency of the lethal allele). Example: solve('bio_recessive_lethal', q0=0.5, t
Recessive lethal allele frequency: q0 = 0.5, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Recessive lethal allele frequency: q0 = 0.5, t = 1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Equilibrium under heterozygote advantage, `bio_heterozygote_advantage`

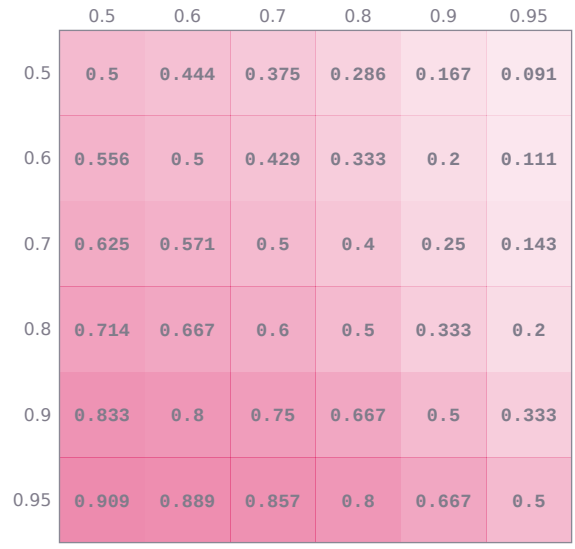
The stable equilibrium frequency of allele A when the heterozygote is fittest (overdominance).

$$\hat{p} = \frac{w_{Aa} - w_{aa}}{2w_{Aa} - w_{AA} - w_{aa}} = \frac{t}{s + t}$$

input	meaning	condition
`w_AA`	relative fitness of AA	
`w_Aa`	relative fitness of Aa	
`w_aa`	relative fitness of aa	

output	meaning
`p_eq`	equilibrium frequency of A

- **Method:** the interior fixed point of the selection recursion
- **Assumes:** $w_Aa > w_AA$ and $w_Aa > w_aa$; fitnesses positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



equilibrium frequency of A · rows w_AA · columns w_aa

Equilibrium frequency of allele A under heterozygote advantage (w_Aa = 1) for homozygote fitnesses w_AA (rows) and w_aa (columns) from 0.5 to 0.95, each cell a run of the package rounded to 3 decimals: A is common where AA is the fitter homozygote. The worked example w_AA = 0.8, w_aa = 0.6 gives p_eq = 0.6667.

```
computational_biology_ai999x.solve('bio_heterozygote_advantage', w_AA=0.8, w_Aa=1, w_aa=0.6)
# -> p_eq = 0.666667   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_heterozygote_advantage', w_AA=0.89, w_Aa=1, w_aa=0.2)
# -> p_eq = 0.879121   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Heterozygote advantage equilibrium: w_AA = 0.8, w_Aa = 1, w_aa = 0.6	SOLVED	p_eq = 0.666667
Heterozygote advantage equilibrium: w_AA = 0.89, w_Aa = 1, w_aa = 0.2	SOLVED	p_eq = 0.879121
Heterozygote advantage equilibrium: w_Aa = 1, w_aa = 0.6	NEEDS_INPUT	bio_heterozygote_advantage needs w_AA (relative fitness of AA). Example: solve('bio_heterozygote_advantage', w_AA=0.8, w
Heterozygote advantage equilibrium: w_AA = 0.8, w_Aa = 1, w_aa = 0.6, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Heterozygote advantage equilibrium: w_AA = 0.8, w_Aa = 1, w_aa = 0.6 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Heterozygote advantage equilibrium: w_AA = 1, w_Aa = 0.9, w_aa = 0.8	INVALID_INPUT	Equilibrium under heterozygote advantage: heterozygote advantage needs w_Aa greater than both homozygote fitnesses.

Mutation-selection balance, `bio_mutation_selection_balance`

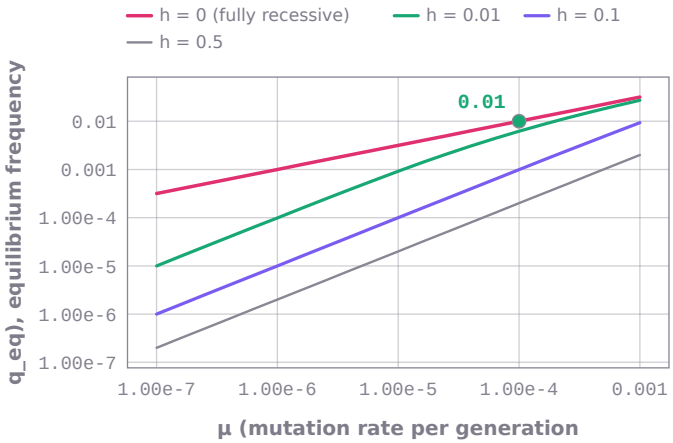
The equilibrium frequency of a deleterious allele maintained by recurrent mutation against selection (fitness 1, $1 - hs$, 1 - s), from the exact recursion.

$$\hat{q} \approx \sqrt{\mu/s}\ (h = 0), \quad \hat{q} \approx \frac{\mu}{hs}\ (h > 0)$$

input	meaning	condition
`mu`	mutation rate A -> a per generation	
`s`	selection coefficient against aa	
`h`	dominance coefficient (0 recessive, 0.5 additive)	

output	meaning
`q_eq`	equilibrium frequency of the deleterious allele

- **Method:** the root of q' - $q = 0$ for selection followed by mutation, by bisection (approximately sqrt(mu/s) for a recessive and mu/(hs) otherwise)
- **Assumes:** $0 < mu \leq 0.01$; $0.01 \leq s \leq 1$; $0 \leq h \leq 1$; selection then mutation, no back mutation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Equilibrium frequency of a deleterious allele between mutation and selection (s = 1) against the mutation rate, for dominance h = 0, 0.01, 0.1 and 0.5: each curve 21 runs of the package. A fully recessive allele persists at about √(μ/s), a partly dominant one near μ/(hs). Marked: the worked example μ = 0.0001, s = 1, h = 0, q_eq = 0.01.

```
computational_biology_ai999x.solve('bio_mutation_selection_balance', mu=0.0001, s=1, h=0)
# -> q_eq = 0.01   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_mutation_selection_balance', mu=1e-05, s=0.1, h=0.5)
# -> q_eq = 0.000199998   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mutation-selection balance: mu = 0.0001, s = 1, h = 0	SOLVED	q_eq = 0.01
Mutation-selection balance: mu = 1e-05, s = 0.1, h = 0.5	SOLVED	q_eq = 0.000199998
Mutation-selection balance: s = 1, h = 0	NEEDS_INPUT	bio_mutation_selection_balance needs mu (mutation rate A -> a per generation). Example: solve('bio_mutation_selection_ba
Mutation-selection balance: mu = 0.0001, s = 1, h = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Mutation-selection balance: mu = 0.0001, s = 1, h = 0 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Loss of heterozygosity by genetic drift, `bio_drift_heterozygosity`

The expected heterozygosity after t generations of genetic drift in a Wright-Fisher population of N diploids, starting from i copies of an allele.

$$H_t = H_0\left(1 - \frac{1}{2N}\right)^t$$

input	meaning	condition
`N`	number of diploid individuals	
`i`	initial copies of allele A (of 2N)	
`t`	number of generations	

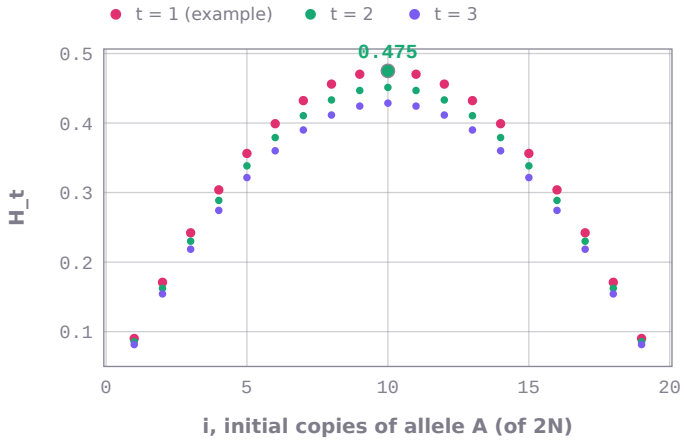
output	meaning
`H0`	initial heterozygosity 2pq
`H_t`	expected heterozygosity after t generations
`retained`	fraction of heterozygosity retained, (1 - 1/(2N))^t

- **Method:** expected heterozygosity falls by the factor $1 - 1/(2N)$ each generation
- **Assumes:** $0 < i < 2N$; constant N, random mating, no mutation or selection
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	1	2	3	4	5
1	0.349	0.072	0.005	0	0
2	0.599	0.277	0.077	0.006	0
3	0.817	0.603	0.364	0.133	0.018
4	0.904	0.778	0.605	0.366	0.134
5	0.951	0.882	0.778	0.606	0.367
6	0.98	0.951	0.905	0.819	0.67

retained · rows N = 5 → 250 · columns t = 10 → 200 generatio

Fraction of heterozygosity retained under random genetic drift, (1 − 1/(2N))^t, for population sizes N = 5, 10, 25, 50, 100, 250 (rows) after 10, 25, 50, 100, 200 generations (columns), each cell computed by the package and rounded to 3 decimals. N = 50 keeps 0.366 after 100 generations; N = 5 keeps 0 after 200.



H_t (expected heterozygosity after t generations) against i (initial copies of allele A (of 2N)) from 1 to 20. One curve for each value of t (number of generations): 1, 2, 3. Other inputs as in the worked example: N = 10. 19 points per curve computed by computational-biology-ai999x; marked: the worked example, i = 10, H_t = 0.475.

```
computational_biology_ai999x.solve('bio_drift_heterozygosity', N=10, i=10, t=1)
# -> H0 = 0.5, H_t = 0.475, retained = 0.95   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_drift_heterozygosity', N=50, i=20, t=100)
# -> H0 = 0.32, H_t = 0.11713, retained = 0.366032   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Drift heterozygosity: N = 10, i = 10, t = 1	SOLVED	H0 = 0.5, H_t = 0.475, retained = 0.95
Drift heterozygosity: N = 50, i = 20, t = 100	SOLVED	H0 = 0.32, H_t = 0.11713, retained = 0.366032
Drift heterozygosity: i = 10, t = 1	NEEDS_INPUT	bio_drift_heterozygosity needs N (number of diploid individuals). Example: solve('bio_drift_heterozygosity', N=10, i=10,
Drift heterozygosity: N = 10, i = 10, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Drift heterozygosity: N = 10, i = 10, t = 1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Drift heterozygosity: N = 10, i = 20, t = 5	INVALID_INPUT	Loss of heterozygosity by genetic drift: i must be a whole number of copies between 1 and 2N - 1.

Fixation probability in the Moran process, `bio_moran_fixation`

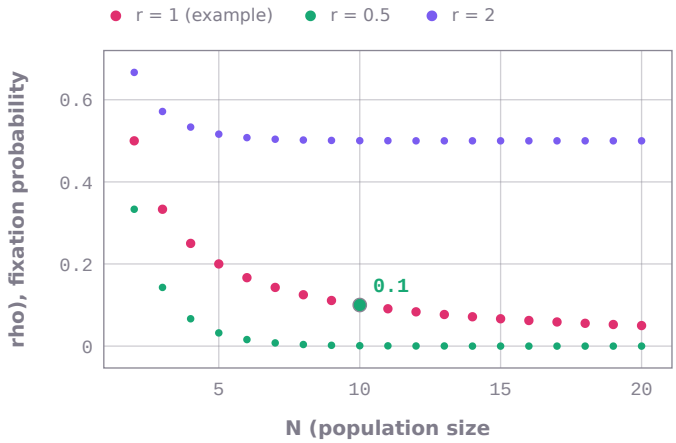
The probability that a mutant with relative fitness r, starting from i copies in a Moran population of size N, eventually takes over the population.

$$\rho_i = \frac{1 - r^{-i}}{1 - r^{-N}}$$

input	meaning	condition
`N`	population size	
`i`	initial number of mutants	
`r`	relative fitness of the mutant	

output	meaning
`rho`	fixation probability

- **Method:** gambler's-ruin solution of the Moran birth-death chain (i/N when r = 1)
- **Assumes:** $0 < i < N; r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rho (fixation probability) against N (population size) from 1 to 20. One curve for each value of r (relative fitness of the mutant): 1, 0.5, 2. Other inputs as in the worked example: i = 1. 19 points per curve computed by computational-biology-ai999x; marked: the worked example, N = 10, rho = 0.1.

```
computational_biology_ai999x.solve('bio_moran_fixation', N=10, i=1, r=1)
# -> rho = 0.1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_moran_fixation', N=10, i=1, r=2)
# -> rho = 0.500489   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moran fixation probability: N = 10, i = 1, r = 1	SOLVED	rho = 0.1
Moran fixation probability: N = 10, i = 1, r = 2	SOLVED	rho = 0.500489
Moran fixation probability: i = 1, r = 1	NEEDS_INPUT	bio_moran_fixation needs N (population size). Example: solve('bio_moran_fixation', N=10, i=1, r=1).
Moran fixation probability: N = 10, i = 1, r = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Moran fixation probability: N = 10, i = 1, r = 1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Effective population size with unequal sex ratio, `bio_ne_sex_ratio`

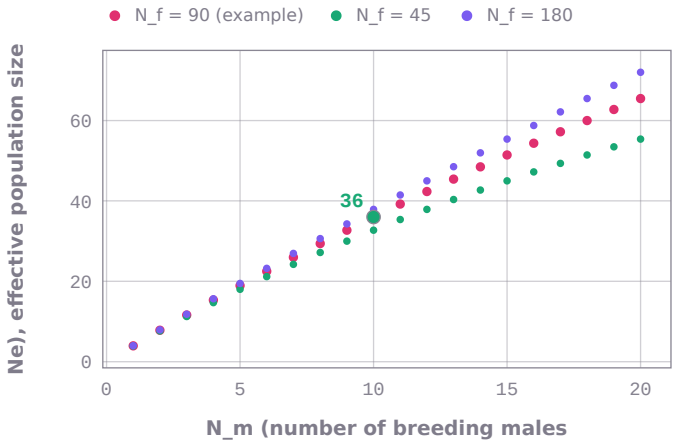
The effective population size of a population with N_m breeding males and N_f breeding females.

$$N_e = \frac{4N_m N_f}{N_m + N_f}$$

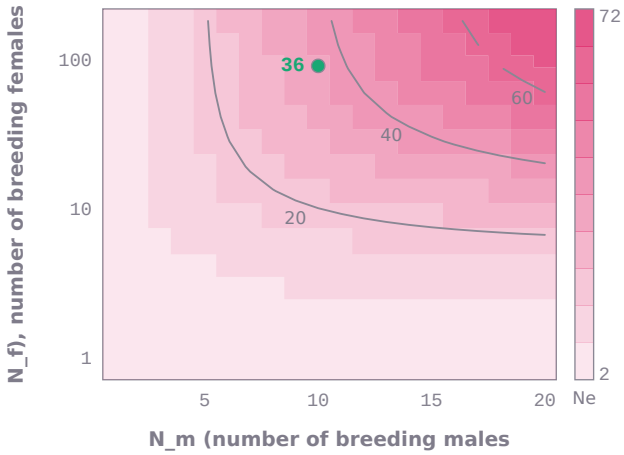
input	meaning	condition
`N_m`	number of breeding males	
`N_f`	number of breeding females	

output	meaning
`Ne`	effective population size
`Ne_over_N`	Ne divided by the census N_m + N_f

- **Method:** Wright's formula
- **Assumes:** random mating within the breeding population; Wright-Fisher reproduction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ne (effective population size) against N_m (number of breeding males) from 1 to 20. One curve for each value of N_f (number of breeding females): 90, 45, 180. 20 points per curve computed by computational-biology-ai999x; marked: the worked example, N_m = 10, Ne = 36.



Ne (effective population size) over N_m (number of breeding males) from 1 to 20 and N_f (number of breeding females) from 1 to 180 (logarithmic vertical axis): each of the 20 × 15 cells is one run of computational-biology-ai999x, shaded by its value (light 2 to dark 72); iso-lines at Ne = 20, 40, 60. Marked: the worked example, N_m = 10, N_f = 90, Ne = 36.

```
computational_biology_ai999x.solve('bio_ne_sex_ratio', N_m=10, N_f=90)
# -> Ne = 36, Ne_over_N = 0.36   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_ne_sex_ratio', N_m=50, N_f=50)
# -> Ne = 100, Ne_over_N = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective population size with unequal sex ratio: N_m = 10, N_f = 90	SOLVED	Ne = 36, Ne_over_N = 0.36
Effective population size with unequal sex ratio: N_m = 50, N_f = 50	SOLVED	Ne = 100, Ne_over_N = 1
Effective population size with unequal sex ratio: N_f = 90	NEEDS_INPUT	bio_ne_sex_ratio needs N_m (number of breeding males). Example: solve('bio_ne_sex_ratio', N_m=10, N_f=90).
Effective population size with unequal sex ratio: N_m = 10, N_f = 90, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Effective population size with unequal sex ratio: N_m = 10, N_f = 90 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Effective size of a fluctuating population, `bio_ne_fluctuating_size`

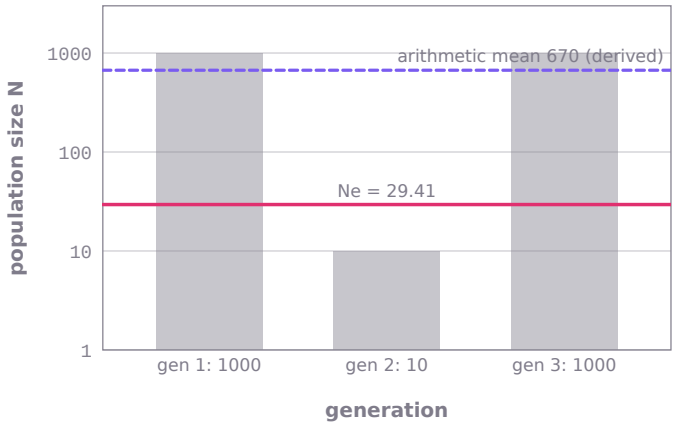
The effective population size over generations of changing size (harmonic mean) and the fraction of heterozygosity retained over those generations.

$$\frac{1}{N_e} = \frac{1}{t}\sum_{i=1}^{t}\frac{1}{N_i}, \quad \frac{H_t}{H_0} = \prod_i \left(1 - \frac{1}{2N_i}\right)$$

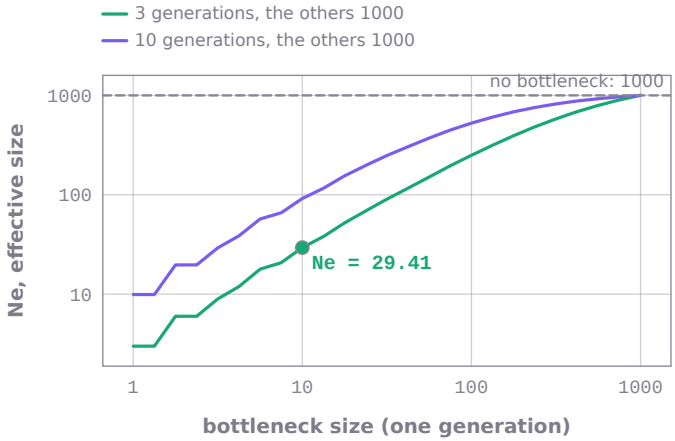
input	meaning	condition
`sizes`	numbers of diploid individuals in successive generations	

output	meaning
`Ne`	effective size (harmonic mean of the sizes)
`retained`	heterozygosity retained, product of (1 - 1/(2N_i))

- **Method:** harmonic mean of the generation sizes; heterozygosity falls by $1 - 1/(2N_i)$ in generation i
- **Assumes:** whole positive sizes; Wright-Fisher reproduction each generation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Census sizes 1000, 10, 1000 in successive generations (logarithmic axis): the package returns the effective size Ne = 29.41, the harmonic mean, which the single bottleneck drags far below the arithmetic mean 670 (computed from the sizes for comparison); heterozygosity retained over the three generations 0.9491. Marked: Ne = 29.41.



Effective size of a population of 1000 (the worked example's largest) that passes through one bottleneck generation of size 1 … 1000, over 3 and over 10 generations: each point a run of the package. A single small generation dominates Ne; its weight fades over more generations. Marked: the worked example's sizes ['1000', '10', '1000'], smallest 10: Ne = 29.41.

```
computational_biology_ai999x.solve('bio_ne_fluctuating_size', sizes=[1000, 10, 1000])
# -> Ne = 29.4118, retained = 0.94905   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_ne_fluctuating_size', sizes=[20, 20])
# -> Ne = 20, retained = 0.950625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective size of a fluctuating population: sizes = [1000, 10, 1000]	SOLVED	Ne = 29.4118, retained = 0.94905
Effective size of a fluctuating population: sizes = [20, 20]	SOLVED	Ne = 20, retained = 0.950625
Effective size of a fluctuating population:	NEEDS_INPUT	bio_ne_fluctuating_size needs sizes (numbers of diploid individuals in successive generations). Example: solve('bio_ne_f
Effective size of a fluctuating population: sizes = [1000, 10, 1000], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Effective size of a fluctuating population: sizes = [1000, 10, 1000] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Fixation index FST from subpopulation allele frequencies, `bio_fst`

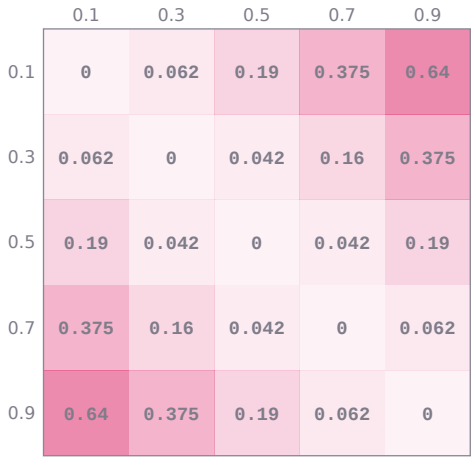
Mean within-subpopulation heterozygosity HS, total heterozygosity HT and FST = (HT - HS)/HT for a biallelic locus in equally weighted subpopulations.

$$F_{ST} = \frac{H_T - H_S}{H_T}$$

input	meaning	condition
p	frequency of one allele in each subpopulation	

output	meaning
HS	mean expected heterozygosity within subpopulations
HT	expected heterozygosity of the pooled population
FST	fixation index (HT - HS)/HT

- **Method:** HS = mean of 2p(1-p), $HT = 2$ pbar (1 - pbar), FST = (HT - HS)/HT
- **Assumes:** equal subpopulation weights; biallelic locus; 0 < mean $p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



FST · rows: p in population 1 · columns: p in population 2

Fixation index between two subpopulations whose allele frequencies p are 0.1 … 0.9 (rows: the first, columns: the second), each cell a run of the package rounded to 3 decimals: zero on the diagonal, largest for opposite frequencies. The worked example p = [0.2, 0.8] gives HS = 0.32, HT = 0.5, FST = 0.36.

```
computational_biology_ai999x.solve('bio_fst', p=[0.2, 0.8])
# -> HS = 0.32, HT = 0.5, FST = 0.36   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_fst', p=[0.5, 0.5, 0.5])
# -> HS = 0.5, HT = 0.5, FST = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
FST: p = [0.2, 0.8]	SOLVED	HS = 0.32, HT = 0.5, FST = 0.36
FST: p = [0.5, 0.5, 0.5]	SOLVED	HS = 0.5, HT = 0.5, FST = 0
FST:	NEEDS_INPUT	bio_fst needs p (frequency of one allele in each subpopulation). Example: solve('bio_fst', p=[0.2, 0.8]).
FST: p = [0.2, 0.8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
FST: p = [0.2, 0.8] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Linkage disequilibrium D, D' and r^2, `bio_linkage_disequilibrium`

The linkage disequilibrium between two biallelic loci from haplotype counts: D, Lewontin's D' and r^2.

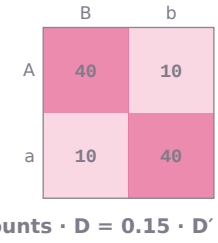
$$D = p_{AB} - p_A p_B, \quad r^2 = \frac{D^2}{p_A q_A p_B q_B}, \quad D' = \frac{D}{D_{\max}}$$

input	meaning	condition
n_AB	number of AB haplotypes	
n_Ab	number of Ab haplotypes	
n_aB	number of aB haplotypes	
n_ab	number of ab haplotypes	

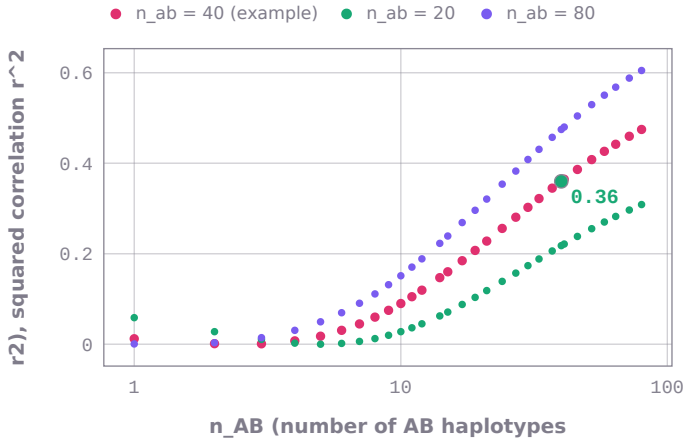
output	meaning
D	disequilibrium D = pAB - pA pB

output	meaning
`D_prime`	Lewontin's D' = D / Dmax (signed)
`r2`	squared correlation r^2

- **Method:** haplotype and allele frequencies (exact fractions); Dmax from the allele frequencies
- **Assumes:** both loci polymorphic; phased haplotype counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Haplotype counts of two loci as a 2 × 2 table (rows: allele A or a, columns: B or b): AB = 40, Ab = 10, aB = 10, ab = 40. The excess of AB and ab over what independent loci would give is the disequilibrium the package returns: D = 0.15, D′ = 0.6, r² = 0.36.



r2 (squared correlation r^2) against n_AB (number of AB haplotypes) from 1 to 80 on a logarithmic axis. One curve for each value of n_ab (number of ab haplotypes): 40, 20, 80. Other inputs as in the worked example: n_Ab = 10, n_aB = 10. 30 points per curve computed by computational-biology-ai999x; marked: the worked example, n_AB = 40, r2 = 0.36.

```
computational_biology_ai999x.solve('bio_linkage_disequilibrium', n_AB=40, n_Ab=10, n_aB=10, n_ab=40)
# -> D = 0.15, D_prime = 0.6, r2 = 0.36   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_linkage_disequilibrium', n_AB=25, n_Ab=25, n_aB=25, n_ab=25)
# -> D = 0, D_prime = 0, r2 = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linkage disequilibrium: n_AB = 40, n_Ab = 10, n_aB = 10, n_ab = 40	SOLVED	D = 0.15, D_prime = 0.6, r2 = 0.36
Linkage disequilibrium: n_AB = 25, n_Ab = 25, n_aB = 25, n_ab = 25	SOLVED	D = 0, D_prime = 0, r2 = 0
Linkage disequilibrium: n_Ab = 10, n_aB = 10, n_ab = 40	NEEDS_INPUT	bio_linkage_disequilibrium needs n_AB (number of AB haplotypes). Example: solve('bio_linkage_disequilibrium', n_AB=40, n
Linkage disequilibrium: n_AB = 40, n_Ab = 10, n_aB = 10, n_ab = 40, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Linkage disequilibrium: n_AB = 40, n_Ab = 10, n_aB = 10, n_ab = 40 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Map distance from a recombination fraction, `bio_map_distance`

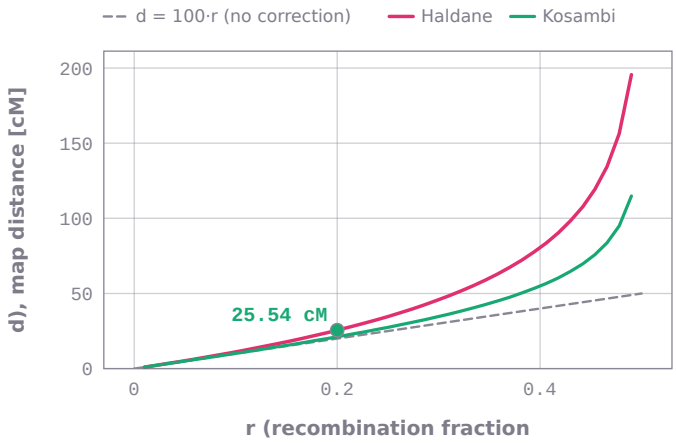
The genetic map distance (cM) corresponding to a recombination fraction, by the Haldane or Kosambi map function.

$$d_H = -\tfrac{1}{2}\ln(1-2r), \quad d_K = \tfrac{1}{4}\ln\frac{1+2r}{1-2r}$$

input	meaning	condition
`r`	recombination fraction (0 to 0.5)	
`map`	map function: haldane (no interference) or kosambi	

output	meaning
`d_cM`	map distance in centimorgans

- **Method:** Haldane $d = -ln$(1-2r)/2; Kosambi $d = ln((1+2r)/(1-2r))$/4 (Morgans, x100 for cM)
- **Assumes:** $0 \le r < 0.5$; map function stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Map distance against recombination fraction r = 0.01 … 0.49 by the Haldane and Kosambi map functions, each curve 41 runs of the package; the dashed line d = 100·r is drawn for comparison. Both corrections grow without bound as r approaches 1/2 (unlinked loci). Marked: the worked example r = 0.2 (haldane), d = 25.54 cM.

```
computational_biology_ai999x.solve('bio_map_distance', r=0.2, map='haldane')
# -> d_cM = 25.5413   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_map_distance', r=0.2, map='kosambi')
# -> d_cM = 21.1824   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Map distance: r = 0.2, map = haldane	SOLVED	d_cM = 25.5413
Map distance: r = 0.2, map = kosambi	SOLVED	d_cM = 21.1824
Map distance: map = haldane	NEEDS_INPUT	bio_map_distance needs r (recombination fraction (0 to 0.5)). Example: solve('bio_map_distance', r=0.2, map='haldane').
Map distance: r = 0.2, map = haldane, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Map distance: r = 0.2, map = haldane and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Map distance: r = 0.2	NEEDS_INPUT	bio_map_distance needs map (map function: haldane (no interference) or kosambi). Example: solve('bio_map_distance', r=0.
Map distance: r = 0.6, map = haldane	INVALID_INPUT	Map distance from a recombination fraction: the recombination fraction must lie in [0, 0.5).

Recombination fraction from a map distance, `bio_recombination_fraction`

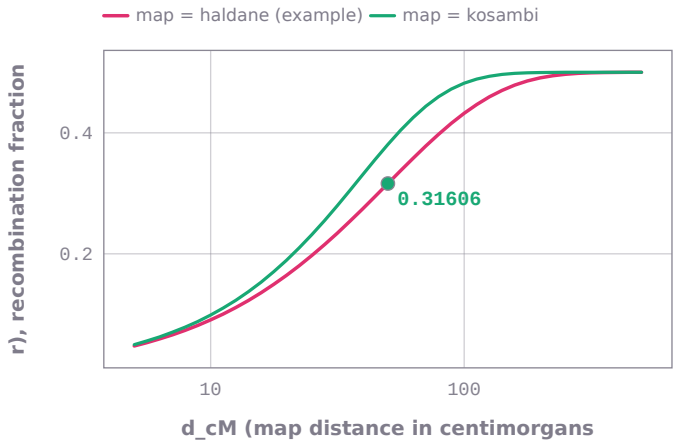
The recombination fraction between two loci a stated map distance apart, by the Haldane or Kosambi map function.

$$r_H = \tfrac{1}{2}(1 - e^{-2d}), \quad r_K = \tfrac{1}{2}\tanh 2d$$

input	meaning	condition
`d_cM`	map distance in centimorgans	
`map`	map function: haldane (no interference) or kosambi	

output	meaning
`r`	recombination fraction

- **Method:** Haldane r = (1 - e^{-2d})/2; Kosambi r = tanh(2d)/2 (d in Morgans)
- **Assumes:** $d \ge 0$; map function stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



r (recombination fraction) against d_cM (map distance in centimorgans) from 5 to 500 on a logarithmic axis. One curve for each value of map (map function: haldane (no interference) or kosambi): haldane, kosambi. 41 points per curve computed by computational-biology-ai999x; marked: the worked example, d_cM = 50, r = 0.31606.

```
computational_biology_ai999x.solve('bio_recombination_fraction', d_cM=50, map='haldane')
# -> r = 0.31606   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_recombination_fraction', d_cM=50, map='kosambi')
# -> r = 0.380797   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Recombination fraction: d_cM = 50, map = haldane	SOLVED	r = 0.31606
Recombination fraction: d_cM = 50, map = kosambi	SOLVED	r = 0.380797
Recombination fraction: map = haldane	NEEDS_INPUT	bio_recombination_fraction needs d_cM (map distance in centimorgans). Example: solve('bio_recombination_fraction', d_cM=
Recombination fraction: d_cM = 50, map = haldane, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Recombination fraction: d_cM = 50, map = haldane and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Inbreeding coefficient from a pedigree, `bio_pedigree_inbreeding`

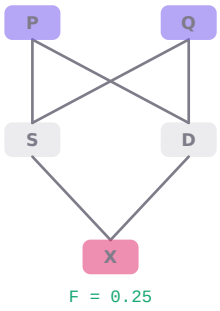
Wright's inbreeding coefficient F of an individual from its pedigree (founders unrelated and not inbred).

$$F_X = \sum_{\text{paths}} \left(\tfrac{1}{2}\right)^{n_1+n_2+1}(1+F_A)$$

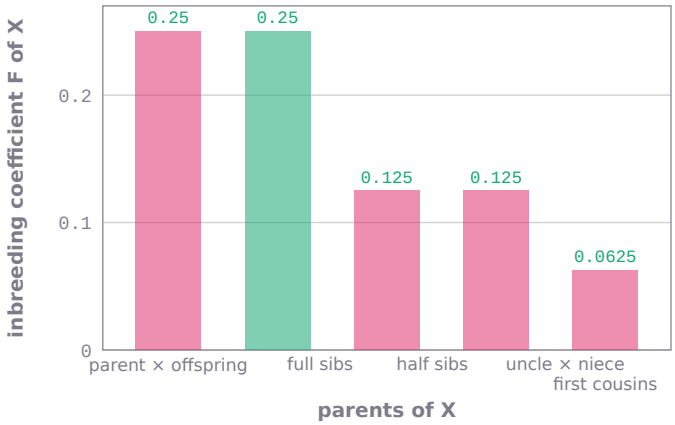
input	meaning	condition
`pedigree`	parent-child pairs, e.g. [P-S, Q-S, P-D, Q-D, S-X, D-X]	
`individual`	name of the individual	

output	meaning
`F`	inbreeding coefficient

- **Method:** Wright's path counting: sum over common ancestors and non-overlapping paths of $(1/2)^{n_1+n_2+1}\ (1+F_A)$
- **Assumes:** every non-founder has exactly two parents listed; no cycles; founders unrelated, $F=0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The pedigree P-S, Q-S, P-D, Q-D, S-X, D-X drawn by generation, each line joining a parent to its child; the individual X is shaded, and the ancestors shared by its two parents (read from the pedigree) in a second colour. The package returns the inbreeding coefficient F = 0.25.



Inbreeding coefficient of an individual whose parents are related in 5 ways, each a run of the package on the pedigree written: parent × offspring: P-S, Q-S, P-X, S-X; full sibs: P-S, Q-S, P-D, Q-D, S-X, D-X; half sibs: P-S, Q-S, P-D, R-D, S-X, D-X; uncle × niece: P-A, Q-A, P-B, Q-B, B-C, M-C, A-X, C-X; first cousins: P-A, Q-A, P-B, Q-B, A-C, M-C, B-D, N-D, C-X, D-X. Highlighted: the worked example's pedigree (full sibs).

```
computational_biology_ai999x.solve('bio_pedigree_inbreeding', pedigree='P-S, Q-S, P-D, Q-D, S-X, D-X', individual='X')
# -> F = 0.25   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_pedigree_inbreeding', pedigree='A-C, B-C, A-D, C-D', individual='D')
# -> F = 0.25   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_pedigree_inbreeding', pedigree='P-S, Q-S, P-D, R-D, S-X, D-X', individual='X')
# -> F = 0.125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pedigree inbreeding coefficient: pedigree = [P-S, Q-S, P-D, Q-D, S-X, D-X], individual = X	SOLVED	F = 0.25
Pedigree inbreeding coefficient: pedigree = [A-C, B-C, A-D, C-D], individual = D	SOLVED	F = 0.25
Pedigree inbreeding coefficient: pedigree = [P-S, Q-S, P-D, R-D, S-X, D-X], individual = X	SOLVED	F = 0.125
Pedigree inbreeding coefficient: individual = X	NEEDS_INPUT	bio_pedigree_inbreeding needs pedigree (parent-child pairs, e.g. [P-S, Q-S, P-D, Q-D, S-X, D-X]). Example: solve('bio_pe
Pedigree inbreeding coefficient: pedigree = [P-S, Q-S, P-D, Q-D, S-X, D-X], individual = X, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Pedigree inbreeding coefficient: pedigree = [P-S, Q-S, P-D, Q-D, S-X, D-X], individual = X and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Pedigree inbreeding coefficient: pedigree = [P-S, S-X, D-X], individual = X	INVALID_INPUT	Inbreeding coefficient from a pedigree: S must have exactly two parents listed (founders have none).

Jukes-Cantor distance, `bio_jukes_cantor_distance`

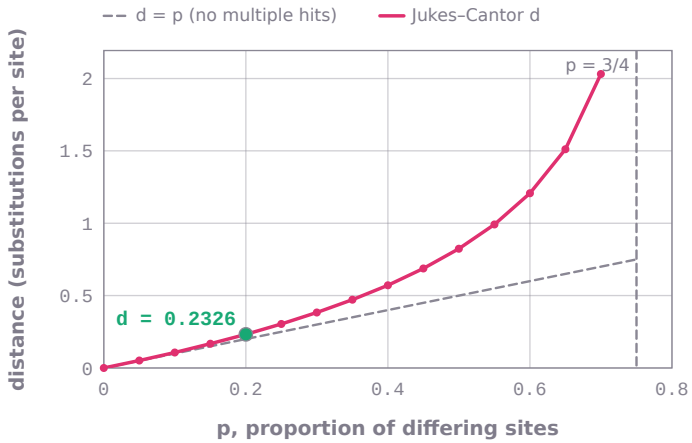
The Jukes-Cantor (JC69) evolutionary distance between two aligned DNA sequences: the expected number of substitutions per site, corrected for multiple hits.

$$d = -\tfrac{3}{4}\ln\left(1 - \tfrac{4}{3}p\right)$$

input	meaning	condition
`seq1`	first aligned DNA sequence	
`seq2`	second aligned DNA sequence	

output	meaning
`p`	proportion of differing sites
`d`	Jukes-Cantor distance (substitutions per site)

- **Method:** d = -$(3/4)$ ln(1 - 4p/3)
- **Assumes:** equal lengths, no gaps; $p < 0.75$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Jukes–Cantor distance against the observed proportion of differing sites: pairs of 20-base sequences differing at 0 … 14 sites, each a run of the package (p and d both returned). The correction grows without bound as p approaches 3/4, the proportion two unrelated sequences share by chance. Marked: the worked example AAAAAAAAAA / AAAAAAAACC, p = 0.2, d = 0.2326.

```
computational_biology_ai999x.solve('bio_jukes_cantor_distance', seq1='AAAAAAAAAA', seq2='AAAAAAAACC')
# -> p = 0.2, d = 0.232616   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_jukes_cantor_distance', seq1='ACGT', seq2='ACGT')
# -> p = 0, d = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Jukes-Cantor distance: seq1 = AAAAAAAAAA, seq2 = AAAAAAAACC	SOLVED	p = 0.2, d = 0.232616
Jukes-Cantor distance: seq1 = ACGT, seq2 = ACGT	SOLVED	p = 0, d = 0
Jukes-Cantor distance: seq2 = AAAAAAAACC	NEEDS_INPUT	bio_jukes_cantor_distance needs seq1 (first aligned DNA sequence). Example: solve('bio_jukes_cantor_distance', seq1='AAA
Jukes-Cantor distance: seq1 = AAAAAAAAAA, seq2 = AAAAAAAACC, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Jukes-Cantor distance: seq1 = AAAAAAAAAA, seq2 = AAAAAAAACC and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Kimura two-parameter distance, `bio_kimura_distance`

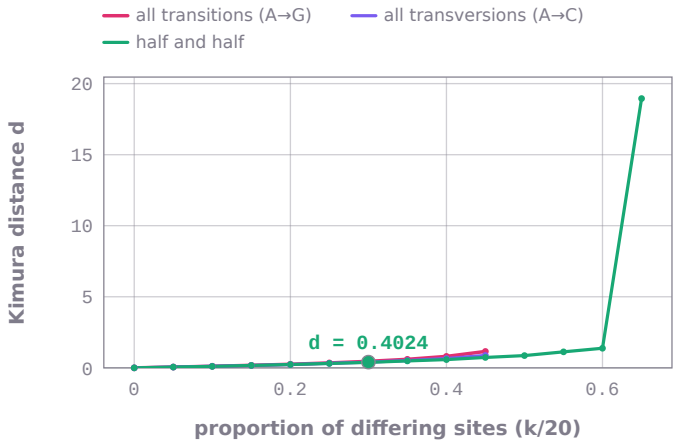
The Kimura two-parameter (K80) distance between two aligned DNA sequences, separating transitions (A<->G, C<->T) from transversions.

$$d = -\tfrac{1}{2}\ln(1-2P-Q) - \tfrac{1}{4}\ln(1-2Q)$$

input	meaning	condition
`seq1`	first aligned DNA sequence	
`seq2`	second aligned DNA sequence	

output	meaning
`d`	K80 distance (substitutions per site)

- **Method:** P = transition proportion, Q = transversion proportion, $d = -ln$(1-2P-Q)/2 - ln(1-2Q)/4
- **Assumes:** equal lengths, no gaps; $1-2P-Q > 0$ and $1-2Q > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kimura two-parameter distance for pairs of 20-base sequences differing at k sites, where every difference is a transition, a transversion, or alternately one and the other; each point is a run of the package (a pair it refuses, past saturation, is left out). Transitions saturate first. Marked: the worked example AAAAAAAAAA / AAAAAAAGGC (3 differences), d = 0.4024.

```
computational_biology_ai999x.solve('bio_kimura_distance', seq1='AAAAAAAAAA', seq2='AAAAAAAGGC')
# -> d = 0.402359   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_kimura_distance', seq1='ACGTACGTAC', seq2='GCGTACGTAC')
# -> d = 0.111572   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kimura distance: seq1 = AAAAAAAAAA, seq2 = AAAAAAAGGC	SOLVED	d = 0.402359
Kimura distance: seq1 = ACGTACGTAC, seq2 = GCGTACGTAC	SOLVED	d = 0.111572
Kimura distance: seq2 = AAAAAAAGGC	NEEDS_INPUT	bio_kimura_distance needs seq1 (first aligned DNA sequence). Example: solve('bio_kimura_distance', seq1='AAAAAAAAAA', se
Kimura distance: seq1 = AAAAAAAAAA, seq2 = AAAAAAAGGC, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Kimura distance: seq1 = AAAAAAAAAA, seq2 = AAAAAAAGGC and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

UPGMA tree heights, `bio_upgma`

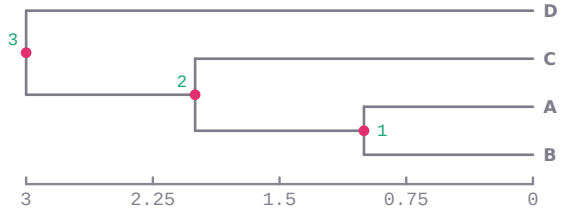
The node heights of the UPGMA (average-linkage) tree of a distance matrix: the root height and every merge height (half the distance between the merged clusters).

$$d_{(ij),k} = \frac{n_i d_{ik} + n_j d_{jk}}{n_i + n_j}, \quad h = d_{ij}/2$$

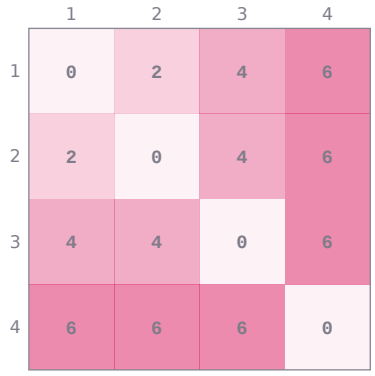
input	meaning	condition
`D`	symmetric distance matrix, zero diagonal	

output	meaning
`root_height`	height of the root
`heights`	all node heights, ascending

- **Method:** repeatedly merge the closest clusters; distances to a merged cluster are size-weighted averages
- **Assumes:** symmetric, zero diagonal, positive distances; no ties in a merge step
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



UPGMA tree of 4 taxa (named A–D in the order of the rows of D = [[0, 2, 4, 6], [2, 0, 4, 6], [4, 4, 0, 6], [6, 6, 6, 0]]), drawn against node height (axis below). The package returns the node heights 1, 2, 3 and the root height 3, written at the nodes; which clusters join at each node is the UPGMA pairing read from D, whose heights match the returned ones.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): D (input). Returned with them by computational-biology-ai999x: root_height = 3.

```
computational_biology_ai999x.solve('bio_upgma', D=[[0, 2, 4, 6], [2, 0, 4, 6], [4, 4, 0, 6], [6, 6, 6, 0]])
# -> root_height = 3, heights = [1.0, 2.0, 3.0]   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_upgma', D=[[0, 17, 21, 31, 23], [17, 0, 30, 34, 21], [21, 30, 0, 28, 39], [31, 34, 28, 0, 43], [23, 21, 39, 43, 0]])
# -> root_height = 16.5, heights = [8.5, 11.0, 14.0, 16.5]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
UPGMA: D = [[0, 2, 4, 6], [2, 0, 4, 6], [4, 4, 0, 6], [6, 6, 6, 0]]	SOLVED	root_height = 3, heights = [1.0, 2.0, 3.0]
UPGMA: D = [[0, 17, 21, 31, 23], [17, 0, 30, 34, 21], [21, 30, 0, 28, 39], [31, 34, 28, 0, 43], [23, 21, 39, 43, 0]]	SOLVED	root_height = 16.5, heights = [8.5, 11.0, 14.0, 16.5]
UPGMA:	NEEDS_INPUT	bio_upgma needs D (symmetric distance matrix, zero diagonal). Example: solve('bio_upgma', D=[[0, 2, 4, 6], [2, 0, 4, 6],
UPGMA: D = [[0, 2, 4, 6], [2, 0, 4, 6], [4, 4, 0, 6], [6, 6, 6, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
UPGMA: D = [[0, 2, 4, 6], [2, 0, 4, 6], [4, 4, 0, 6], [6, 6, 6, 0]] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
UPGMA: D = [[0, 2], [3, 0]]	INVALID_INPUT	UPGMA tree heights: D must be symmetric.

Neighbour-joining branch lengths, `bio_neighbor_joining`

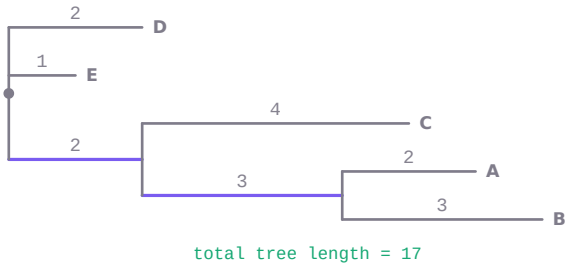
The total tree length and the terminal branch lengths of the neighbour-joining tree of a distance matrix.

$$Q_{ij} = (r-2)d_{ij} - R_i - R_j, \quad \ell_i = \tfrac{1}{2}d_{ij} + \frac{R_i - R_j}{2(r-2)}$$

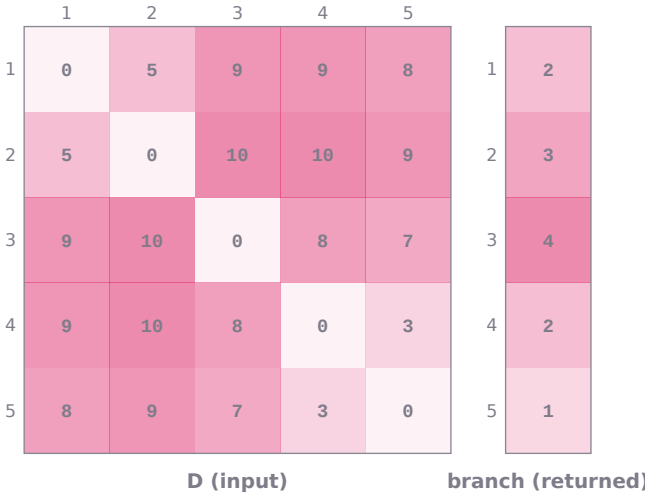
input	meaning	condition
`D`	symmetric distance matrix, zero diagonal (at least 3 taxa)	

output	meaning
`total_length`	sum of all branch lengths
`branch`	terminal branch length of each taxon, in input order

- **Method:** Saitou-Nei: join the pair minimising $Q = (r-2)$ d_ij - R_i - R_j; branch lengths from the row sums
- **Assumes:** at least 3 taxa; no ties in the joining criterion; negative branch lengths kept as computed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Neighbour-joining tree of 5 taxa (A–E, the rows of D = [[0, 5, 9, 9, 8], [5, 0, 10, 10, 9], [9, 10, 0, 8, 7], [9, 10, 8, 0, 3], [8, 9, 7, 3, 0]]), drawn to scale from the last joining point (an unrooted tree has no root; the dot is where its three last branches meet). The package returns the terminal branch lengths 2, 3, 4, 2, 1 (A–E) and the total length 17; the topology and the internal branches (second colour) are the neighbour-joining pairing read from D, which reproduces both.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): D (input); branch (returned). Returned with them by computational-biology-ai999x: total_length = 17.

```
computational_biology_ai999x.solve('bio_neighbor_joining', D=[[0, 5, 9, 9, 8], [5, 0, 10, 10, 9], [9, 10, 0, 8, 7], [9, 10, 8, 0, 3], [8, 9, 7, 3, 0]])
# -> total_length = 17, branch = [2.0, 3.0, 4.0, 2.0, 1.0]   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_neighbor_joining', D=[[0, 3, 4], [3, 0, 5], [4, 5, 0]])
# -> total_length = 6, branch = [1.0, 2.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Neighbour joining: D = [[0, 5, 9, 9, 8], [5, 0, 10, 10, 9], [9, 10, 0, 8, 7], [9, 10, 8, 0, 3], [8, 9, 7, 3, 0]]	SOLVED	total_length = 17, branch = [2.0, 3.0, 4.0, 2.0, 1.0]
Neighbour joining: D = [[0, 3, 4], [3, 0, 5], [4, 5, 0]]	SOLVED	total_length = 6, branch = [1.0, 2.0, 3.0]
Neighbour joining:	NEEDS_INPUT	bio_neighbor_joining needs D (symmetric distance matrix, zero diagonal (at least 3 taxa)). Example: solve('bio_neighbor_
Neighbour joining: D = [[0, 5, 9, 9, 8], [5, 0, 10, 10, 9], [9, 10, 0, 8, 7], [9, 10, 8, 0, 3], [8, 9, 7, 3, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Neighbour joining: D = [[0, 5, 9, 9, 8], [5, 0, 10, 10, 9], [9, 10, 0, 8, 7], [9, 10, 8, 0, 3], [8, 9, 7, 3, 0]] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Number of possible phylogenetic trees, `bio_tree_count`

The number of distinct labelled bifurcating trees for n taxa, rooted or unrooted.

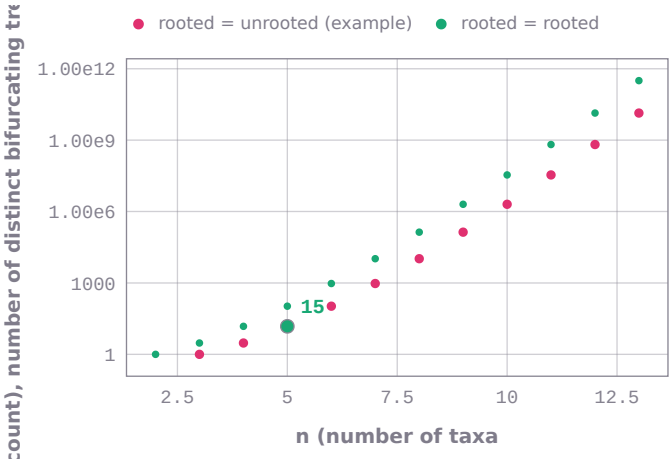
$$N_{\text{rooted}} = (2n-3)!!, \quad N_{\text{unrooted}} = (2n-5)!!$$

input	meaning	condition
`n`	number of taxa	
`rooted`	rooted or unrooted trees	

output	meaning
`count`	number of distinct bifurcating trees

- **Method:** double factorials: $(2n-3)!!$ rooted, $(2n-5)!!$ unrooted

- **Assumes:** $n \geq 2$ (rooted) or $n \geq 3$ (unrooted); $n \leq 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



count (number of distinct bifurcating trees) against n (number of taxa) from 1 to 13, logarithmic output axis. One curve for each value of rooted (rooted or unrooted trees): unrooted, rooted. 11 points per curve computed by computational-biology-ai999x; marked: the worked example, n = 5, count = 15.

```
computational_biology_ai999x.solve('bio_tree_count', n=5, rooted='unrooted')
# -> count = 15   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_tree_count', n=7, rooted='rooted')
# -> count = 10395   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Number of trees: n = 5, rooted = unrooted	SOLVED	count = 15
Number of trees: n = 7, rooted = rooted	SOLVED	count = 10395
Number of trees: rooted = unrooted	NEEDS_INPUT	bio_tree_count needs n (number of taxa). Example: solve('bio_tree_count', n=5, rooted='unrooted').
Number of trees: n = 5, rooted = unrooted, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Number of trees: n = 5, rooted = unrooted and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Robinson-Foulds distance between trees, `bio_robinson_foulds`

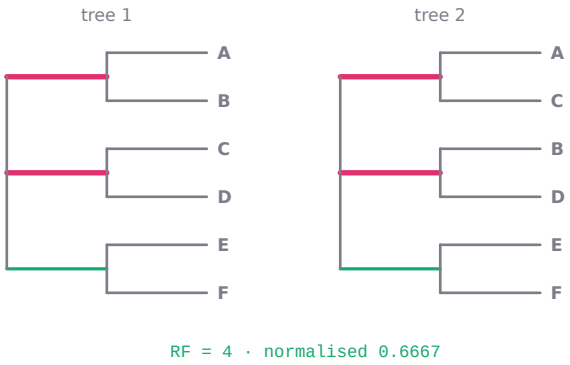
The Robinson-Foulds (symmetric-difference) distance between two trees on the same taxa, comparing their bipartitions (trees treated as unrooted), and the distance as a proportion.

$$\mathrm{RF} = |S_1 \setminus S_2| + |S_2 \setminus S_1|$$

input	meaning	condition
`tree1`	first tree in Newick format	
`tree2`	second tree in Newick format	

output	meaning
`rf`	number of bipartitions found in only one tree
`rf_normalized`	RF divided by the total number of non-trivial bipartitions of both trees

- **Method:** non-trivial bipartitions of each tree; count those not shared
- **Assumes:** the same taxa; at least 4 taxa; trees compared unrooted; branch lengths ignored
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tree 1 = ((A,B),(C,D),(E,F)); and tree 2 = ((A,C),(B,D),(E,F)); drawn side by side. Each internal branch splits the taxa in two; branches whose split the other tree lacks are drawn thick (read from the two trees), shared ones in the second colour. The package counts them: rf = 4, normalised 0.6667.

```
computational_biology_ai999x.solve('bio_robinson_foulds', tree1='((A,B),(C,D),(E,F));', tree2='((A,C),(B,D),(E,F));')
# -> rf = 4, rf_normalized = 0.666667   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_robinson_foulds', tree1='(((A,B),C),(D,E));', tree2='((A,B),(C,(D,E)));')
# -> rf = 0, rf_normalized = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Robinson-Foulds distance: tree1 = ((A,B),(C,D),(E,F));, tree2 = ((A,C),(B,D),(E,F));	SOLVED	rf = 4, rf_normalized = 0.666667
Robinson-Foulds distance: tree1 = (((A,B),C),(D,E));, tree2 = ((A,B),(C,(D,E)));	SOLVED	rf = 0, rf_normalized = 0
Robinson-Foulds distance: tree2 = ((A,C),(B,D),(E,F));	NEEDS_INPUT	bio_robinson_foulds needs tree1 (first tree in Newick format). Example: solve('bio_robinson_foulds', tree1='((A,B),(C,D)
Robinson-Foulds distance: tree1 = ((A,B),(C,D),(E,F));, tree2 = ((A,C),(B,D),(E,F));, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Robinson-Foulds distance: tree1 = ((A,B),(C,D),(E,F));, tree2 = ((A,C),(B,D),(E,F)); and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Parsimony score of a tree (Fitch), `bio_fitch_parsimony`

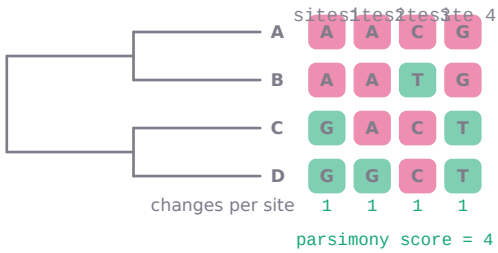
The minimum number of character changes (small parsimony score) needed on a given rooted bifurcating tree to explain aligned sequences at its leaves.

$$c_v(a) = \sum_{w \in \text{children}(v)} \min_b \left(c_w(b) + [a \neq b] \right)$$

input	meaning	condition
`tree`	rooted bifurcating tree in Newick format	
`states`	aligned sequence of each leaf, written name:SEQUENCE separated by spaces	

output	meaning
`score`	parsimony score (number of changes)

- **Method:** dynamic programming over the tree site by site (equivalent to Fitch's algorithm with unit costs)
- **Assumes:** strictly bifurcating rooted tree; one aligned sequence per leaf; unit cost per change
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The tree ((A,B),(C,D)); with the character states of each taxon at 4 sites (A:AACG B:AATG C:GACT D:GGCT); letters of a site are coloured by state. Under each site, the minimum number of changes the package returns for that site alone (1, 1, 1, 1); their sum is the Fitch parsimony score of all sites, score = 4.

```
computational_biology_ai999x.solve('bio_fitch_parsimony', tree='((A,B),(C,D));', states='A:AACG B:AATG C:GACT D:GGCT')
# -> score = 4   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_fitch_parsimony', tree='((A,C),(B,D));', states='A:A B:A C:G D:G')
# -> score = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parsimony score: tree = ((A,B),(C,D));, states = A:AACG B:AATG C:GACT D:GGCT	SOLVED	score = 4
Parsimony score: tree = ((A,C),(B,D));, states = A:A B:A C:G D:G	SOLVED	score = 2
Parsimony score: states = A:AACG B:AATG C:GACT D:GGCT	NEEDS_INPUT	bio_fitch_parsimony needs tree (rooted bifurcating tree in Newick format). Example: solve('bio_fitch_parsimony', tree='(
Parsimony score: tree = ((A,B),(C,D));, states = A:AACG B:AATG C:GACT D:GGCT, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Parsimony score: tree = ((A,B),(C,D));, states = A:AACG B:AATG C:GACT D:GGCT and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Parsimony score: tree = (A,B,C);, states = A:A B:C C:G	INVALID_INPUT	Parsimony score of a tree (Fitch): the tree must be rooted and strictly bifurcating (two children per internal node).

Expected coalescence times, `bio_coalescent_times`

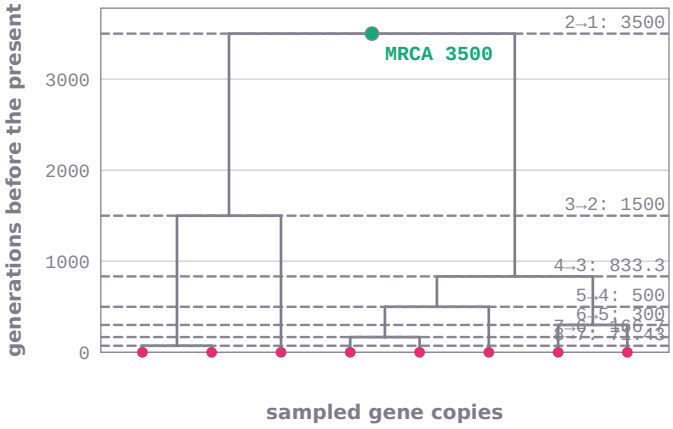
The expected time to the most recent common ancestor and the expected total branch length of the genealogy of n gene copies from a diploid population of effective size Ne (in generations).

$$E[T_{MRCA}] = 4N_e\left(1-\frac{1}{n}\right), \quad E[L] = 4N_e\sum_{i=1}^{n-1}\frac{1}{i}$$

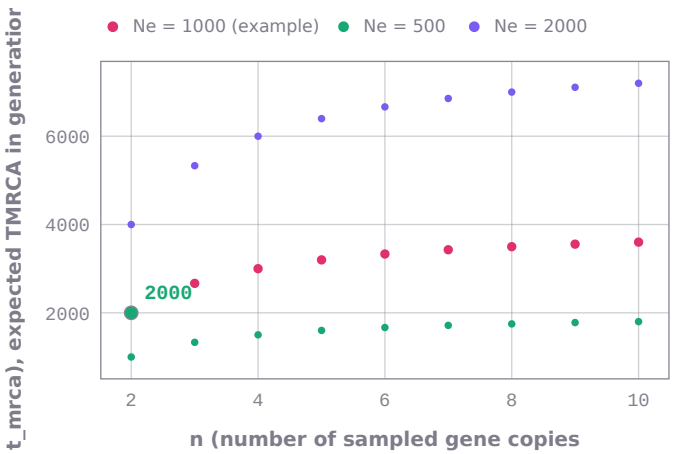
input	meaning	condition
`n`	number of sampled gene copies	
`Ne`	diploid effective population size	

output	meaning
`t_mrca`	expected TMRCA in generations
`total_length`	expected total branch length in generations

- **Method:** sum of exponential waiting times while k lineages remain, mean 4Ne/(k(k-1))
- **Assumes:** neutral Kingman coalescent; diploid: a pair coalesces at rate $1/(2Ne)$ per generation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Expected genealogy of 8 gene copies in a population of effective size Ne = 1000: the dashed levels are the expected times of the successive coalescences, T(n) − T(j − 1), from the expected times to the most recent common ancestor T(k) the package returns for k = 2 … 8 (T(1) = 0). Which lineages join at each level is drawn arbitrarily; the long last wait, two lineages becoming one, is about half the total. Marked: t_mrca = 3500 generations.



t_mrca (expected TMRCA in generations) against n (number of sampled gene copies) from 1 to 10. One curve for each value of Ne (diploid effective population size): 1000, 500, 2000. 9 points per curve computed by computational-biology-ai999x; marked: the worked example, n = 2, t_mrca = 2000.

```
computational_biology_ai999x.solve('bio_coalescent_times', n=2, Ne=1000)
# -> t_mrca = 2000, total_length = 4000   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_coalescent_times', n=10, Ne=10000)
# -> t_mrca = 36000, total_length = 113159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coalescent times: n = 2, Ne = 1000	SOLVED	t_mrca = 2000, total_length = 4000
Coalescent times: n = 10, Ne = 10000	SOLVED	t_mrca = 36000, total_length = 113159
Coalescent times: Ne = 1000	NEEDS_INPUT	bio_coalescent_times needs n (number of sampled gene copies). Example: solve('bio_coalescent_times', n=2, Ne=1000).
Coalescent times: n = 2, Ne = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Coalescent times: n = 2, Ne = 1000 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Nucleotide diversity, `bio_nucleotide_diversity`

The average number of pairwise differences k and the nucleotide diversity pi (per site) of a set of aligned DNA sequences.

$$\pi = \frac{1}{L}\binom{n}{2}^{-1}\sum_{i<j} d_{ij}$$

input	meaning	condition
`seqs`	aligned DNA sequences separated by spaces	

output	meaning
`k`	mean number of pairwise differences
`pi`	nucleotide diversity per site

- **Method:** average the differences over all pairs of sequences; divide by the alignment length
- **Assumes:** at least 2 aligned DNA sequences of equal length, no gaps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k = 1.333 differences per pair · π = 0.3333 per site

A sample of 3 aligned sequences (AAAA AAAT AATT); the segregating sites (marked s, read from the sample) are shaded where a sequence differs from the first. The package returns the mean number of differences between two sequences k = 1.333 and the nucleotide diversity π = 0.3333.

```
computational_biology_ai999x.solve('bio_nucleotide_diversity', seqs='AAAA AAAT AATT')
# -> k = 1.33333, pi = 0.333333   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_nucleotide_diversity', seqs='ACGTACGT ACGTACGA ACGAACGA ACGTACGT')
# -> k = 1.16667, pi = 0.145833   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nucleotide diversity: seqs = AAAA AAAT AATT	SOLVED	k = 1.33333, pi = 0.333333
Nucleotide diversity: seqs = ACGTACGT ACGTACGA ACGAACGA ACGTACGT	SOLVED	k = 1.16667, pi = 0.145833
Nucleotide diversity:	NEEDS_INPUT	bio_nucleotide_diversity needs seqs (aligned DNA sequences separated by spaces). Example: solve('bio_nucleotide_diversit
Nucleotide diversity: seqs = AAAA AAAT AATT, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Nucleotide diversity: seqs = AAAA AAAT AATT and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Watterson's estimator of theta, `bio_watterson_theta`

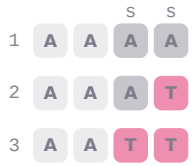
The number of segregating sites S and Watterson's estimator of the population mutation rate theta, per sequence and per site.

$$\hat{\theta}_W = \frac{S}{a_n}, \quad a_n = \sum_{i=1}^{n-1} \frac{1}{i}$$

input	meaning	condition
`seqs`	aligned DNA sequences separated by spaces	

output	meaning
`S`	number of segregating sites
`theta`	Watterson's theta per sequence (S / a_n)
`theta_site`	Watterson's theta per site

- **Method:** count segregating sites and divide by the harmonic number a_n = sum_{$i = 1$}^{$n - 1$} $1/i$
- **Assumes:** at least 2 aligned DNA sequences of equal length, no gaps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A sample of 3 aligned sequences (AAAA AAAT AATT); the segregating sites (marked s) are shaded where a sequence differs from the first. The package counts S = 2 segregating sites and returns Watterson's θ = 1.333 (0.3333 per site).

```
computational_biology_ai999x.solve('bio_watterson_theta', seqs='AAAA AAAT AATT')
# -> S = 2, theta = 1.33333, theta_site = 0.333333   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_watterson_theta', seqs='ACGT ACGA')
# -> S = 1, theta = 1, theta_site = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Watterson theta: seqs = AAAA AAAT AATT	SOLVED	S = 2, theta = 1.33333, theta_site = 0.333333
Watterson theta: seqs = ACGT ACGA	SOLVED	S = 1, theta = 1, theta_site = 0.25
Watterson theta:	NEEDS_INPUT	bio_watterson_theta needs seqs (aligned DNA sequences separated by spaces). Example: solve('bio_watterson_theta', seqs='
Watterson theta: seqs = AAAA AAAT AATT, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Watterson theta: seqs = AAAA AAAT AATT and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Tajima's D, `bio_tajima_d`

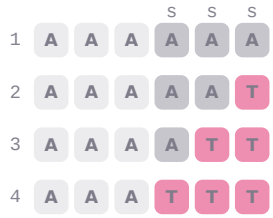
Tajima's D neutrality statistic of aligned DNA sequences (the scaled difference between pi and Watterson's theta).

$$D = \frac{\hat{\theta}_\pi - \hat{\theta}_W}{\sqrt{e_1 S + e_2 S(S-1)}}$$

input	meaning	condition
seqs	aligned DNA sequences separated by spaces (at least 4)	

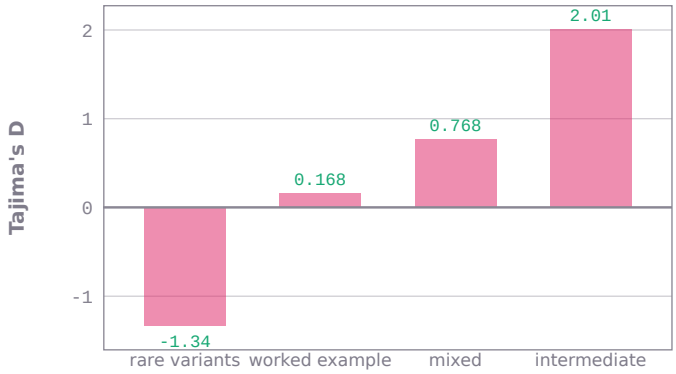
output	meaning
D	Tajima's D
k	mean number of pairwise differences

- **Method:** $D = (k - S/a_1)/sqrt$(e1 $S + e_2$ S $(S-1)$) with Tajima's constants
- **Assumes:** at least 4 aligned DNA sequences; at least one segregating site
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k = 1.667 · θ_W = 1.636 · D = 0.1677

The sample AAAAAA AAAAAT AAAATT AAATTT with its segregating sites marked s and shaded where a sequence differs from the first. Tajima's D compares the mean pairwise difference k = 1.667 (returned with D) with Watterson's θ = 1.636 (the package's Watterson capability on the same sample); the package returns D = 0.1677.



Tajima's D the package returns for four samples: variants each carried by one sequence (an excess of rare variants gives D < 0), the worked example, a mixed sample, and two groups differing at four sites (variants at intermediate frequency give D > 0). Samples: rare variants: AAAAAAAA TAAAAAAA ATAAAAAA AATAAAAA AAATAAAA AAAATAAA; worked example: AAAAAA AAAAAT AAAATT AAATTT; mixed: AAAAAAAA AAAAAAAT AAAAAATT TAAAAAAA AAAAAATT TTAAAAAA; intermediate: AAAAAAAA AAAAAAAA AAAAAAAA TTTTAAAA TTTTAAAA TTTTAAAA.

```
computational_biology_ai999x.solve('bio_tajima_d', seqs='AAAAAA AAAAAT AAAATT AAATTT')
# -> D = 0.167656, k = 1.66667   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_tajima_d', seqs='ACGTACGTAA ACGTACGTAA ACGTACGTAA TCGTACGTAA')
# -> D = -0.612372, k = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tajima's D: seqs = AAAAAA AAAAAT AAAATT AAATTT	SOLVED	D = 0.167656, k = 1.66667
Tajima's D: seqs = ACGTACGTAA ACGTACGTAA ACGTACGTAA TCGTACGTAA	SOLVED	D = -0.612372, k = 0.5
Tajima's D:	NEEDS_INPUT	bio_tajima_d needs seqs (aligned DNA sequences separated by spaces (at least 4)). Example: solve('bio_tajima_d', seqs='A
Tajima's D: seqs = AAAAAA AAAAAT AAAATT AAATTT, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Tajima's D: seqs = AAAAAA AAAAAT AAAATT AAATTT and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Lineweaver-Burk fit of Michaelis-Menten data, `bio_lineweaver_burk`

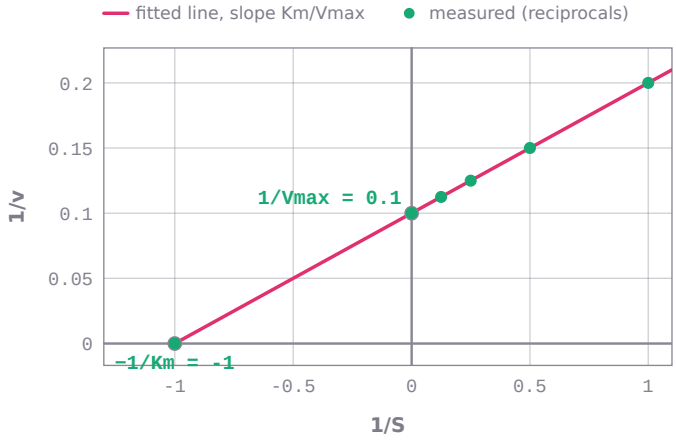
Vmax and Km from a least-squares line through the double-reciprocal (Lineweaver-Burk) plot of $1/v$ against $1/S$.

$$\frac{1}{v} = \frac{K_m}{V_{max}}\frac{1}{S} + \frac{1}{V_{max}}$$

input	meaning	condition
S	substrate concentrations (any one unit)	
v	initial rates (any one unit)	

output	meaning
Vmax	maximum rate (unit of v)
Km	Michaelis constant (unit of S)
r2	coefficient of determination of the linear fit

- **Method:** ordinary least squares of $1/v$ on $1/S$: intercept 1/Vmax, slope Km/Vmax
- **Assumes:** positive S and v; positive slope and intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lineweaver–Burk (double-reciprocal) plot of the measurements S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]: points at (1/S, 1/v), and the straight line 1/v = (Km/Vmax)(1/S) + 1/Vmax drawn with the package's fit Vmax = 10, Km = 1 (r^2 = 1). Marked: the intercepts 1/Vmax = 0.1 and −1/Km = -1, derived from the returned values.

```
computational_biology_ai999x.solve('bio_lineweaver_burk', S=[1, 2, 4, 8], v=[5, 6.6666666667, 8, 8.8888888889])
# -> Vmax = 10, Km = 1, r2 = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_lineweaver_burk', S=[0.5, 1, 2, 5], v=[2.2, 3.5, 5.1, 6.6])
# -> Vmax = 8.69516, Km = 1.47542, r2 = 0.999593   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lineweaver-Burk fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]	SOLVED	Vmax = 10, Km = 1, r2 = 1
Lineweaver-Burk fit: S = [0.5, 1, 2, 5], v = [2.2, 3.5, 5.1, 6.6]	SOLVED	Vmax = 8.69516, Km = 1.47542, r2 = 0.999593
Lineweaver-Burk fit: v = [5, 6.6666666667, 8, 8.8888888889]	NEEDS_INPUT	bio_lineweaver_burk needs S (substrate concentrations (any one unit)). Example: solve('bio_lineweaver_burk', S=[1, 2, 4,
Lineweaver-Burk fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Lineweaver-Burk fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Hanes-Woolf fit of Michaelis-Menten data, `bio_hanes_woolf`

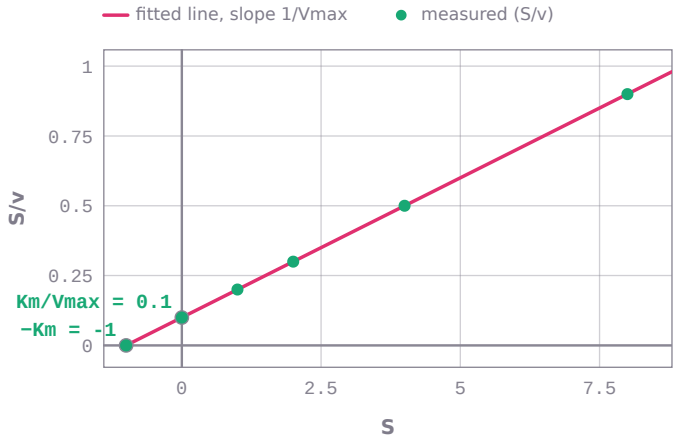
Vmax and Km from a least-squares line through the Hanes-Woolf plot of S/v against S.

$$\frac{S}{v} = \frac{1}{V_{max}}S + \frac{K_m}{V_{max}}$$

input	meaning	condition
S	substrate concentrations (any one unit)	
v	initial rates (any one unit)	

output	meaning
`Vmax`	maximum rate (unit of v)
`Km`	Michaelis constant (unit of S)
`r2`	coefficient of determination of the linear fit

- **Method:** ordinary least squares of S/v on S: slope 1/Vmax, intercept Km/Vmax
- **Assumes:** positive S and v; positive slope and intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hanes–Woolf plot of S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]: points at (S, S/v) and the line S/v = S/Vmax + Km/Vmax with the package's fit Vmax = 10, Km = 1 (r^2 = 1). Marked: the intercepts −Km and Km/Vmax = 0.1, derived from the returned values.

```
computational_biology_ai999x.solve('bio_hanes_woolf', S=[1, 2, 4, 8], v=[5, 6.6666666667, 8, 8.8888888889])
# -> Vmax = 10, Km = 1, r2 = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hanes_woolf', S=[0.5, 1, 2, 5], v=[2.2, 3.5, 5.1, 6.6])
# -> Vmax = 8.47376, Km = 1.39737, r2 = 0.999394   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hanes-Woolf fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]	SOLVED	Vmax = 10, Km = 1, r2 = 1
Hanes-Woolf fit: S = [0.5, 1, 2, 5], v = [2.2, 3.5, 5.1, 6.6]	SOLVED	Vmax = 8.47376, Km = 1.39737, r2 = 0.999394
Hanes-Woolf fit: v = [5, 6.6666666667, 8, 8.8888888889]	NEEDS_INPUT	bio_hanes_woolf needs S (substrate concentrations (any one unit)). Example: solve('bio_hanes_woolf', S=[1, 2, 4, 8], v=[
Hanes-Woolf fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hanes-Woolf fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Eadie-Hofstee fit of Michaelis-Menten data, `bio_eadie_hofstee`

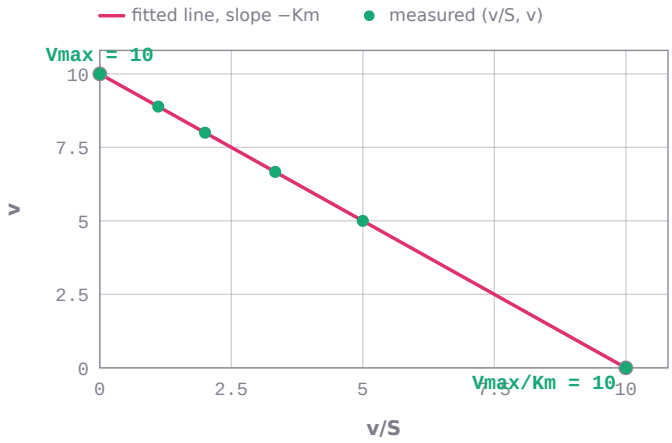
Vmax and Km from a least-squares line through the Eadie-Hofstee plot of v against v/S.

$$v = V_{\max} - K_m \frac{v}{S}$$

input	meaning	condition
`S`	substrate concentrations (any one unit)	
`v`	initial rates (any one unit)	

output	meaning
`Vmax`	maximum rate (unit of v)
`Km`	Michaelis constant (unit of S)
`r2`	coefficient of determination of the linear fit

- **Method:** ordinary least squares of v on v/S: intercept Vmax, slope -Km
- **Assumes:** positive S and v; negative slope, positive intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Eadie–Hofstee plot of S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]: points at (v/S, v) and the line v = Vmax − Km·(v/S) with the package's fit Vmax = 10, Km = 1 (r^2 = 1). Marked: the intercepts Vmax and Vmax/Km = 10 (derived).

```
computational_biology_ai999x.solve('bio_eadie_hofstee', S=[1, 2, 4, 8], v=[5, 6.6666666667, 8, 8.8888888889])
# -> Vmax = 10, Km = 1, r2 = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_eadie_hofstee', S=[0.5, 1, 2, 5], v=[2.2, 3.5, 5.1, 6.6])
# -> Vmax = 8.60596, Km = 1.44637, r2 = 0.995784   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Eadie-Hofstee fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]	SOLVED	Vmax = 10, Km = 1, r2 = 1
Eadie-Hofstee fit: S = [0.5, 1, 2, 5], v = [2.2, 3.5, 5.1, 6.6]	SOLVED	Vmax = 8.60596, Km = 1.44637, r2 = 0.995784
Eadie-Hofstee fit: v = [5, 6.6666666667, 8, 8.8888888889]	NEEDS_INPUT	bio_eadie_hofstee needs S (substrate concentrations (any one unit)). Example: solve('bio_eadie_hofstee', S=[1, 2, 4, 8],
Eadie-Hofstee fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Eadie-Hofstee fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Michaelis-Menten nonlinear least-squares fit, `bio_michaelis_menten_fit`

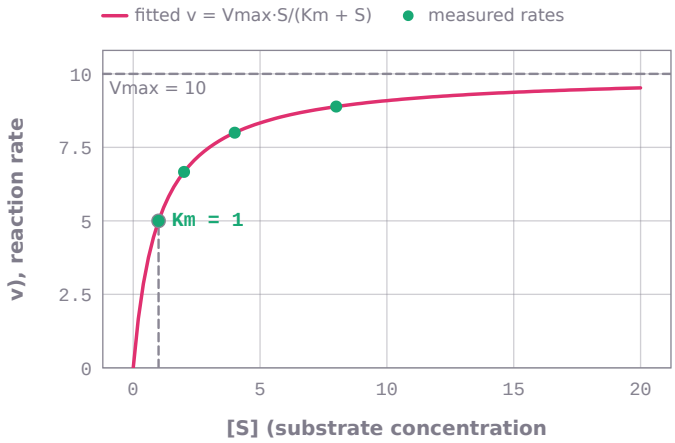
Vmax and Km of the Michaelis-Menten equation fitted to rate data by nonlinear least squares (unweighted).

$$\min_{V_{max},K_m} \sum_i \left(v_i - \frac{V_{max} S_i}{K_m + S_i}\right)^2$$

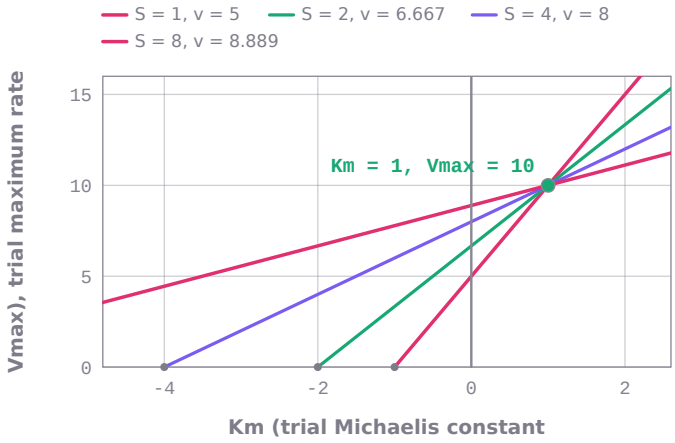
input	meaning	condition
`S`	substrate concentrations (any one unit)	
`v`	initial rates (any one unit)	

output	meaning
`Vmax`	maximum rate (unit of v)
`Km`	Michaelis constant (unit of S)

- **Method:** variable projection: Vmax is linear given Km; the best Km zeroes the derivative of the projected fit
- **Assumes:** at least 3 distinct positive S; positive v; an interior least-squares optimum
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Michaelis–Menten fit to 4 measured rates (S = [1, 2, 4, 8]): the package returns Vmax = 10 and Km = 1; the curve drawn is v = Vmax·S/(Km + S) with those two values (the double-reciprocal fit gives Vmax = 10, Km = 1, r² = 1). Marked: S = Km = 1, v = Vmax/2 = 5.



Direct linear plot of the 4 measured rates (S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]): each measurement is the line through (−S, 0) and (0, v), the set of (Km, Vmax) pairs that would explain it exactly. The lines meet at the parameters the package's nonlinear fit returns. Marked: Km = 1, Vmax = 10.

```
computational_biology_ai999x.solve('bio_michaelis_menten_fit', S=[1, 2, 4, 8], v=[5, 6.6666666667, 8, 8.8888888889])
# -> Vmax = 10, Km = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_michaelis_menten_fit', S=[0.5, 1, 2, 5], v=[2.2, 3.5, 5.1, 6.6])
# -> Vmax = 8.50006, Km = 1.40001   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Michaelis-Menten fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889]	SOLVED	Vmax = 10, Km = 1
Michaelis-Menten fit: S = [0.5, 1, 2, 5], v = [2.2, 3.5, 5.1, 6.6]	SOLVED	Vmax = 8.50006, Km = 1.40001
Michaelis-Menten fit: v = [5, 6.6666666667, 8, 8.8888888889]	NEEDS_INPUT	bio_michaelis_menten_fit needs S (substrate concentrations (any one unit)). Example: solve('bio_michaelis_menten_fit', S
Michaelis-Menten fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Michaelis-Menten fit: S = [1, 2, 4, 8], v = [5, 6.6666666667, 8, 8.8888888889] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Substrate remaining in a Michaelis-Menten reaction, `bio_michaelis_menten_progress`

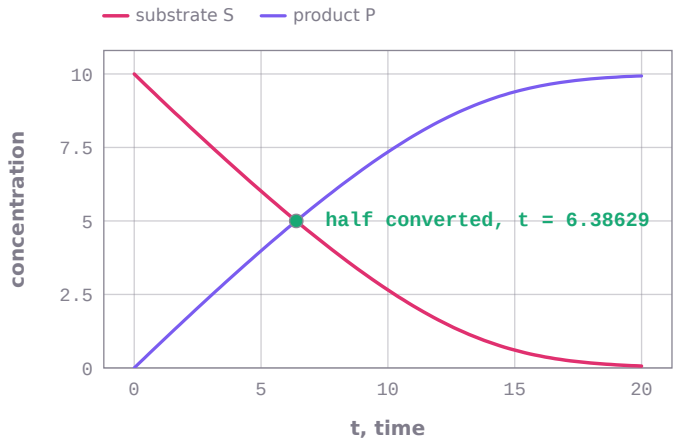
The substrate and product concentrations after a time t of an irreversible Michaelis-Menten reaction (integrated rate law; constant enzyme, no product inhibition).

$$K_m \ln\frac{S_0}{S} + (S_0 - S) = V_{\max} t$$

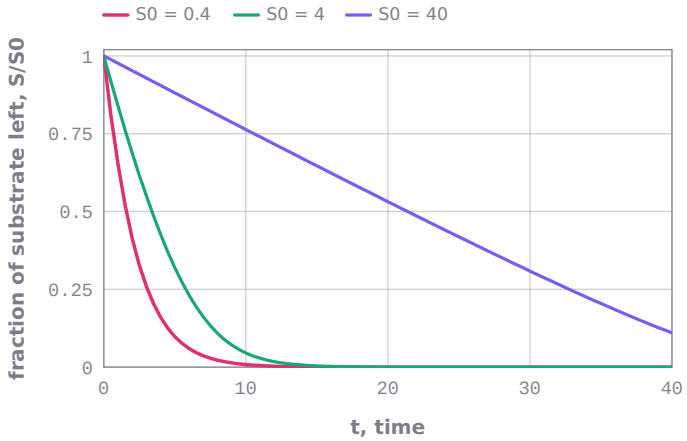
input	meaning	condition
`S0`	initial substrate concentration	
`Vmax`	maximum rate (concentration per time)	
`Km`	Michaelis constant (concentration)	
`t`	elapsed time (time unit of Vmax)	

output	meaning
S	substrate concentration at t
P	product concentration at t

- **Method:** solve Km ln(S0/S) + (S0 - S) = Vmax t by bisection (for P below half conversion, for ln S above)
- **Assumes:** positive S0, Vmax, Km; $t \geq 0$; consistent units
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Substrate and product over time for an enzyme reaction with S0 = 10, Vmax = 1 and Km = 2 (integrated Michaelis–Menten law), computed by the package at 101 times. Half the substrate is converted at t = 6.38629; marked: t = 6.38629, S = 5.



Substrate left as a fraction of the start, S/S0, against time for an enzyme with Vmax = 1 and Km = 2 (the worked example's) starting from S0 = 0.4, 4 and 40: S is the package's integrated Michaelis–Menten value at 80 times, divided by S0. Far below Km the decay is exponential; far above it the enzyme is saturated and S falls at a nearly constant rate.

```
computational_biology_ai999x.solve('bio_michaelis_menten_progress', S0=10, Vmax=1, Km=2, t=5)
# -> S = 6.01624, P = 3.98376   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_michaelis_menten_progress', S0=1, Vmax=0.5, Km=10, t=2)
# -> S = 0.912765, P = 0.0872347   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Michaelis-Menten progress curve: S0 = 10, Vmax = 1, Km = 2, t = 5	SOLVED	S = 6.01624, P = 3.98376
Michaelis-Menten progress curve: S0 = 1, Vmax = 0.5, Km = 10, t = 2	SOLVED	S = 0.912765, P = 0.0872347
Michaelis-Menten progress curve: Vmax = 1, Km = 2, t = 5	NEEDS_INPUT	bio_michaelis_menten_progress needs S0 (initial substrate concentration). Example: solve('bio_michaelis_menten_progress'
Michaelis-Menten progress curve: S0 = 10, Vmax = 1, Km = 2, t = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Michaelis-Menten progress curve: S0 = 10, Vmax = 1, Km = 2, t = 5 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Time to reach a conversion in a Michaelis-Menten reaction, `bio_michaelis_menten_time`

The time an irreversible Michaelis-Menten reaction needs to convert a stated fraction of its substrate.

$$t = \frac{K_m \ln \frac{1}{1-X} + S_0 X}{V_{max}}$$

input	meaning	condition
`S0`	initial substrate concentration	
`Vmax`	maximum rate (concentration per time)	
`Km`	Michaelis constant (concentration)	
`X`	fraction of substrate converted (0 to 1, or a percentage)	

output	meaning
`t`	time (time unit of Vmax)

- **Method:** integrated Michaelis-Menten equation
- **Assumes:** $0 < X < 1$; positive S0, Vmax, Km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Time an enzyme needs to convert a fraction X of its substrate (S0 = 10, Vmax = 1) for Km = 0.5, 2 and 8, each curve 49 runs of the package; the last few per cent take longest. Marked: the worked example X = 0.5, Km = 2, t = 6.386.

```
computational_biology_ai999x.solve('bio_michaelis_menten_time', S0=10, Vmax=1, Km=2, X=0.5)
# -> t = 6.38629   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_michaelis_menten_time', S0=100, Vmax=5, Km=1, X=0.9)
# -> t = 18.4605   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Michaelis-Menten time to conversion: S0 = 10, Vmax = 1, Km = 2, X = 0.5	SOLVED	t = 6.38629
Michaelis-Menten time to conversion: S0 = 100, Vmax = 5, Km = 1, X = 0.9	SOLVED	t = 18.4605
Michaelis-Menten time to conversion: Vmax = 1, Km = 2, X = 0.5	NEEDS_INPUT	bio_michaelis_menten_time needs S0 (initial substrate concentration). Example: solve('bio_michaelis_menten_time', S0=10,
Michaelis-Menten time to conversion: S0 = 10, Vmax = 1, Km = 2, X = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Michaelis-Menten time to conversion: S0 = 10, Vmax = 1, Km = 2, X = 0.5 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Michaelis-Menten time to conversion: S0 = 100, Vmax = 5, Km = 1, X = 90%	SOLVED	t = 18.4605
Michaelis-Menten time to conversion: S0 = 100, Vmax = 5, Km = 1, X = 90	INVALID_INPUT	Time to reach a conversion in a Michaelis-Menten reaction: the fraction converted must lie strictly between 0 and 1.

Hill coefficient from a Hill plot, `bio_hill_plot`

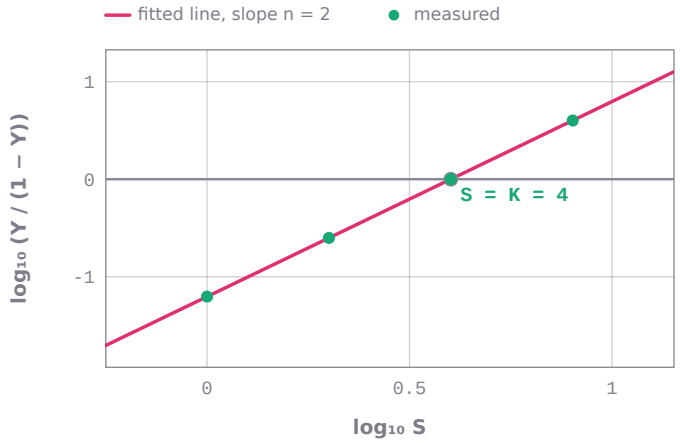
The Hill coefficient n and half-saturation constant K from a least-squares line through the Hill plot of log10($Y/(1-Y)$) against log10 S (Y = fractional saturation).

$$\log \frac{Y}{1-Y} = n \log S - n \log K$$

input	meaning	condition
`S`	ligand or substrate concentrations	
`Y`	fractional saturations $(0 < Y < 1)$	

output	meaning
`n`	Hill coefficient
`K`	half-saturation concentration (unit of S)
`r2`	coefficient of determination of the line

- **Method:** ordinary least squares of log10($Y/(1-Y)$) on log10 S: slope n, intercept -n log10 K
- **Assumes:** $0 < Y < 1$; positive S; rising Hill plot
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hill plot of the fractional saturations Y = [0.0588235294118, 0.2, 0.5, 0.8] at S = [1, 2, 4, 8]: points at (log S, log(Y/(1 − Y))) and the line of slope n through log K, with the package's fit n = 2, K = 4 (r² = 1); a slope above 1 means cooperative binding. Marked: half saturation at S = K.

```
computational_biology_ai999x.solve('bio_hill_plot', S=[1, 2, 4, 8], Y=[0.0588235294118, 0.2, 0.5, 0.8])
# -> n = 2, K = 4, r2 = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hill_plot', S=[10, 20, 40], Y=[0.1, 0.4, 0.8])
# -> n = 2.58496, K = 23.3965, r2 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hill plot: S = [1, 2, 4, 8], Y = [0.0588235294118, 0.2, 0.5, 0.8]	SOLVED	n = 2, K = 4, r2 = 1
Hill plot: S = [10, 20, 40], Y = [0.1, 0.4, 0.8]	SOLVED	n = 2.58496, K = 23.3965, r2 = 1
Hill plot: Y = [0.0588235294118, 0.2, 0.5, 0.8]	NEEDS_INPUT	bio_hill_plot needs S (ligand or substrate concentrations). Example: solve('bio_hill_plot', S=[1, 2, 4, 8], Y=[0.0588235
Hill plot: S = [1, 2, 4, 8], Y = [0.0588235294118, 0.2, 0.5, 0.8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hill plot: S = [1, 2, 4, 8], Y = [0.0588235294118, 0.2, 0.5, 0.8] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Hill equation nonlinear least-squares fit, `bio_hill_fit`

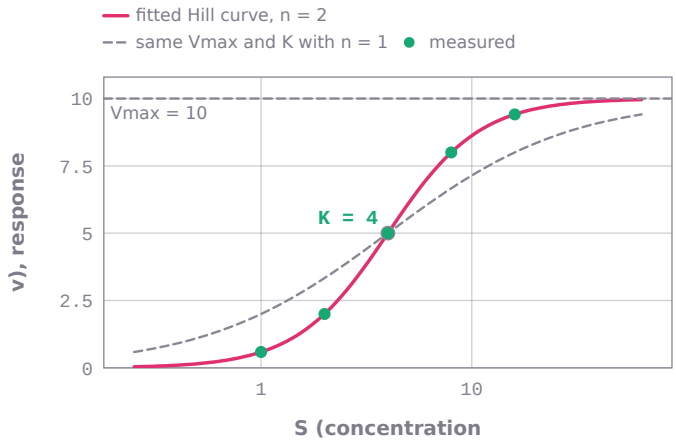
Vmax, the half-saturation constant K and the Hill coefficient n fitted to rate or binding data by nonlinear least squares (unweighted).

$$v = \frac{V_{\max} S^n}{K^n + S^n}$$

input	meaning	condition
`S`	concentrations (any one unit)	
`v`	rates or responses (any one unit)	

output	meaning
`Vmax`	maximum rate
`K`	half-saturation concentration (unit of S)
`n`	Hill coefficient

- **Method:** Levenberg-Marquardt on v = Vmax $S^n/(K^n + S^n)$, started from the Hill-plot line
- **Assumes:** at least 4 distinct positive concentrations; positive responses; convergence
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hill-equation fit to v = [0.588235294118, 2, 5, 8, 9.41176470588] at S = [1, 2, 4, 8, 16] (logarithmic S): the curve v = Vmax·Sⁿ/(Kⁿ + Sⁿ) drawn with the package's fit Vmax = 10, K = 4, n = 2, beside the non-cooperative curve with the same Vmax and K (n = 1, dashed) for comparison. Marked: S = K, v = Vmax/2.

```
computational_biology_ai999x.solve('bio_hill_fit', S=[1, 2, 4, 8, 16], v=[0.588235294118, 2, 5, 8, 9.41176470588])
# -> Vmax = 10, K = 4, n = 2   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hill_fit', S=[0.5, 1, 2, 4, 8, 16], v=[0.3, 1.1, 3.2, 6.1, 8.3, 9.2])
# -> Vmax = 9.58247, K = 2.9375, n = 1.86335   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hill fit: S = [1, 2, 4, 8, 16], v = [0.588235294118, 2, 5, 8, 9.41176470588]	SOLVED	Vmax = 10, K = 4, n = 2
Hill fit: S = [0.5, 1, 2, 4, 8, 16], v = [0.3, 1.1, 3.2, 6.1, 8.3, 9.2]	SOLVED	Vmax = 9.58247, K = 2.9375, n = 1.86335
Hill fit: v = [0.588235294118, 2, 5, 8, 9.41176470588]	NEEDS_INPUT	bio_hill_fit needs S (concentrations (any one unit)). Example: solve('bio_hill_fit', S=[1, 2, 4, 8, 16], v=[0.5882352941
Hill fit: S = [1, 2, 4, 8, 16], v = [0.588235294118, 2, 5, 8, 9.41176470588], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Hill fit: S = [1, 2, 4, 8, 16], v = [0.588235294118, 2, 5, 8, 9.41176470588] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Competitive inhibition global fit, `bio_competitive_inhibition_fit`

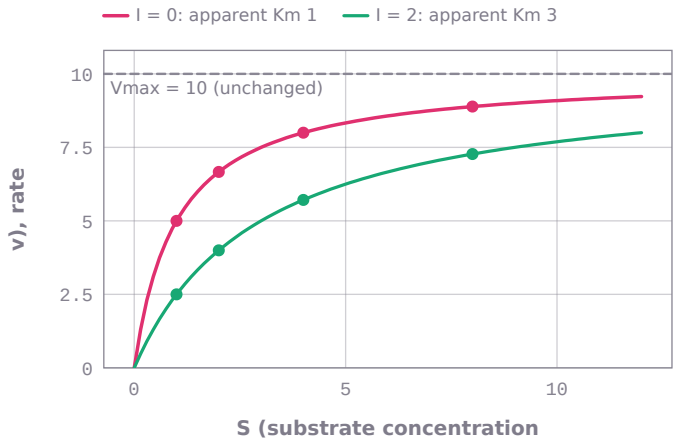
Vmax, Km and the inhibition constant Ki of a competitive inhibitor fitted globally to rates measured at several substrate and inhibitor concentrations (nonlinear least squares, unweighted).

$$v = \frac{V_{max} S}{K_m \left(1 + \frac{[I]}{K_i}\right) + S}$$

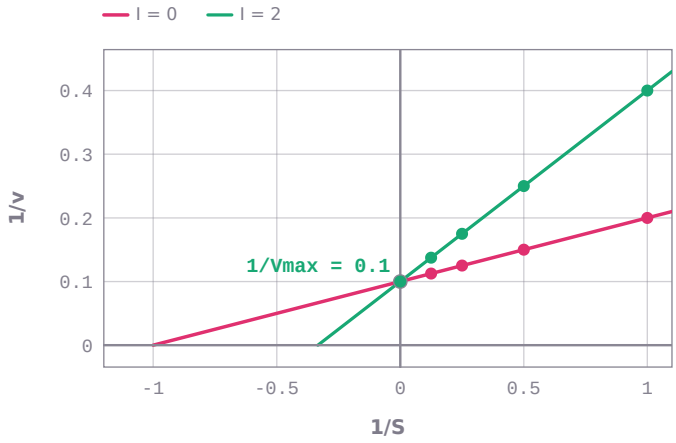
input	meaning	condition
`S`	substrate concentrations (one per measurement)	
`I`	inhibitor concentrations (one per measurement, 0 for none)	
`v`	measured rates	

output	meaning
`Vmax`	maximum rate
`Km`	Michaelis constant (unit of S)
`Ki`	inhibition constant (unit of I)

- **Method:** Levenberg-Marquardt on v = Vmax S / (Km (1 + I/Ki) + S), started from the uninhibited fit
- **Assumes:** rates without inhibitor at 3 or more S; rates with inhibitor; convergence
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rates measured at S = [1, 2, 4, 8, 1, 2, 4, 8] with inhibitor I = [0, 0, 0, 0, 2, 2, 2, 2] (dots), and the curves v = Vmax·S/(Km(1 + I/Ki) + S) drawn with the package's fit Vmax = 10, Km = 1, Ki = 1. A competitive inhibitor raises the apparent Km (Km(1 + I/Ki), derived) and leaves Vmax unchanged.



The same measurements in double-reciprocal form (points at 1/S, 1/v) with the lines of the package's fit Vmax = 10, Km = 1, Ki = 1: a competitive inhibitor turns the line about the common intercept 1/Vmax (marked, derived), the signature of competition for the active site.

```
computational_biology_ai999x.solve('bio_competitive_inhibition_fit', S=[1, 2, 4, 8, 1, 2, 4, 8], I=[0, 0, 0, 0, 2, 2, 2, 2], v=[5, 6.66666666667, 8, 8.88888888889, 2.5, 4, 5.71428571429, 7.27272727273])
# -> Vmax = 10, Km = 1, Ki = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Competitive inhibition fit: S = [1, 2, 4, 8, 1, 2, 4, 8], I = [0, 0, 0, 0, 2, 2, 2, 2], v = [5, 6.66666666667, 8, 8.88888888889, 2.5, 4, 5.71428571429, 7.27272727273]	SOLVED	Vmax = 10, Km = 1, Ki = 1
Competitive inhibition fit: I = [0, 0, 0, 0, 2, 2, 2, 2], v = [5, 6.66666666667, 8, 8.88888888889, 2.5, 4, 5.71428571429, 7.27272727273]	NEEDS_INPUT	bio_competitive_inhibition_fit needs S (substrate concentrations (one per measurement)). Example: solve('bio_competitive
Competitive inhibition fit: S = [1, 2, 4, 8, 1, 2, 4, 8], I = [0, 0, 0, 0, 2, 2, 2, 2], v = [5, 6.66666666667, 8, 8.88888888889, 2.5, 4, 5.71428571429, 7.27272727273], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Competitive inhibition fit: S = [1, 2, 4, 8, 1, 2, 4, 8], I = [0, 0, 0, 0, 2, 2, 2, 2], v = [5, 6.66666666667, 8, 8.88888888889, 2.5, 4, 5.71428571429, 7.27272727273] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Aromaticity of a protein, `bio_aromaticity`

The aromaticity (relative frequency of Phe + Trp + Tyr, Lobry 1994) of a protein and its number of aromatic residues.

$$\mathrm{Ar} = \frac{n_F + n_W + n_Y}{n}$$

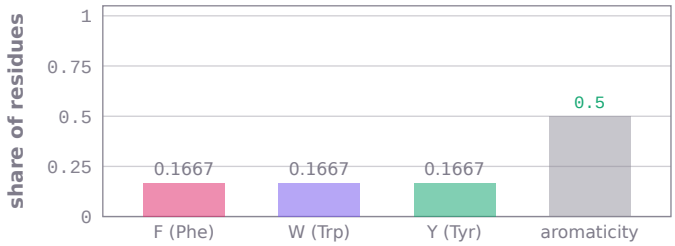
input	meaning	condition
`seq`	protein sequence	

output	meaning
`aromaticity`	fraction of F, W and Y residues
`aromatic_count`	number of F, W and Y residues

- **Method:** count F, W and Y and divide by the length

- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)





The worked example's 6-residue protein written 6 residues per row (row start numbers on the left), phenylalanine F, tryptophan W and tyrosine Y shaded; below, the share of each (counted from the sequence) and the aromaticity the package returns: aromatic_count = 3, aromaticity = 0.5.



The worked example MFWYAK: its aromatic residues F, W and Y shaded; the package returns aromatic_count = 3 and aromaticity = 0.5.

```
computational_biology_ai999x.solve('bio_aromaticity', seq='MFWYAK')
# -> aromaticity = 0.5, aromatic_count = 3   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_aromaticity', seq='GGGG')
# -> aromaticity = 0, aromatic_count = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Aromaticity: seq = MFWYAK	SOLVED	aromaticity = 0.5, aromatic_count = 3
Aromaticity: seq = GGGG	SOLVED	aromaticity = 0, aromatic_count = 0
Aromaticity:	NEEDS_INPUT	bio_aromaticity needs seq (protein sequence). Example: solve('bio_aromaticity', seq='MFWYAK').
Aromaticity: seq = MFWYAK, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Aromaticity: seq = MFWYAK and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Amino-acid composition of a protein, `bio_amino_acid_composition`

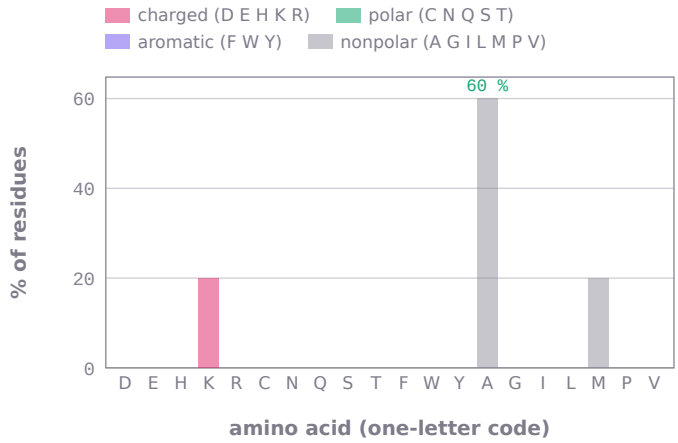
The percentage of each of the 20 standard amino acids in a protein, its most common residue (alphabetically first among ties) and its length.

$$\%a = 100\,\frac{n_a}{n}$$

input	meaning	condition
`seq`	protein sequence	

output	meaning
`percent`	percentage of each amino acid (one-letter code -> %)
`most_common`	most common amino acid (alphabetically first among ties)
`length`	number of residues

- **Method:** count each residue and divide by the length
- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Amino-acid composition of the worked example's 5-residue protein (MAAAK): one bar per amino acid, the percentage the package returns, grouped by side-chain class. Marked: the most common residue, A (60 %).

```
computational_biology_ai999x.solve('bio_amino_acid_composition', seq='MAAAK')
# -> percent = {'A': 60.0, 'C': 0.0, 'D': 0.0, 'E': 0.0, 'F': 0.0, 'G': 0.0, 'H': 0.0, 'I': 0.0, 'K': 20.0, 'L': 0.0,
'M': 20.0, 'N': 0.0, 'P': 0.0, 'Q': 0.0, 'R': 0.0, 'S': 0.0, 'T': 0.0, 'V': 0.0, 'W': 0.0, 'Y': 0.0}, most_common = A,
length = 5   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_amino_acid_composition',
seq='MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGL
# -> percent = {'A': 3.94736842105, 'C': 1.97368421053, 'D': 3.28947368421, 'E': 7.89473684211, 'F': 3.94736842105,
'G': 9.21052631579, 'H': 3.28947368421, 'I': 3.28947368421, 'K': 7.89473684211, 'L': 11.8421052632, 'M': 1.31578947368,
'N': 4.60526315789, 'P': 5.26315789474, 'Q': 3.94736842105, 'R': 3.94736842105, 'S': 6.57894736842, 'T': 8.55263157895,
'V': 3.28947368421, 'W': 0.657894736842, 'Y': 5.26315789474}, most_common = L, length = 152   (SOLVED)
```

Questions in words (documented forms), as executed:

question
Amino acid composition: seq = MAAAK
Amino acid composition: seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEK
Amino acid composition:
Amino acid composition: seq = MAAAK, zz = 1
Amino acid composition: seq = MAAAK and 7 more

Secondary-structure propensity fractions, `bio_secondary_structure_fraction`

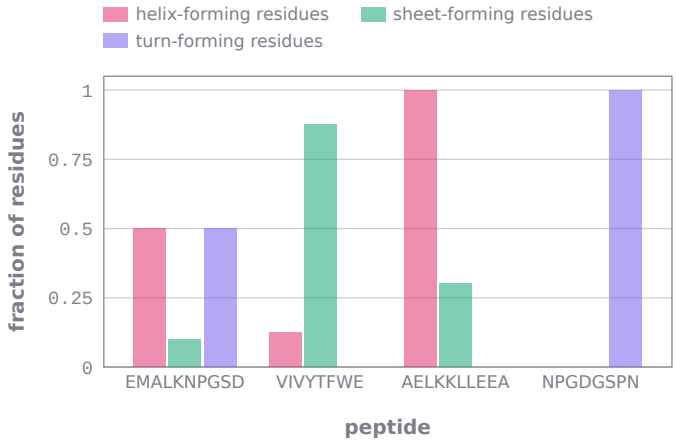
The fractions of residues that tend to be in helices (E M A L K), turns (N P G S D) and sheets (V I Y F W L T), as ProtParam defines them (L counts for helix and sheet).

$$f_{\text{helix}} = \frac{n_E + n_M + n_A + n_L + n_K}{n}$$

input	meaning	condition
`seq`	protein sequence	

output	meaning
`helix`	fraction of helix-favouring residues
`turn`	fraction of turn-favouring residues
`sheet`	fraction of sheet-favouring residues

- **Method:** count the residues of each propensity group
- **Assumes:** standard amino acids only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Fractions of helix-, sheet- and turn-forming residues the package returns for four peptides: the worked example EMALKNPGSD (helix 0.5, turn 0.5, sheet 0.1), a β-branched VIVYTFWE, a helix-like AELKKLLEEA and a turn-rich NPGDGSPN. The three classes overlap, so the bars of a peptide need not sum to 1.

```
computational_biology_ai999x.solve('bio_secondary_structure_fraction', seq='EMALKNPGSD')
# -> helix = 0.5, turn = 0.5, sheet = 0.1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_secondary_structure_fraction', seq='VVVV')
# -> helix = 0, turn = 0, sheet = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Secondary structure fraction: seq = EMALKNPGSD	SOLVED	helix = 0.5, turn = 0.5, sheet = 0.1
Secondary structure fraction: seq = VVVV	SOLVED	helix = 0, turn = 0, sheet = 1
Secondary structure fraction:	NEEDS_INPUT	bio_secondary_structure_fraction needs seq (protein sequence). Example: solve('bio_secondary_structure_fraction', seq='E
Secondary structure fraction: seq = EMALKNPGSD, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Secondary structure fraction: seq = EMALKNPGSD and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Leslie matrix population projection, `bio_leslie_projection`

The age-class numbers and total population after t time steps of an age-structured (Leslie) model.

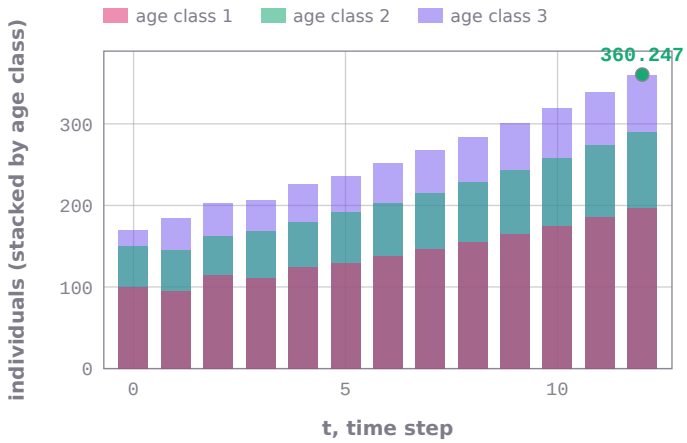
$$\mathbf{n}_{t+1} = L\,\mathbf{n}_t, \quad L = \begin{pmatrix} F_1 & F_2 & \cdots \\ S_1 & 0 & \\ & \ddots & \end{pmatrix}$$

input	meaning	condition
`F`	fecundity of each age class	
`S`	survival from each age class to the next (one fewer than F)	
`n0`	initial number in each age class	
`t`	number of time steps	

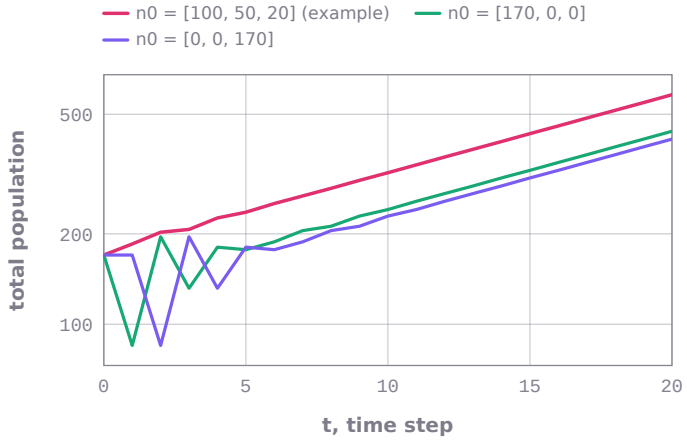
output	meaning
`n_t`	number in each age class after t steps

output	meaning
`total`	total population after t steps

- **Method:** multiply the age vector by the Leslie matrix t times
- **Assumes:** $0 < S_i \leq 1$; $F_i \geq 0$, at least one positive; $t \leq 500$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Leslie-matrix projection of an age-structured population (fecundities F = [0, 1.5, 1], survival S = [0.5, 0.8], start n0 = [100, 50, 20]): each bar is the population the package returns at that step, split by age class. Growth settles to λ = 1.06152 per step (r = 0.0596993); at t = 12 the classes are [197.307, 92.9426, 69.9967]; marked: total = 360.247.



Total population projected by the Leslie matrix F = [0, 1.5, 1], S = [0.5, 0.8] from three starts of 170 individuals (the worked example n0 = [100, 50, 20], all newborn, all in the oldest class) on a logarithmic axis, each step a run of the package: after the starting oscillations all three grow in parallel at the dominant rate λ = 1.062 per step (the package's growth-rate capability).

```
computational_biology_ai999x.solve('bio_leslie_projection', F=[0, 1.5, 1], S=[0.5, 0.8], n0=[100, 50, 20], t=1)
# -> n_t = [95.0, 50.0, 40.0], total = 185   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_leslie_projection', F=[0, 2], S=[0.5], n0=[10, 0], t=2)
# -> n_t = [10.0, 0.0], total = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Leslie projection: F = [0, 1.5, 1], S = [0.5, 0.8], n0 = [100, 50, 20], t = 1	SOLVED	n_t = [95.0, 50.0, 40.0], total = 185
Leslie projection: F = [0, 2], S = [0.5], n0 = [10, 0], t = 2	SOLVED	n_t = [10.0, 0.0], total = 10
Leslie projection: S = [0.5, 0.8], n0 = [100, 50, 20], t = 1	NEEDS_INPUT	bio_leslie_projection needs F (fecundity of each age class). Example: solve('bio_leslie_projection', F=[0, 1.5, 1], S=[0
Leslie projection: F = [0, 1.5, 1], S = [0.5, 0.8], n0 = [100, 50, 20], t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Leslie projection: F = [0, 1.5, 1], S = [0.5, 0.8], n0 = [100, 50, 20], t = 1 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Leslie matrix growth rate and stable age distribution, `bio_leslie_growth_rate`

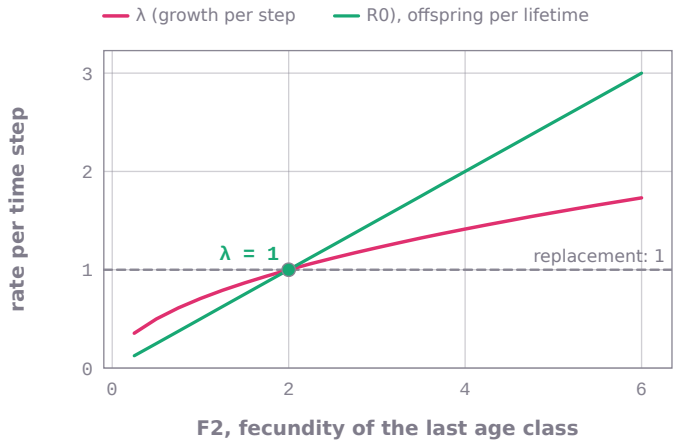
The asymptotic growth rate (dominant eigenvalue lambda and $r = ln$ lambda), the net reproductive rate R0 and the stable age distribution of a Leslie model.

$$\sum_i F_i\, l_i\, \lambda^{-i} = 1, \quad R_0 = \sum_i F_i l_i$$

input	meaning	condition
`F`	fecundity of each age class	
`S`	survival from each age class to the next (one fewer than F)	

output	meaning
`lambda1`	dominant eigenvalue (growth factor per step)
`r`	intrinsic rate ln(lambda)
`R0`	net reproductive rate (expected offspring per newborn)
`stable`	stable age distribution (fractions summing to 1)

- **Method:** the positive root of the Euler-Lotka equation sum F_i l_i $\lambda^{-i} = 1$; stable distribution proportional to l_i lambda^-$(i-1)$
- **Assumes:** $0 < S_i \leq 1$; $F_i \geq 0$, at least one positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Growth rate λ and net reproductive rate R0 of the Leslie matrix F = [0, 2], S = [0.5] as the fecundity of the last class runs from 0.25 to 6, each point a run of the package: both cross 1 at the same fecundity, where the population just replaces itself. Marked: the worked example, F = [0, 2].

```
computational_biology_ai999x.solve('bio_leslie_growth_rate', F=[0, 2], S=[0.5])
# -> lambda1 = 1, r = 0, R0 = 1, stable = [0.666666666667, 0.333333333333]   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_leslie_growth_rate', F=[0, 1.5, 1], S=[0.5, 0.8])
# -> lambda1 = 1.06152, r = 0.0596993, R0 = 1.15, stable = [0.547643499894, 0.257953155019, 0.194403345087]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Leslie growth rate: F = [0, 2], S = [0.5]	SOLVED	lambda1 = 1, r = 0, R0 = 1, stable = [0.666666666667, 0.333333333333]
Leslie growth rate: F = [0, 1.5, 1], S = [0.5, 0.8]	SOLVED	lambda1 = 1.06152, r = 0.0596993, R0 = 1.15, stable = [0.547643499894, 0.257953155019, 0.194403345087]
Leslie growth rate: S = [0.5]	NEEDS_INPUT	bio_leslie_growth_rate needs F (fecundity of each age class). Example: solve('bio_leslie_growth_rate', F=[0, 2], S=[0.5]
Leslie growth rate: F = [0, 2], S = [0.5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Leslie growth rate: F = [0, 2], S = [0.5] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Shannon diversity index, `bio_shannon_diversity`

The Shannon diversity index H' of a community in the stated logarithm base, Pielou's evenness J' and the effective number of species exp(H').

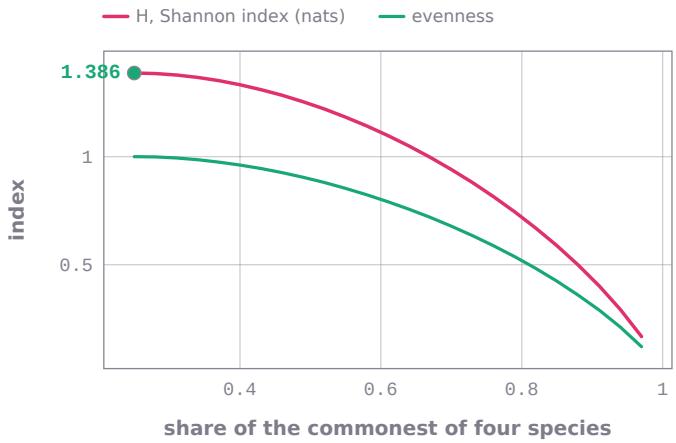
$$H' = -\sum_i p_i \log_b p_i, \quad J' = \frac{H'}{\log_b S}$$

input	meaning	condition
`counts`	number of individuals of each species	
`base`	logarithm base: e (nats), 2 (bits) or 10	

output	meaning
`H`	Shannon index H' in the stated base
`evenness`	Pielou's evenness J' = H'/ln S

output	meaning
`effective_number`	effective number of species exp(H') (Hill number of order 1)

- **Method:** H' = -sum p_i log p_i with $p_i = n_i/N$
- **Assumes:** whole non-negative counts; zero counts are not species of the sample
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Shannon index H and Pielou evenness for communities of four species and 300 individuals, from perfectly even ([75, 75, 75, 75]) to dominated by one species ([291, 3, 3, 3]): 25 communities, each a run of the package (base = e). Marked: the worked example counts = [10, 10, 10, 10], H = 1.386.

```
computational_biology_ai999x.solve('bio_shannon_diversity', counts=[10, 10, 10, 10], base='e')
# -> H = 1.38629, evenness = 1, effective_number = 4   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_shannon_diversity', counts=[50, 30, 20], base='2')
# -> H = 1.48548, evenness = 0.937231, effective_number = 2.80009   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shannon diversity: counts = [10, 10, 10, 10], base = e	SOLVED	H = 1.38629, evenness = 1, effective_number = 4
Shannon diversity: counts = [50, 30, 20], base = 2	SOLVED	H = 1.48548, evenness = 0.937231, effective_number = 2.80009
Shannon diversity: base = e	NEEDS_INPUT	bio_shannon_diversity needs counts (number of individuals of each species). Example: solve('bio_shannon_diversity', coun
Shannon diversity: counts = [10, 10, 10, 10], base = e, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Shannon diversity: counts = [10, 10, 10, 10], base = e and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Shannon diversity: counts = [10, 10, 10, 10]	NEEDS_INPUT	bio_shannon_diversity needs base (logarithm base: e (nats), 2 (bits) or 10). Example: solve('bio_shannon_diversity', cou

Simpson diversity indices, `bio_simpson_diversity`

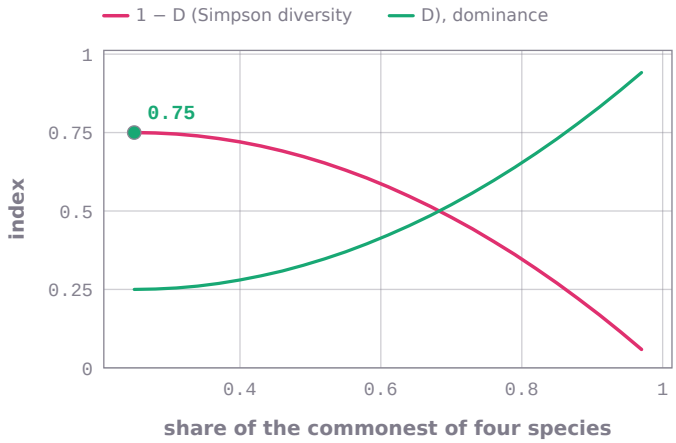
Simpson's dominance D, Simpson's diversity $1 - D$ and the inverse Simpson index $1/D$, with the stated estimator (infinite population: sum p^2; finite sample: sum n($n - 1$)/(N(N-1))).

$$D = \sum_i p_i^2 \quad \text{or} \quad D = \frac{\sum n_i(n_i - 1)}{N(N-1)}$$

input	meaning	condition
`counts`	number of individuals of each species	
`estimator`	infinite (sum p^2) or finite-sample correction	

output	meaning
`D`	Simpson's dominance D
`simpson`	Simpson's diversity $1 - D$
`inverse`	inverse Simpson $1/D$

- **Method:** D = sum p_i^2 or sum n_i($n_i - 1$)/(N(N - 1))
- **Assumes:** whole non-negative counts; estimator stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Simpson diversity 1 − D and the probability D that two individuals belong to the same species for communities of four species and 300 individuals, from perfectly even ([75, 75, 75, 75]) to dominated by one species ([291, 3, 3, 3]): 25 communities, each a run of the package (estimator = infinite). Marked: the worked example counts = [10, 10, 10, 10], simpson = 0.75.

```
computational_biology_ai999x.solve('bio_simpson_diversity', counts=[10, 10, 10, 10], estimator='infinite')
# -> D = 0.25, simpson = 0.75, inverse = 4   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_simpson_diversity', counts=[3, 2, 1], estimator='finite')
# -> D = 0.266667, simpson = 0.733333, inverse = 3.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Simpson diversity: counts = [10, 10, 10, 10], estimator = infinite	SOLVED	D = 0.25, simpson = 0.75, inverse = 4
Simpson diversity: counts = [3, 2, 1], estimator = finite	SOLVED	D = 0.266667, simpson = 0.733333, inverse = 3.75
Simpson diversity: estimator = infinite	NEEDS_INPUT	bio_simpson_diversity needs counts (number of individuals of each species). Example: solve('bio_simpson_diversity', coun
Simpson diversity: counts = [10, 10, 10, 10], estimator = infinite, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Simpson diversity: counts = [10, 10, 10, 10], estimator = infinite and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Chao1 species richness estimator, `bio_chao1`

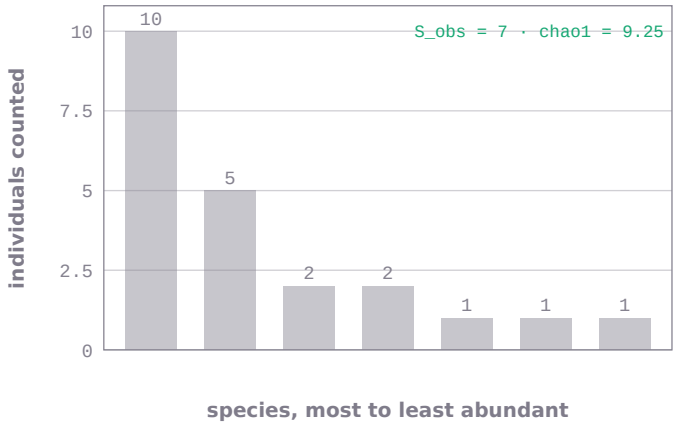
The Chao1 estimate of total species richness from abundance counts (bias-corrected or classic form), with the observed richness and the numbers of singletons and doubletons.

$$\hat{S} = S_{obs} + \frac{F_1(F_1 - 1)}{2(F_2 + 1)}$$

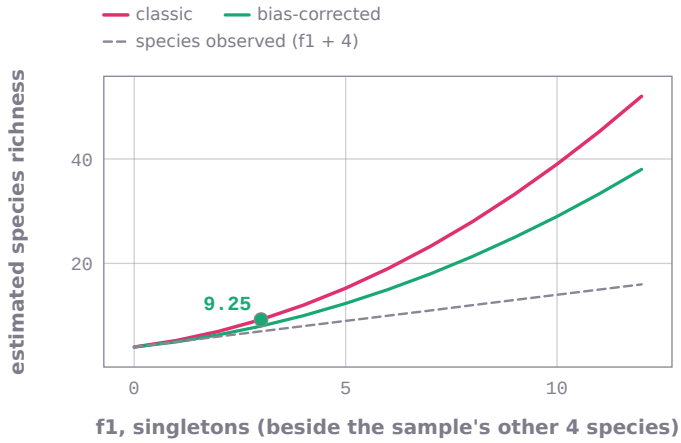
input	meaning	condition
`counts`	number of individuals of each species	
`form`	bias_corrected or classic Chao1	

output	meaning
`chao1`	estimated species richness
`s_obs`	observed number of species
`singletons`	species seen once (F1)
`doubletons`	species seen twice (F2)

- **Method:** S_obs + F1^2/(2 F2) (classic) or S_obs + F1($F_1 - 1$)/(2(F2 + 1)) (bias-corrected)
- **Assumes:** whole counts; the classic form needs $F_1 > 0$ and $F_2 > 0$, otherwise the bias-corrected form is used (as scikit-bio does)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The sample counts = [1, 1, 1, 2, 2, 5, 10] as a rank-abundance bar chart: 7 species observed, 3 of them once and 2 twice. From these rare counts the package's Chao1 (classic) estimates the species present, seen or not: chao1 = 9.25.



Chao1 richness of samples with f1 = 0 … 12 singletons beside the worked example's other species (counts [2, 2, 5, 10]), by the classic and the bias-corrected formula, each point a run of the package; the dashed line is the number of species observed. With few doubletons, singletons weigh heavily. Marked: the worked example itself (3 singletons, classic form): 9.25.

```
computational_biology_ai999x.solve('bio_chao1', counts=[1, 1, 1, 2, 2, 5, 10], form='classic')
# -> chao1 = 9.25, s_obs = 7, singletons = 3, doubletons = 2   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_chao1', counts=[1, 1, 1, 2, 2, 5, 10], form='bias_corrected')
# -> chao1 = 8, s_obs = 7, singletons = 3, doubletons = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chao1: counts = [1, 1, 1, 2, 2, 5, 10], form = classic	SOLVED	chao1 = 9.25, s_obs = 7, singletons = 3, doubletons = 2
Chao1: counts = [1, 1, 1, 2, 2, 5, 10], form = bias_corrected	SOLVED	chao1 = 8, s_obs = 7, singletons = 3, doubletons = 2
Chao1: form = classic	NEEDS_INPUT	bio_chao1 needs counts (number of individuals of each species). Example: solve('bio_chao1', counts=[1, 1, 1, 2, 2, 5, 10
Chao1: counts = [1, 1, 1, 2, 2, 5, 10], form = classic, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Chao1: counts = [1, 1, 1, 2, 2, 5, 10], form = classic and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

ACE species richness estimator, `bio_ace`

The abundance-based coverage estimator (ACE) of species richness, with a stated rare-species threshold (10 is the usual value).

$$S_{ace} = S_{abund} + \frac{S_{rare}}{C_{ace}} + \frac{F_1}{C_{ace}}\gamma^2$$

input	meaning	condition
`counts`	number of individuals of each species	
`rare_threshold`	largest count of a rare species (usually 10)	

output	meaning
`ace`	estimated species richness

- **Method:** abundant species counted directly; rare species scaled by the sample coverage C = 1 - F1/N_rare with the coefficient-of-variation term (EstimateS form, as scikit-bio)
- **Assumes:** whole counts; not every rare species a singleton
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The sample counts = [1, 1, 2, 3, 5, 8, 20, 50] (logarithmic), with the rare-species threshold 10: ACE estimates the richness from the species at or below it. 8 species are observed (the bars); the package returns ace = 9.719.

```
computational_biology_ai999x.solve('bio_ace', counts=[1, 1, 2, 3, 5, 8, 20, 50], rare_threshold=10)
# -> ace = 9.7193   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_ace', counts=[12, 15, 20], rare_threshold=10)
# -> ace = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ACE richness estimator: counts = [1, 1, 2, 3, 5, 8, 20, 50], rare_threshold = 10	SOLVED	ace = 9.7193
ACE richness estimator: counts = [12, 15, 20], rare_threshold = 10	SOLVED	ace = 3
ACE richness estimator: rare_threshold = 10	NEEDS_INPUT	bio_ace needs counts (number of individuals of each species). Example: solve('bio_ace', counts=[1, 1, 2, 3, 5, 8, 20, 50
ACE richness estimator: counts = [1, 1, 2, 3, 5, 8, 20, 50], rare_threshold = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
ACE richness estimator: counts = [1, 1, 2, 3, 5, 8, 20, 50], rare_threshold = 10 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Good's coverage of a sample, `bio_goods_coverage`

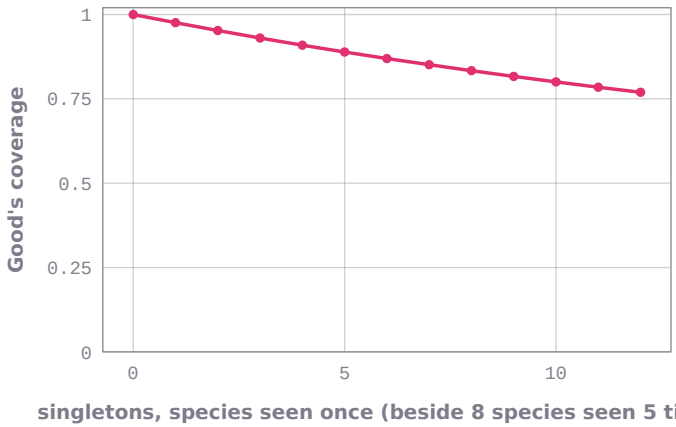
Good's coverage estimator: the estimated fraction of the community's individuals belonging to species already seen in the sample.

$$C = 1 - \frac{F_1}{N}$$

input	meaning	condition
`counts`	number of individuals of each species	

output	meaning
`coverage`	Good's coverage $1 - F_1/N$

- **Method:** one minus the proportion of individuals that are singletons
- **Assumes:** whole counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Good's sample coverage of samples made of 8 species seen 5 times each plus 0 … 12 species seen once: each point a run of the package. Every singleton is evidence of species not yet seen, and coverage falls as they accumulate. The worked example counts = [1, 1, 2, 6] gives coverage = 0.8.

```
computational_biology_ai999x.solve('bio_goods_coverage', counts=[1, 1, 2, 6])
# -> coverage = 0.8   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_goods_coverage', counts=[5, 5])
# -> coverage = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Good's coverage: counts = [1, 1, 2, 6]	SOLVED	coverage = 0.8
Good's coverage: counts = [5, 5]	SOLVED	coverage = 1
Good's coverage:	NEEDS_INPUT	bio_goods_coverage needs counts (number of individuals of each species). Example: solve('bio_goods_coverage', counts=[1,
Good's coverage: counts = [1, 1, 2, 6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Good's coverage: counts = [1, 1, 2, 6] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Margalef and Menhinick richness indices, `bio_richness_indices`

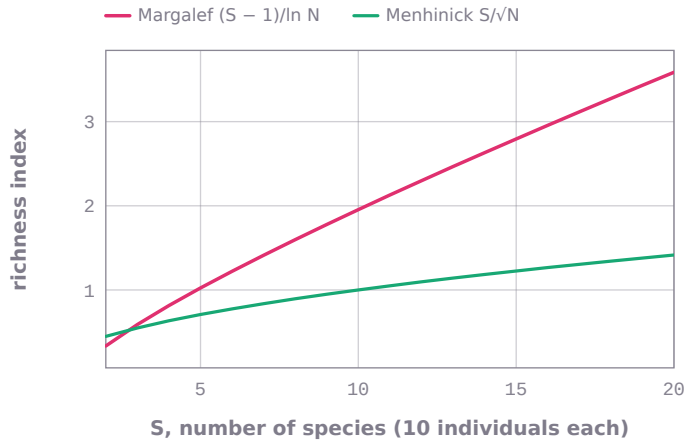
Margalef's richness index $(S-1)/ln$ N and Menhinick's richness index S/sqrt(N) of a sample.

$$D_{Mg} = \frac{S-1}{\ln N}, \quad D_{Mn} = \frac{S}{\sqrt{N}}$$

input	meaning	condition
`counts`	number of individuals of each species	

output	meaning
`margalef`	Margalef's index $(S-1)/ln$ N
`menhinick`	Menhinick's index S/sqrt(N)

- **Method:** species and individuals counted
- **Assumes:** whole counts; $N \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Margalef and Menhinick richness indices for samples of S = 2 … 20 species with 10 individuals each (N = 10·S), each point a run of the package: both grow with S though N grows with it. The worked example counts = [10, 20, 30, 40] gives margalef = 0.6514, menhinick = 0.4.

```
computational_biology_ai999x.solve('bio_richness_indices', counts=[10, 20, 30, 40])
# -> margalef = 0.651442, menhinick = 0.4   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_richness_indices', counts=[1, 1, 1, 1])
# -> margalef = 2.16404, menhinick = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Richness indices: counts = [10, 20, 30, 40]	SOLVED	margalef = 0.651442, menhinick = 0.4
Richness indices: counts = [1, 1, 1, 1]	SOLVED	margalef = 2.16404, menhinick = 2
Richness indices:	NEEDS_INPUT	bio_richness_indices needs counts (number of individuals of each species). Example: solve('bio_richness_indices', counts
Richness indices: counts = [10, 20, 30, 40], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Richness indices: counts = [10, 20, 30, 40] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Berger-Parker and McIntosh indices, `bio_dominance_indices`

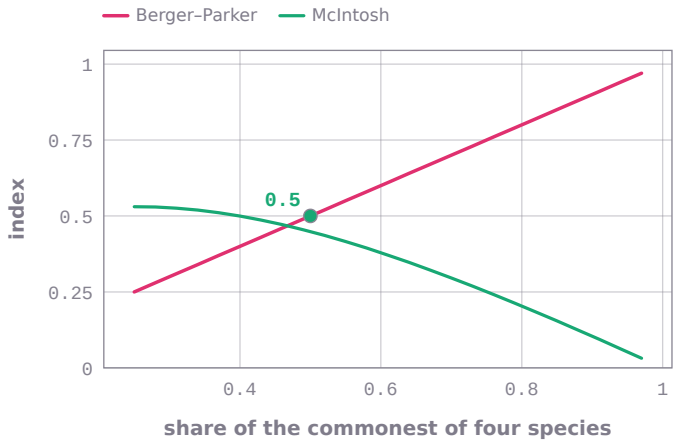
The Berger-Parker dominance (share of the most abundant species) and McIntosh's index of a sample.

$$d = \frac{n_{\max}}{N}, \quad D = \frac{N - U}{N - \sqrt{N}}$$

input	meaning	condition
`counts`	number of individuals of each species	

output	meaning
`berger_parker`	Berger-Parker dominance n_max/N
`mcintosh`	McIntosh index (N - U)/(N - sqrt N), $U = sqrt$(sum n_i^2)

- **Method:** largest count over N; McIntosh's U statistic
- **Assumes:** whole counts; $N \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Berger–Parker index (share of the commonest species) and McIntosh dominance for communities of four species and 300 individuals, from perfectly even ([75, 75, 75, 75]) to dominated by one species ([291, 3, 3, 3]): 25 communities, each a run of the package. Marked: the worked example counts = [50, 30, 20], berger_parker = 0.5.

```
computational_biology_ai999x.solve('bio_dominance_indices', counts=[50, 30, 20])
# -> berger_parker = 0.5, mcintosh = 0.426176   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_dominance_indices', counts=[25, 25, 25, 25])
# -> berger_parker = 0.25, mcintosh = 0.555556   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Berger-Parker and McIntosh indices: counts = [50, 30, 20]	SOLVED	berger_parker = 0.5, mcintosh = 0.426176
Berger-Parker and McIntosh indices: counts = [25, 25, 25, 25]	SOLVED	berger_parker = 0.25, mcintosh = 0.555556
Berger-Parker and McIntosh indices:	NEEDS_INPUT	bio_dominance_indices needs counts (number of individuals of each species). Example: solve('bio_dominance_indices', coun
Berger-Parker and McIntosh indices: counts = [50, 30, 20], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Berger-Parker and McIntosh indices: counts = [50, 30, 20] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Brillouin diversity index, `bio_brillouin_diversity`

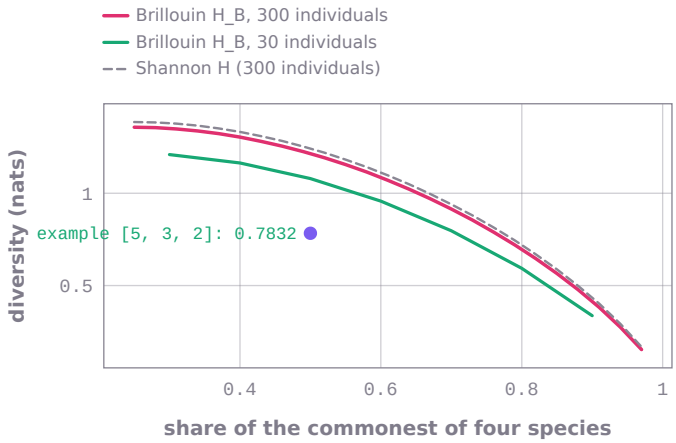
Brillouin's diversity index of a fully censused collection (natural logarithm).

$$H_B = \frac{\ln N! - \sum_i \ln n_i!}{N}$$

input	meaning	condition
`counts`	number of individuals of each species	

output	meaning
`H_B`	Brillouin index (nats per individual)

- **Method:** logarithm of the multinomial coefficient over N (exact integer)
- **Assumes:** whole counts; $N \leq 20000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Brillouin index of fully censused four-species collections of 300 and of 30 individuals along a gradient from even to dominated, each point a run of the package, with the Shannon index of the 300-individual collections (the package's Shannon capability, natural logarithm) dashed: Brillouin's index is lower, and more so for small collections. Marked: the worked example counts = [5, 3, 2] (three species), H_B = 0.7832.

```
computational_biology_ai999x.solve('bio_brillouin_diversity', counts=[5, 3, 2])
# -> H_B = 0.783201   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_brillouin_diversity', counts=[1, 1])
# -> H_B = 0.346574   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Brillouin diversity: counts = [5, 3, 2]	SOLVED	H_B = 0.783201
Brillouin diversity: counts = [1, 1]	SOLVED	H_B = 0.346574
Brillouin diversity:	NEEDS_INPUT	bio_brillouin_diversity needs counts (number of individuals of each species). Example: solve('bio_brillouin_diversity',
Brillouin diversity: counts = [5, 3, 2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Brillouin diversity: counts = [5, 3, 2] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Rarefied species richness, `bio_rarefaction`

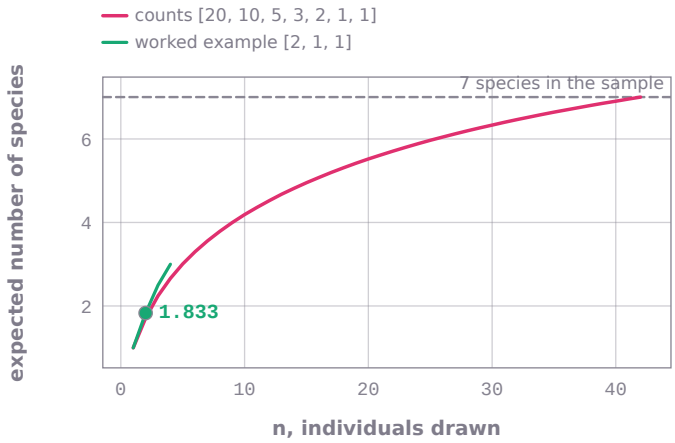
The expected number of species in a random subsample of n individuals drawn without replacement (Hurlbert-Sanders rarefaction).

$$E[S_n] = \sum_i \left[1 - \binom{N - N_i}{n} \Big/ \binom{N}{n}\right]$$

input	meaning	condition
`counts`	number of individuals of each species	
`n`	subsample size (individuals)	

output	meaning
`expected_species`	expected number of species in the subsample

- **Method:** one minus the chance that a species is missed, summed (exact)
- **Assumes:** whole counts; $1 \le n \le N$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rarefaction curves: the expected number of species in a random subsample of n individuals, for a sample with counts [20, 10, 5, 3, 2, 1, 1] (n = 1 … 42) and for the worked example counts [2, 1, 1]; every point a run of the package. The curve flattens as the rare species are all found. Marked: the worked example n = 2, 1.833 species.

```
computational_biology_ai999x.solve('bio_rarefaction', counts=[2, 1, 1], n=2)
# -> expected_species = 1.83333   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_rarefaction', counts=[10, 5, 1], n=16)
# -> expected_species = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rarefaction: counts = [2, 1, 1], n = 2	SOLVED	expected_species = 1.83333
Rarefaction: counts = [10, 5, 1], n = 16	SOLVED	expected_species = 3
Rarefaction: n = 2	NEEDS_INPUT	bio_rarefaction needs counts (number of individuals of each species). Example: solve('bio_rarefaction', counts=[2, 1, 1]
Rarefaction: counts = [2, 1, 1], n = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Rarefaction: counts = [2, 1, 1], n = 2 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Beta diversity between two samples, `bio_beta_diversity`

The Bray-Curtis dissimilarity (abundances) and the Jaccard and Sorensen dissimilarities (presence and absence) between two community samples.

$$BC = \frac{\sum |x_i - y_i|}{\sum (x_i + y_i)}, \quad J = 1 - \frac{|A \cap B|}{|A \cup B|}$$

input	meaning	condition
`x`	abundances of each species in the first sample	
`y`	abundances of the same species in the second sample	

output	meaning
`bray_curtis`	Bray-Curtis dissimilarity
`jaccard`	Jaccard dissimilarity (1 - shared/union)
`sorensen`	Sorensen dissimilarity (1 - 2 shared/(S_x + S_y))

- **Method:** sum |x - y| / sum (x + y); presence-absence set differences
- **Assumes:** the same species order in both samples; each sample non-empty
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two community samples, x = [6, 7, 4, 0] and y = [10, 0, 6, 3] (individuals of the same four species), and the dissimilarities the package returns: Bray–Curtis 0.4444 (from the abundances), Jaccard 0.5 and Sørensen 0.3333 (from presence and absence only).

```
computational_biology_ai999x.solve('bio_beta_diversity', x=[6, 7, 4, 0], y=[10, 0, 6, 3])
# -> bray_curtis = 0.444444, jaccard = 0.5, sorensen = 0.333333   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_beta_diversity', x=[1, 2], y=[1, 2])
# -> bray_curtis = 0, jaccard = 0, sorensen = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Beta diversity: x = [6, 7, 4, 0], y = [10, 0, 6, 3]	SOLVED	bray_curtis = 0.444444, jaccard = 0.5, sorensen = 0.333333
Beta diversity: x = [1, 2], y = [1, 2]	SOLVED	bray_curtis = 0, jaccard = 0, sorensen = 0
Beta diversity: y = [10, 0, 6, 3]	NEEDS_INPUT	bio_beta_diversity needs x (abundances of each species in the first sample). Example: solve('bio_beta_diversity', x=[6,
Beta diversity: x = [6, 7, 4, 0], y = [10, 0, 6, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Beta diversity: x = [6, 7, 4, 0], y = [10, 0, 6, 3] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Species-area relationship fit, `bio_species_area`

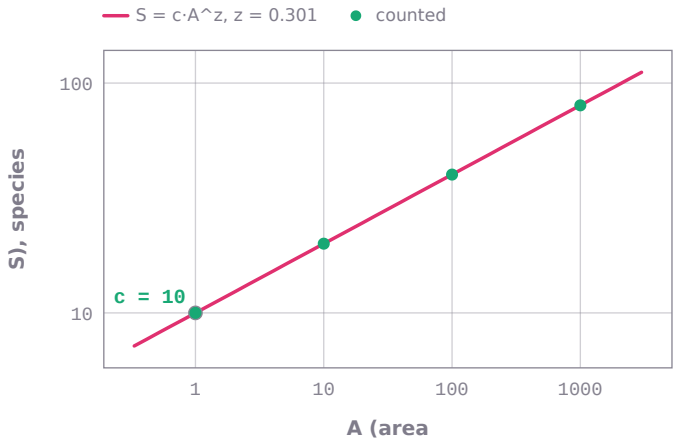
The constants c and z of the Arrhenius species-area relationship $S = c\,A^{z}$ fitted by least squares on log S against log A.

$$\ln S = \ln c + z \ln A$$

input	meaning	condition
A	areas (any one unit)	
S	species counts	

output	meaning
c	constant c (species in a unit area)
z	exponent z
r2	r^2 of the log-log fit

- **Method:** ordinary least squares of ln S on ln A: slope z, intercept ln c
- **Assumes:** positive areas and counts; c refers to the area unit used
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Species–area relation on logarithmic axes: the counts S = [10, 20, 40, 80] on areas A = [1, 10, 100, 1000] (dots) and the power law S = c·A^z drawn with the package's log–log fit c = 10, z = 0.301 ($r^2 = 1$). Marked: c, the species expected on a unit area.

```
computational_biology_ai999x.solve('bio_species_area', A=[1, 10, 100, 1000], S=[10, 20, 40, 80])
# -> c = 10, z = 0.30103, r2 = 1   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_species_area', A=[2, 8], S=[5, 10])
# -> c = 3.53553, z = 0.5, r2 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Species-area relationship: A = [1, 10, 100, 1000], S = [10, 20, 40, 80]	SOLVED	c = 10, z = 0.30103, r2 = 1
Species-area relationship: A = [2, 8], S = [5, 10]	SOLVED	c = 3.53553, z = 0.5, r2 = 1
Species-area relationship: S = [10, 20, 40, 80]	NEEDS_INPUT	bio_species_area needs A (areas (any one unit)). Example: solve('bio_species_area', A=[1, 10, 100, 1000], S=[10, 20, 40,
Species-area relationship: A = [1, 10, 100, 1000], S = [10, 20, 40, 80], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Species-area relationship: A = [1, 10, 100, 1000], S = [10, 20, 40, 80] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Mark-recapture population estimate (Petersen MLE), `bio_petersen_estimate`

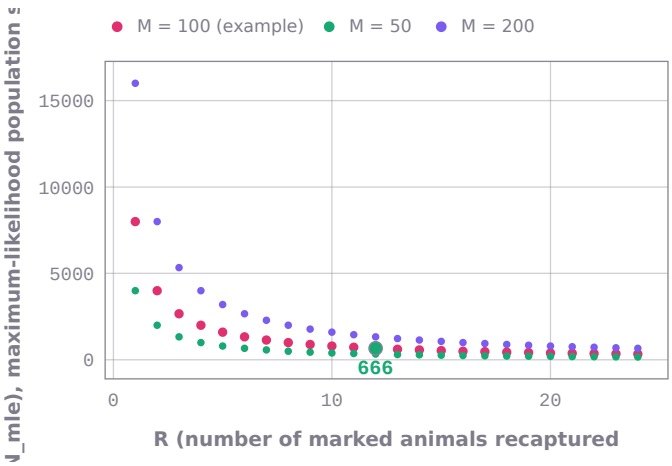
The maximum-likelihood population size from one mark-recapture experiment: M animals marked, C caught later, R of them marked (hypergeometric likelihood).

$$\hat{N} = \left\lfloor \frac{MC}{R} \right\rfloor$$

input	meaning	condition
`M`	number marked and released	
`C`	number caught in the second sample	
`R`	number of marked animals recaptured	

output	meaning
`N_mle`	maximum-likelihood population size (largest maximiser)

- **Method:** the hypergeometric likelihood increases while $N \le MC/R$: the maximiser is floor(MC/R)
- **Assumes:** closed population; $R \ge 1$, R <= min(M, C); when MC/R is whole, $N = MC/R$ and $MC/R - 1$ tie; the larger is given
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



N_mle (maximum-likelihood population size (largest maximiser)) against R (number of marked animals recaptured) from 1 to 24. One curve for each value of M (number marked and released): 100, 50, 200. Other inputs as in the worked example: C = 80. 24 points per curve computed by computational-biology-ai999x; marked: the worked example, R = 12, N_mle = 666.

```
computational_biology_ai999x.solve('bio_petersen_estimate', M=100, C=80, R=12)
# -> N_mle = 666   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_petersen_estimate', M=50, C=40, R=10)
# -> N_mle = 200   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Petersen estimate: M = 100, C = 80, R = 12	SOLVED	N_mle = 666
Petersen estimate: M = 50, C = 40, R = 10	SOLVED	N_mle = 200
Petersen estimate: C = 80, R = 12	NEEDS_INPUT	bio_petersen_estimate needs M (number marked and released). Example: solve('bio_petersen_estimate', M=100, C=80, R=12).
Petersen estimate: M = 100, C = 80, R = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Petersen estimate: M = 100, C = 80, R = 12 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Petersen estimate: M = 10, C = 5, R = 6	INVALID_INPUT	Mark-recapture population estimate (Petersen MLE): R cannot exceed M or C.

Schumacher-Eschmeyer multiple-census estimate, `bio_schumacher_eschmeyer`

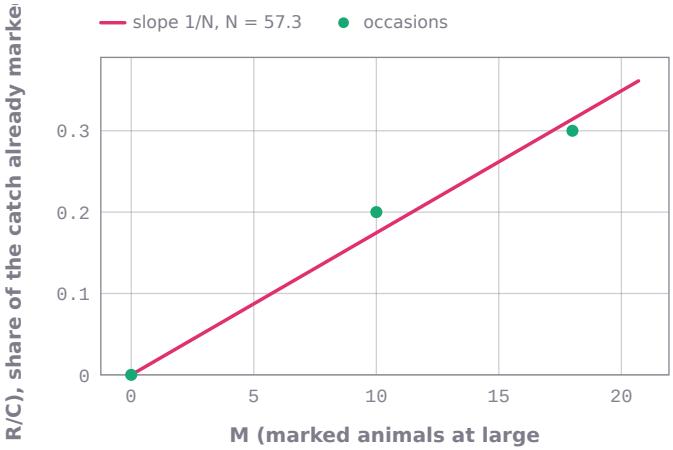
The Schumacher-Eschmeyer estimate of a closed population's size from repeated mark-recapture samples.

$$\hat{N} = \frac{\sum_t C_t M_t^2}{\sum_t R_t M_t}$$

input	meaning	condition
`C`	number caught on each occasion	
`R`	number of those already marked	
`M`	number of marked animals at large before each occasion	

output	meaning
`N`	estimated population size

- **Method:** weighted regression of R/C on M through the origin
- **Assumes:** closed population; whole counts, $R \leq C$; at least one recapture of a marked animal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Schumacher–Eschmeyer mark–recapture: on each occasion the share of the catch already marked, R/C (from C = [10, 10, 10], R = [0, 2, 3]), against the marked animals at large M = [0, 10, 18]. In a closed population R/C ≈ M/N; the line through the origin has slope 1/N with the package's estimate N = 57.3.

```
computational_biology_ai999x.solve('bio_schumacher_eschmeyer', C=[10, 10, 10], R=[0, 2, 3], M=[0, 10, 18])
# -> N = 57.2973   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_schumacher_eschmeyer', C=[50, 40], R=[0, 10], M=[0, 50])
# -> N = 200   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Schumacher-Eschmeyer estimate: C = [10, 10, 10], R = [0, 2, 3], M = [0, 10, 18]	SOLVED	N = 57.2973
Schumacher-Eschmeyer estimate: C = [50, 40], R = [0, 10], M = [0, 50]	SOLVED	N = 200
Schumacher-Eschmeyer estimate: R = [0, 2, 3], M = [0, 10, 18]	NEEDS_INPUT	bio_schumacher_eschmeyer needs C (number caught on each occasion). Example: solve('bio_schumacher_eschmeyer', C=[10, 10,
Schumacher-Eschmeyer estimate: C = [10, 10, 10], R = [0, 2, 3], M = [0, 10, 18], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Schumacher-Eschmeyer estimate: C = [10, 10, 10], R = [0, 2, 3], M = [0, 10, 18] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Chemostat steady state (Monod kinetics), `bio_chemostat_steady_state`

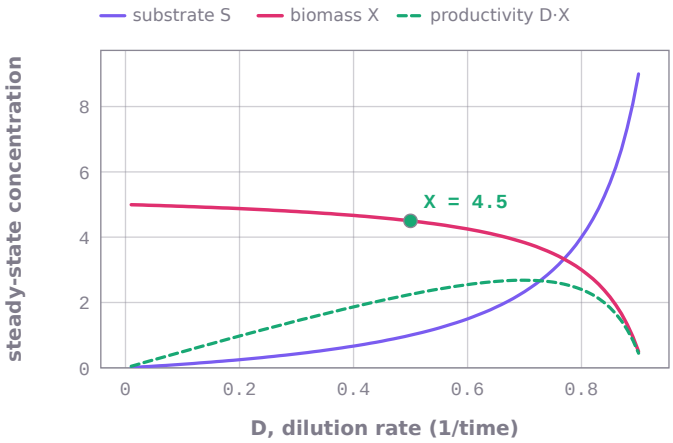
The steady-state substrate and biomass concentrations of a chemostat with Monod growth kinetics.

$$\bar{S} = \frac{K_s D}{\mu_{max} - D}, \quad \bar{X} = Y(S_0 - \bar{S})$$

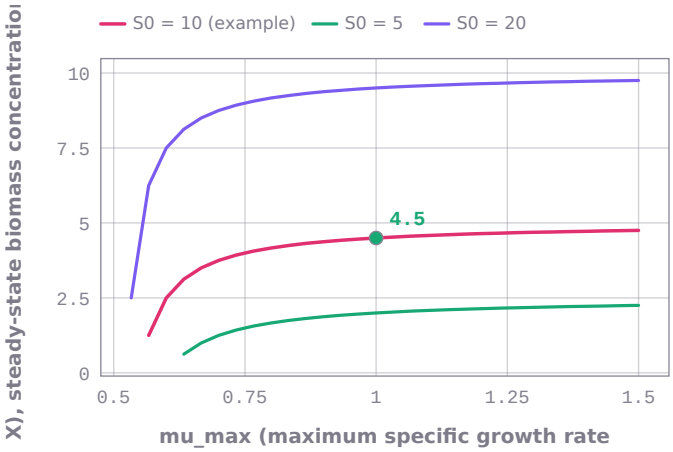
input	meaning	condition
`mu_max`	maximum specific growth rate (1/time)	
`Ks`	half-saturation constant (concentration)	
`Y`	biomass yield (biomass per substrate)	
`D`	dilution rate (1/time, same unit as mu_max)	
`S0`	feed substrate concentration	

output	meaning
`S`	steady-state substrate concentration
`X`	steady-state biomass concentration

- **Method:** growth balances dilution: mu(S) = D gives S; the substrate balance gives X
- **Assumes:** D < mu_max; no washout $(S < S_0)$; consistent units
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Chemostat operating diagram (Monod growth, μmax = 1, Ks = 1, Y = 0.5, feed S0 = 10): steady-state substrate and biomass computed by the package at 90 dilution rates up to washout; biomass productivity D·X peaks near D = 0.7 (2.68333). Marked: D = 0.5, X = 4.5, S = 1.



X (steady-state biomass concentration) against mu_max (maximum specific growth rate (1/time)) from 0.5 to 1.5. One curve for each value of S0 (feed substrate concentration): 10, 5, 20. Other inputs as in the worked example: Ks = 1.0, Y = 0.5, D = 0.5. 29 points per curve computed by computational-biology-ai999x; marked: the worked example, mu_max = 1, X = 4.5.

```
computational_biology_ai999x.solve('bio_chemostat_steady_state', mu_max=1, Ks=1, Y=0.5, D=0.5, S0=10)
# -> S = 1, X = 4.5   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_chemostat_steady_state', mu_max=0.6, Ks=0.2, Y=0.4, D=0.3, S0=5)
# -> S = 0.2, X = 1.92   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chemostat steady state: mu_max = 1, Ks = 1, Y = 0.5, D = 0.5, S0 = 10	SOLVED	S = 1, X = 4.5
Chemostat steady state: mu_max = 0.6, Ks = 0.2, Y = 0.4, D = 0.3, S0 = 5	SOLVED	S = 0.2, X = 1.92
Chemostat steady state: Ks = 1, Y = 0.5, D = 0.5, S0 = 10	NEEDS_INPUT	bio_chemostat_steady_state needs mu_max (maximum specific growth rate (1/time)). Example: solve('bio_chemostat_steady_st
Chemostat steady state: mu_max = 1, Ks = 1, Y = 0.5, D = 0.5, S0 = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Chemostat steady state: mu_max = 1, Ks = 1, Y = 0.5, D = 0.5, S0 = 10 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Chemostat steady state: mu_max = 1, Ks = 1, Y = 0.5, D = 0.95, S0 = 10	INVALID_INPUT	Chemostat steady state (Monod kinetics): washout: the steady-state substrate would exceed the feed concentration.

Flux balance analysis (optimal objective), `bio_flux_balance_analysis`

The maximum of a linear objective (e.g. a biomass or product flux) over all steady-state flux distributions of a metabolic network within flux bounds: flux balance analysis.

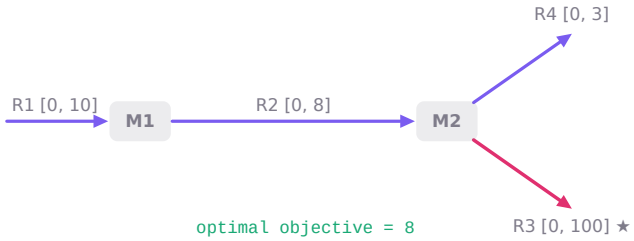
$$\max_{v} \; c^{\top} v \;\; \text{s.t. } Sv = 0, \;\; l \le v \le u$$

input	meaning	condition
`S`	stoichiometric matrix (metabolites x reactions)	
`lb`	lower bound of each reaction flux	

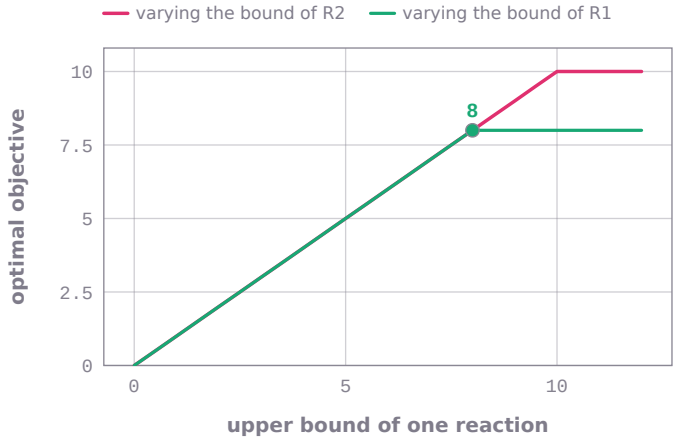
input	meaning	condition
`ub`	upper bound of each reaction flux	
`c`	objective coefficient of each reaction	

output	meaning
`objective`	optimal objective value

- **Method:** linear programming solved exactly in rational arithmetic (two-phase simplex, Bland's rule); the optimal value is unique even when the optimal fluxes are not
- **Assumes:** steady state S $\boldsymbol{v} = \boldsymbol{0}$; finite bounds $\boldsymbol{lb} \leq \boldsymbol{ub}$; a feasible flux distribution exists
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The metabolic network of the stoichiometric matrix S = [[1, -1, 0, 0], [0, 1, -1, -1]] (metabolites M1–M2, reactions R1–R4, each arrow running from what a reaction consumes to what it makes, with its flux bounds [lb, ub] = [0, 0, 0, 0], [10, 8, 100, 3]). ★ marks the reaction the objective c = [0, 0, 1, 0] maximises; under steady state the package's flux balance returns objective = 8.



Optimal objective of the network S = [[1, -1, 0, 0], [0, 1, -1, -1]] (bounds ub = [10, 8, 100, 3], objective c = [0, 0, 1, 0]) as the upper bound of R2, or of the uptake R1, runs from 0 to 12 with the others kept: each point a run of the package. The objective follows the tighter of the two bounds and levels off when the other limits it. Marked: the worked example, objective = 8.

```
computational_biology_ai999x.solve('bio_flux_balance_analysis', S=[[1, -1, 0, 0], [0, 1, -1, -1]], lb=[0, 0, 0, 0], ub=
[10, 8, 100, 3], c=[0, 0, 1, 0])
# -> objective = 8   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_flux_balance_analysis', S=[[1, -1, -1]], lb=[0, 0, 0], ub=[10, 100, 100], c=[0,
1, 2])
# -> objective = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flux balance analysis: S = [[1, -1, 0, 0], [0, 1, -1, -1]], lb = [0, 0, 0, 0], ub = [10, 8, 100, 3], c = [0, 0, 1, 0]	SOLVED	objective = 8
Flux balance analysis: S = [[1, -1, -1]], lb = [0, 0, 0], ub = [10, 100, 100], c = [0, 1, 2]	SOLVED	objective = 20
Flux balance analysis: lb = [0, 0, 0, 0], ub = [10, 8, 100, 3], c = [0, 0, 1, 0]	NEEDS_INPUT	bio_flux_balance_analysis needs S (stoichiometric matrix (metabolites x reactions)). Example: solve('bio_flux_balance_an
Flux balance analysis: S = [[1, -1, 0, 0], [0, 1, -1, -1]], lb = [0, 0, 0, 0], ub = [10, 8, 100, 3], c = [0, 0, 1, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Flux balance analysis: S = [[1, -1, 0, 0], [0, 1, -1, -1]], lb = [0, 0, 0, 0], ub = [10, 8, 100, 3], c = [0, 0, 1, 0] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Faith's phylogenetic diversity, `bio_faith_pd`

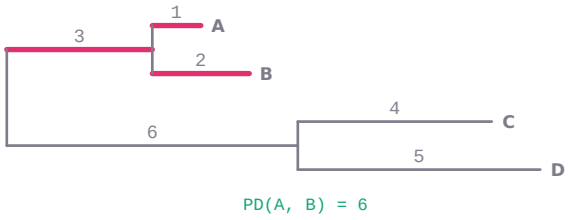
Faith's phylogenetic diversity of a set of observed taxa: the total branch length of the rooted tree connecting them to the root.

$$PD = \sum_{b \in T(\text{taxa}) \cup R} \ell_b$$

input	meaning	condition
`tree`	rooted tree in Newick format with branch lengths	
`taxa`	observed taxa (leaf names separated by spaces)	

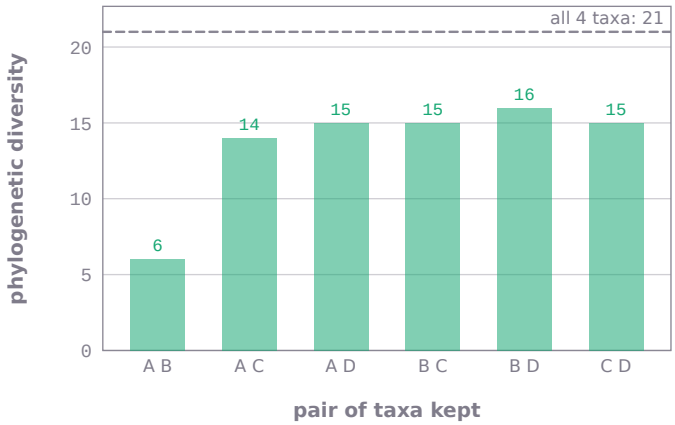
output	meaning
`pd`	Faith's PD (sum of branch lengths)

- **Method:** sum the lengths of every branch on a path from an observed taxon to the root, each once
- **Assumes:** every branch has a length; taxa are leaves of the tree; rooted convention (path to the root included)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



`PD(A, B) = 6`

The tree ((A:1,B:2):3,(C:4,D:5):6); drawn to scale (branch lengths written on the branches). The branches joining the taxa A, B to the root are drawn thick: their summed length is Faith's phylogenetic diversity, which the package returns as pd = 6.



Faith's phylogenetic diversity of every pair of taxa of the tree ((A:1,B:2):3,(C:4,D:5):6);, each bar a run of the package: a pair from different clades keeps more of the tree than two sister taxa. Line: all 4 taxa together, 21.

```
computational_biology_ai999x.solve('bio_faith_pd', tree='((A:1,B:2):3,(C:4,D:5):6);', taxa='A B')
# -> pd = 6   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_faith_pd', tree='((A:1,B:2):3,(C:4,D:5):6);', taxa='A C')
# -> pd = 14   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Faith's phylogenetic diversity: tree = ((A:1,B:2):3, (C:4,D:5):6);, taxa = A B	SOLVED	pd = 6
Faith's phylogenetic diversity: tree = ((A:1,B:2):3, (C:4,D:5):6);, taxa = A C	SOLVED	pd = 14
Faith's phylogenetic diversity: taxa = A B	NEEDS_INPUT	bio_faith_pd needs tree (rooted tree in Newick format with branch lengths). Example: solve('bio_faith_pd', tree='((A:1,B
Faith's phylogenetic diversity: tree = ((A:1,B:2):3, (C:4,D:5):6);, taxa = A B, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Faith's phylogenetic diversity: tree = ((A:1,B:2):3, (C:4,D:5):6);, taxa = A B and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp

Hardy-Weinberg genotype and carrier frequencies, `bio_hardy_weinberg_frequencies`

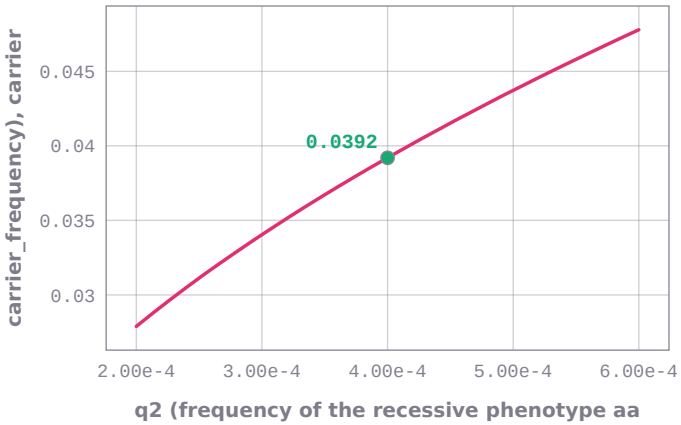
The allele frequencies p and q, the genotype frequencies p^2, 2pq and q^2, the carrier (heterozygote) frequency and the carrier frequency among unaffected people, from one allele frequency or from the frequency of the recessive phenotype; optionally the expected genotype counts in N individuals.

$$p + q = 1, \quad p^2 + 2pq + q^2 = 1, \quad q = \sqrt{f_{aa}}$$

input	meaning	condition
`p`	frequency of the dominant allele A (give one of p, q, q2)	
`q`	frequency of the recessive allele a (give one of p, q, q2)	
`q2`	frequency of the recessive phenotype aa (give one of p, q, q2)	
`N`	number of individuals (optional, for expected counts)	

output	meaning
`p`	frequency of allele A
`q`	frequency of allele a
`p2`	frequency of AA (p^2)
`het_2pq`	frequency of Aa (2pq)
`q2`	frequency of aa (q^2)
`carrier_frequency`	carrier (heterozygote) frequency in the population, 2pq
`carrier_among_unaffected`	probability that an unaffected person is a carrier, $2pq/(1-q^2)$
`dominant_phenotype`	frequency of the dominant phenotype, p^2+2pq
`exp_AA`	expected AA count (N p^2)
`exp_Aa`	expected Aa count (2Npq)
`exp_aa`	expected aa count (N q^2)

- **Method:** $q = 1 - p$ (or sqrt of the recessive phenotype frequency); genotype frequencies p^2, 2pq, q^2
- **Assumes:** exactly one of p, q, q2, strictly between 0 and 1; Hardy-Weinberg equilibrium (random mating, no selection, migration or mutation, large population); complete dominance, one autosomal biallelic locus
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



carrier_frequency (carrier (heterozygote) frequency in the population, 2pq) against q2 (frequency of the recessive phenotype aa (give one of p, q, q2)) from 0.0002 to 0.0006. 31 points computed by computational-biology-ai999x; marked: the worked example, q2 = 0.0004, carrier_frequency = 0.0392.

```
computational_biology_ai999x.solve('bio_hardy_weinberg_frequencies', q2=0.0004)
# -> p = 0.98, q = 0.02, p2 = 0.9604, het_2pq = 0.0392, q2 = 0.0004, carrier_frequency = 0.0392,
carrier_among_unaffected = 0.0392157, dominant_phenotype = 0.9996   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_hardy_weinberg_frequencies', p=0.7, N=1000)
# -> p = 0.7, q = 0.3, p2 = 0.49, het_2pq = 0.42, q2 = 0.09, carrier_frequency = 0.42, carrier_among_unaffected =
0.461538, dominant_phenotype = 0.91, exp_AA = 490, exp_Aa = 420, exp_aa = 90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hardy-Weinberg frequencies: q2 = 0.0004	SOLVED	p = 0.98, q = 0.02, p2 = 0.9604, het_2pq = 0.0392, q2 = 0.0004, carrier_frequency = 0.0392, carrier_among_unaffected = 0.0392157, dominant_phenotype = 0.9996
Hardy-Weinberg frequencies: p = 0.7, N = 1000	SOLVED	p = 0.7, q = 0.3, p2 = 0.49, het_2pq = 0.42, q2 = 0.09, carrier_frequency = 0.42, carrier_among_unaffected = 0.461538, dominant_phenotype = 0.91, exp_AA = 490, exp_Aa = 420, exp_aa = 90
Carrier frequency: q2 = 0.04%	SOLVED	p = 0.98, q = 0.02, p2 = 0.9604, het_2pq = 0.0392, q2 = 0.0004, carrier_frequency = 0.0392, carrier_among_unaffected = 0.0392157, dominant_phenotype = 0.9996
Hardy-Weinberg frequencies: q2 = 1.2	INVALID_INPUT	Hardy-Weinberg genotype and carrier frequencies: q2 must be a fraction between 0 and 1.
Hardy-Weinberg frequencies: p = 0.7, q2 = 0.09	INVALID_INPUT	Hardy-Weinberg genotype and carrier frequencies: give exactly one of p (frequency of A), q (frequency of a) or q2 (frequ

Recombination frequency from two-point testcross counts, `bio_testcross_recombination`

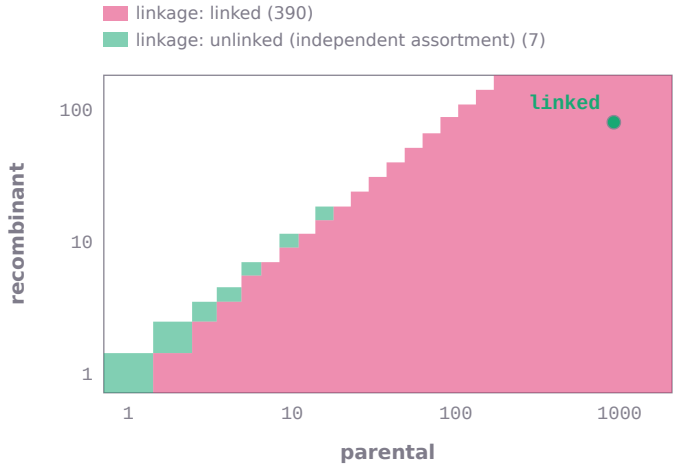
The recombination frequency between two genes from the numbers of parental and recombinant offspring of a testcross, its standard error and the corresponding (uncorrected) map distance.

$$RF = \frac{n_{rec}}{n_{par} + n_{rec}}, \quad d = 100\,RF \text{ cM}$$

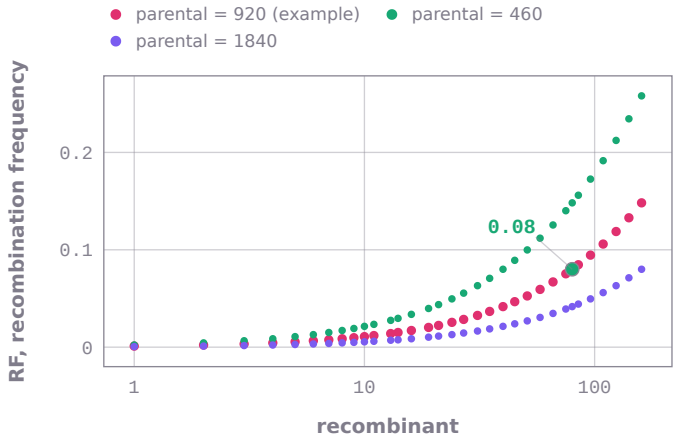
input	meaning	condition
`parental`	number of offspring of the parental classes (both together)	
`recombinant`	number of offspring of the recombinant classes (both together)	

output	meaning
`RF`	recombination frequency
`RF_percent`	recombination frequency in %
`d_cM`	map distance in centimorgans (map units), uncorrected
`SE`	standard error of RF
`linkage`	linked $(RF < 0.5)$ or unlinked

- **Method:** RF = recombinants / total; $d = 100$ RF cM; $SE = sqrt$(RF (1 - RF) / n)
- **Assumes:** counts whole numbers; recombinant <= parental $(RF \leq 0.5)$; a testcross (every gamete of the heterozygote visible)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



linked (RF < 0.5) or unlinked (linkage) over parental (number of offspring of the parental classes (both together)) from 1 to 1840 and recombinant (number of offspring of the recombinant classes (both together)) from 1 to 160 (logarithmic axes): each of the 29 × 20 cells is one run of computational-biology-ai999x, coloured by the verdict it returned, linked: 390 cells, unlinked (independent assortment): 7 cells; blank cells: inputs the capability refuses. Marked: the worked example, parental = 920, recombinant = 80 → linked.



RF (recombination frequency) against recombinant (number of offspring of the recombinant classes (both together)) from 1 to 160 on a logarithmic axis. One curve for each value of parental (number of offspring of the parental classes (both together)): 920, 460, 1840. 33 points per curve computed by computational-biology-ai999x; marked: the worked example, recombinant = 80, RF = 0.08.

```
computational_biology_ai999x.solve('bio_testcross_recombination', parental=920, recombinant=80)
# -> RF = 0.08, RF_percent = 8, d_cM = 8, SE = 0.00857904, linkage = linked   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_testcross_recombination', parental=300, recombinant=200)
# -> RF = 0.4, RF_percent = 40, d_cM = 40, SE = 0.0219089, linkage = linked   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-point testcross: parental = 920, recombinant = 80	SOLVED	RF = 0.08, RF_percent = 8, d_cM = 8, SE = 0.00857904, linkage = linked

question	outcome	result
Two-point testcross: parental = 300, recombinant = 200	SOLVED	RF = 0.4, RF_percent = 40, d_cM = 40, SE = 0.0219089, linkage = linked
Two-point testcross: recombinant = 80	NEEDS_INPUT	bio_testcross_recombination needs parental (number of offspring of the parental classes (both together)). Example: solve
Two-point testcross: parental = 920, recombinant = 80, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Two-point testcross: parental = 920, recombinant = 80 and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Two-point testcross: parental = 100, recombinant = 300	INVALID_INPUT	Recombination frequency from two-point testcross counts: the recombinant (less frequent) classes cannot outnumber the pa

Three-point testcross: gene order, map distances and interference, `bio_three_point_testcross`

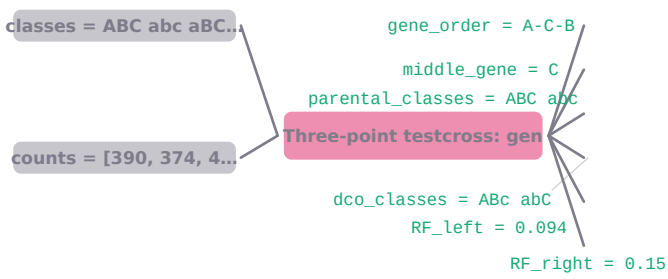
From the eight offspring classes of a three-point testcross: the parental and double-crossover classes, the gene in the middle, the recombination frequency and map distance of each interval (double crossovers counted in both), the coefficient of coincidence and the interference.

$$RF_1 = \frac{n_{SCO,1} + n_{DCO}}{n}, \quad c = \frac{n_{DCO}/n}{RF_1\, RF_2}, \quad I = 1 - c$$

input	meaning	condition
`classes`	the 8 offspring classes, three letters each in one locus order, upper case = dominant allele, separated by spaces	
`counts`	number of offspring in each class, in the same order	

output	meaning
`gene_order`	gene order (outer genes in their written order)
`middle_gene`	the gene in the middle
`parental_classes`	the two parental (most numerous) classes
`dco_classes`	the two double-crossover (least numerous) classes
`RF_left`	recombination frequency between the first outer gene and the middle gene
`RF_right`	recombination frequency between the middle gene and the second outer gene
`d_left_cM`	map distance of the first interval in cM
`d_right_cM`	map distance of the second interval in cM
`d_total_cM`	map distance between the outer genes (sum of the intervals) in cM
`RF_outer_direct`	recombination frequency counted directly between the outer genes (misses double crossovers)
`coincidence`	coefficient of coincidence: observed / expected double crossovers
`interference`	interference 1 - coincidence

- **Method:** parental = two largest classes, double crossovers = two smallest; the middle gene is the one whose alleles the double crossovers swap relative to the parentals; RF of an interval = (its single crossovers + double crossovers)/n; coincidence = observed DCO / (RF1 RF2)
- **Assumes:** 8 distinct classes of the same three genes; parental and double-crossover pairs reciprocal and clearly the largest/smallest; uncorrected map distances (1 % recombination = 1 cM)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Three-point testcross: gene order, map distances and interference as inputs and results: stated classes = ABC abc aBC Abc AbC aBc ABc abC, counts = [390, 374, 45, 41, 72, 70, 5, 3]; returned by computational-biology-ai999x: gene_order = A-C-B, middle_gene = C, parental_classes = ABC abc, dco_classes = ABc abC, RF_left = 0.094, RF_right = 0.15.

```
computational_biology_ai999x.solve('bio_three_point_testcross', classes='ABC abc aBC Abc AbC aBc ABc abC', counts=[390, 374, 45, 41, 72, 70, 5, 3])
# -> gene_order = A-C-B, middle_gene = C, parental_classes = ABC abc, dco_classes = ABc abC, RF_left = 0.094, RF_right = 0.15, d_left_cM = 9.4, d_right_cM = 15, d_total_cM = 24.4, RF_outer_direct = 0.228, coincidence = 0.567376, interference = 0.432624   (SOLVED)
```

```
computational_biology_ai999x.solve('bio_three_point_testcross', classes='DEf deF dEf DeF DEF def Def dEF', counts=[420, 409, 60, 52, 30, 26, 2, 1])
# -> gene_order = D-E-F, middle_gene = E, parental_classes = DEf deF, dco_classes = Def dEF, RF_left = 0.115, RF_right = 0.059, d_left_cM = 11.5, d_right_cM = 5.9, d_total_cM = 17.4, RF_outer_direct = 0.168, coincidence = 0.442152, interference = 0.557848   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Three-point testcross: classes = ABC abc aBC Abc AbC aBc ABc abC, counts = [390, 374, 45, 41, 72, 70, 5, 3]	SOLVED	gene_order = A-C-B, middle_gene = C, parental_classes = ABC abc, dco_classes = ABc abC, RF_left = 0.094, RF_right = 0.15, d_left_cM = 9.4, d_right_cM = 15, d_total_cM = 24.4, RF_outer_direct = 0.228, coincidence = 0.567376, interference = 0.432624
Three-point testcross: classes = DEf deF dEf DeF DEF def Def dEF, counts = [420, 409, 60, 52, 30, 26, 2, 1]	SOLVED	gene_order = D-E-F, middle_gene = E, parental_classes = DEf deF, dco_classes = Def dEF, RF_left = 0.115, RF_right = 0.059, d_left_cM = 11.5, d_right_cM = 5.9, d_total_cM = 17.4, RF_outer_direct = 0.168, coincidence = 0.442152, interference = 0.557848
Three-point testcross: counts = [390, 374, 45, 41, 72, 70, 5, 3]	NEEDS_INPUT	bio_three_point_testcross needs classes (the 8 offspring classes, three letters each in one locus order, upper case = do
Three-point testcross: classes = ABC abc aBC Abc AbC aBc ABc abC, counts = [390, 374, 45, 41, 72, 70, 5, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Three-point testcross: classes = ABC abc aBC Abc AbC aBc ABc abC, counts = [390, 374, 45, 41, 72, 70, 5, 3] and 7 more	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: computational_biology_ai999x.temp
Three-point testcross: classes = ABC abc aBC Abc AbC aBc ABc abC, counts = [390, 45, 374, 41, 72, 70, 5, 3]	INVALID_INPUT	Three-point testcross: gene order, map distances and interference: the two most numerous classes must be reciprocal (the
Three-point testcross: classes = ABC abc aBC Abc AbC aBc ABc ABc, counts = [390, 374, 45, 41, 72, 70, 5, 3]	INVALID_INPUT	Three-point testcross: gene order, map distances and interference: each of the 8 classes must be given once.